



**UNIVERSIDAD
DE ANTIOQUIA**

1 8 0 3

Documentación Plataforma OpenFMB

Jhon Eduar Garcia

Daniel Ospina Acero
Jaime Alberto Vergara Tejada
Supervisores

Sergio Armando Gutiérrez Betancur
Coordinator
saguti@ieee.org

23 de julio de 2026

Índice

1. Proyecto de referencia de OpenFMB.	4
1.1. Instalar Docker y Docker Compose.	4
1.2. Descargar los archivos del proyecto de referencia de OpenFMB.	4
1.3. Ejecutar proyecto de referencia.	5
1.4. Visualizar diagramas en la herramienta HMI.	6
1.5. Detener la ejecución de OpenFMB.	8
2. Conexión del Medidor Modbus A9MEM3155 SCHNEIDER ELECTRIC con OpenFMB.	8
2.1. Conexión física del medidor A9MEM3155 con adaptador RS485 a Ethernet.	8
2.2. Configuración del medidor Modbus A9MEM3155.	8
2.3. Configuración del adaptador RS485 a Ethernet.	8
2.4. Estructura del proyecto de OpenFMB para leer el medidor A9MEM3155.	10
2.5. Obtener herramienta <i>OpenFMB Adapter Configuration Tool</i> (OACT).	11
2.6. Creación de ficheros de configuración con Herramienta (OACT).	11
2.7. Fichero del adaptador OpenFMB.	16
2.8. Fichero del plugin modbus-master del medidor Modbus A9MEM3155 a OpenFMB.	17
2.9. Configuración de la interfaz gráfica HMI de OpenFMB.	20
2.10. Fichero docker-compose.yml para ejecutar plataforma OpenFMB.	21
2.11. Ejecución de la plataforma OpenFMB con medidor A9MEM3155.	22
2.12. Visualización de diagramas interactivos con el HMI de OpenFMB.	23
2.13. Conexión de carga monofásica al medidor A9MEM3155.	27
2.14. Ajustes en OpenFMB para la lectura de nuevas variables.	27
3. Aplicación de factores de escala sobre los datos.	29
3.1. Escalamiento de datos desde el plugin modbus-master del adaptador de OpenFMB.	29
3.2. Escalamiento de datos desde la herramienta HMI de OpenFMB.	31
4. Acceso remoto a la interfaz HMI de OpenFMB.	32
4.1. Conexión en red de Ubuntu Server y el anfitrión Windows.	32
4.2. Validar la disponibilidad de docker y docker compose en Ubuntu Sever.	34
4.3. Obtener los archivos del proyecto.	34
4.4. Ejecutar la interfaz humano máquina (HMI) desde el anfitrión.	38
4.5. Ejecución de contenedor HMI desde host Windows.	39
5. Medidores A9MEM3155 en cascada con OpenFMB.	41
5.1. Conexión física de medidores en cascada a un mismo adaptador RS485 a Ethernet.	41
5.2. Configurar multi-esclavo en adaptador RS485 a Ethernet.	42
5.3. ‘Templating’ en OpenFMB para escalar el número de medidores A9MEM3155.	42
5.4. Ejecución de la plataforma y visualización de los 2 medidores en HMI de OpenFMB.	44
6. Escritura en registros de emulador Modbus Pal con OpenFMB.	45
6.1. Direccionamiento IP para máquina virtual linux y host Windows.	46
6.2. Obtener emulador Modbus Pal y creación de escalvos simulados.	46
6.3. Ficheros de configuración con OACT para escritura con OpenFMB.	49
6.4. Modificación de fichero del plugin modbus-outstation	56
6.5. Modificación del fichero del plugin modbus-master	57
6.6. Fichero docker-compose.yml para escritura en Modbus Pal con OpenFMB.	57
6.7. Escritura de registros Modbus con Python.	58
6.8. Ejecución de pruebas de escritura en Modbus Pal con OpenFMB.	59
7. Control ON/OFF con OpenFMB + Raspberry PI 4 MODEL B.	60
7.1. Escritura de coils con Python.	60
7.2. Configurar de Raspberry PI.	61
7.3. Crear ficheros de configuración de OpenFMB para control ON/OFF.	62
7.4. ejecución del entorno de prueba OpenFMB + Raspberry PI 4.	65

8. Adaptador OpenFMB como puente entre protocolos NATS y MQTT.	68
8.1. Configuración del servicio de mensajes NATS.	68
8.2. Configuración del servicio de mensajes MQTT.	69
8.3. Python para implementar publicadores y suscriptores en redes NATS y MQTT.	70
8.4. Configuración del Adaptador OpenFMB.	75
8.5. Ejecución del entorno de pruebas completo <i>broker</i> NATS + OpenFMB + <i>broker</i> MQTT.	76
9. Conexión de OpenFMB con base de datos.	78
9.1. Ejecutar de base de datos timescaledb con Docker Compose.	78
9.2. Estructura de la tabla SQL empleada para almacenar información de OpenFMB.	79
9.3. Configuraciones internas en el contenedor timescaledb.	79
9.4. Configuraciones del adaptador OpenFMB para conexión con base de datos timescaledb.	79
10. Visualización de información de base de datos con Grafana.	80
10.1. Configuración de Grafana con Docker Compose.	80
10.2. Acceso a la interfaz web de Grafana.	80
10.3. Agregar la conexión de Grafana con timescaledb.	81
10.4. Creación de nuevo dashboard en Grafana.	82
10.5. Selección de columnas de la base de datos a mostrar en Dashboard.	83
11. Escritura de dispositivos vinculados a OpenFMB con interfaz gráfica HMI.	84
11.1. Escritura de registros Modbus con HMI de OpenFMB.	84
11.2. Control de coils Modbus con HMI de OpenFMB.	91
12. Identificadores ‘mRID’ dentro OpenFMB.	97

1. Proyecto de referencia de OpenFMB.

OpenFMB provee un proyecto de demostración con el que se puede observar el funcionamiento básico de los componentes de esta plataforma. En esta instancia de prueba se replica la información de un archivo de texto que contiene información de una topología real compuesta por equipos de generación, almacenamiento y consumo de energía.

En esta sección se muestra el paso a paso, desde la ejecución del proyecto de referencia de OpenFMB hasta la visualización del diagrama de demostración de la plataforma, usando la herramienta *Human Machine Interface* (HMI).

1.1. Instalar Docker y Docker Compose.

Docker Compose es una herramienta de software que, en el contexto de OpenFMB, sirve para definir y gestionar eficientemente múltiples aplicativos que forman parte de la plataforma. Entre estos están: adaptadores, interfaces HMI y otros servicios necesarios para la gestión de comunicaciones en sistemas como microrredes.

La instalación de Docker es un prerequisite para instalar Docker Compose, en el siguiente enlace se redirige a su sitio web donde se encuentra el paso a paso para obtener Docker y Docker Compose en Ubuntu, se puede navegar en este sitio para instalar este software en otras distribuciones de Linux o en Windows.

- <https://docs.docker.com/engine/install/ubuntu/>

Para verificar que la instalación se hizo correctamente, se pueden ejecutar algunos comandos en la terminal de Linux para validar la versión instalada de Docker y Docker Compose, también el estado de ejecución del servicio Docker.

- `sudo docker --version`
- `sudo docker-compose --version`
- `sudo systemctl status docker`

Respectivamente, estos comandos permiten conocer la versión de Docker, Docker Compose y el estado del servicio Docker (Activo/Inactivo).

1.2. Descargar los archivos del proyecto de referencia de OpenFMB.

Los archivos necesarios para la ejecución de esta demostración se encuentran en el repositorio de Open Energy Solutions, estos se pueden descargar desde [la página de OpenFMB](#) o haciendo clic [aquí](#).

Luego de descomprimir el archivo descargado, se obtiene el proyecto con las carpetas hmi, replay y el fichero docker-compose.yml. La primera carpeta contiene información de los paneles de la interfaz gráfica HMI, la segunda contiene la configuración del adaptador OpenFMB, que es el núcleo de software de la plataforma y, por último, el fichero de configuración docker-compose.yml, descrito con sintaxis tipo YAML, define los servicios NATS, replay (instancia del adaptador OpenFMB) y HMI, con sus respectivas imágenes, además de otras configuraciones para la ejecución del proyecto de referencia de OpenFMB. En secciones posteriores se aborda el uso y configuración de estos servicios, ficheros de configuración y directorios de persistencia de datos, etc.

Para manipular este conjunto de archivos de manera ordenada, se puede usar un editor de código, en este caso Visual Studio Code. La estructura de archivos descrita se visualiza en la Figura 1.

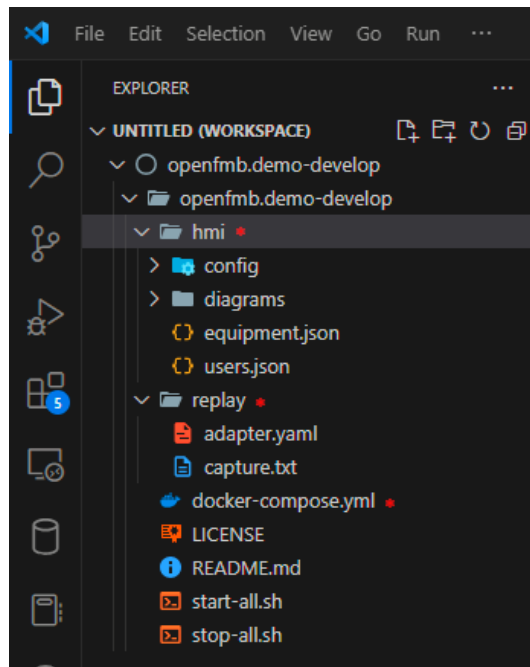


Figura 1: Ficheros del proyecto de referencia de OpenFMB.

1.3. Ejecutar proyecto de referencia.

Sin editar ninguno de los documentos descargados previamente, se abre una terminal de comandos en la carpeta **openfmb.demo-develop** donde está el fichero **docker-compose.yml** y los otros directorios.

Se verifica la correcta instalación de los prerequisites de software Docker y Docker Compose por medio de los comandos explicados en la **Sección 1.1**, esto se muestra en la Figura 2.

```
# Usuario @ DESKTOP-4EC3332 in ~\Desktop\openfmb.demo-develop\openfmb.demo-develop [19:03:50]
$ docker --version
Docker version 26.1.4, build 5650f9b
# Usuario @ DESKTOP-4EC3332 in ~\Desktop\openfmb.demo-develop\openfmb.demo-develop [14:31:42]
$ docker-compose --version
Docker Compose version v2.27.1-desktop.1
# Usuario @ DESKTOP-4EC3332 in ~\Desktop\openfmb.demo-develop\openfmb.demo-develop [14:31:48]
$
```

Figura 2: Comandos para verificar la instalación de Docker y Docker Compose.

Posteriormente, se pone en ejecución la plataforma de OpenFMB según las configuraciones del fichero **docker-compose.yml**, esta ejecución se inicia por medio del siguiente comando.

- `docker-compose up`

Si el inicio fue exitoso, se puede ver mensajes del estado de los aplicativos que componen la plataforma OpenFMB: NATS, HMI y replay (adaptador OpenFMB).

```
nats-1 | [1] 2024/12/21 19:47:47.858164 [INF] Server is ready
nats-1 | [1] 2024/12/21 19:47:47.858335 [INF] Cluster name is my_cluster
nats-1 | [1] 2024/12/21 19:47:47.858409 [INF] Listening for route connections on 0.0.0.0:6222
hmi-1 | 2024-12-21 19:47:48[hmi_server::logs][INFO] ACTOR CREATED /user/SysEventLogger
hmi-1 | 2024-12-21 19:47:48[hmi_server::hmi:hmi][DEBUG] Received start processing message: StartProcessingMessages { pubsub_op
ns { connection_url: "nats:4222", authentication_type: None, env: Dev, token: None, creds_file: None, username: None, password: None
root_cert: None, client_cert: None, client_key: None } }
hmi-1 | 2024-12-21 19:47:48[hmi_server::hmi:hmi_subscriber][INFO] HmiSubscriber connects to NATS with options: CoordinatorOpti
nats:4222", authentication_type: None, env: Dev, token: None, creds_file: None, username: None, password: None, security_type: None,
t_cert: None, client_key: None }
hmi-1 | 2024-12-21 19:47:48[hmi_server::hmi:hmi_publisher][INFO] HmiPublisher connects to NATS with options: CoordinatorOpti
ts:4222", authentication_type: None, env: Dev, token: None, creds_file: None, username: None, password: None, security_type: None, r
cert: None, client_key: None }
hmi-1 | 2024-12-21 19:47:48[hmi_server::hmi::coordinator][INFO] NATS with default option
hmi-1 | 2024-12-21 19:47:48[hmi_server::hmi::coordinator][INFO] NATS with default option
replay-1 | 2024-12-21 19:47:48.485 [application] [info] No configuration specified for adapter: capture
hmi-1 | 2024-12-21 19:47:48[warp::server][INFO] Server::run; addr=0.0.0.0:32771
hmi-1 | 2024-12-21 19:47:48[warp::server][INFO] listening on http://0.0.0.0:32771
replay-1 | 2024-12-21 19:47:48.487 [application] [info] No configuration specified for adapter: dnp3-master
replay-1 | 2024-12-21 19:47:48.489 [application] [info] No configuration specified for adapter: dnp3-outstation
replay-1 | 2024-12-21 19:47:48.489 [application] [info] No configuration specified for adapter: log
replay-1 | 2024-12-21 19:47:48.490 [application] [info] No configuration specified for adapter: modbus-master
replay-1 | 2024-12-21 19:47:48.491 [application] [info] No configuration specified for adapter: modbus-outstation
replay-1 | 2024-12-21 19:47:48.491 [application] [info] No configuration specified for adapter: mqtt
replay-1 | 2024-12-21 19:47:48.491 [application] [info] Initializing plugin: nats
hmi-1 | 2024-12-21 19:47:48[rustls::anchors][DEBUG] add_pem_file processed 141 valid and 0 invalid certs
hmi-1 | 2024-12-21 19:47:48[rustls::anchors][DEBUG] add_pem_file processed 141 valid and 0 invalid certs
replay-1 | 2024-12-21 19:47:48.513 [nats] [info] configured NATS publisher for subject: openfmb.essmodule.ESSReadingProfile.*
replay-1 | 2024-12-21 19:47:48.513 [nats] [info] configured NATS publisher for subject: openfmb.essmodule.ESSStatusProfile.*
hmi-1 | 2024-12-21 19:47:48[hmi_server::hmi:hmi_publisher][INFO] ***** HmiPublisher successfully connected
hmi-1 | 2024-12-21 19:47:48[hmi_server::hmi:hmi_subscriber][INFO] ***** HmiSubscriber successfully connected
```

Figura 3: Ejecución de proyecto de referencia de OpenFMB.

La salida de la consola informa que el sistema de mensajes NATS se inició correctamente, luego se muestra que el adaptador y la interfaz HMI se conectaron a este bus de mensajes.

1.4. Visualizar diagramas en la herramienta HMI.

Como se mencionó antes, la interfaz HMI es uno de los aplicativos proporcionados por Open Energy Solutions y, cuando está en ejecución, cuenta con una interfaz web accesible con un puerto 32771 por defecto. Con la plataforma en marcha, se puede acceder a esta interfaz utilizando cualquier navegador y escribiendo la dirección: 'localhost:32771/'. Esto debe proporcionar una vista como la de la Figura 4.

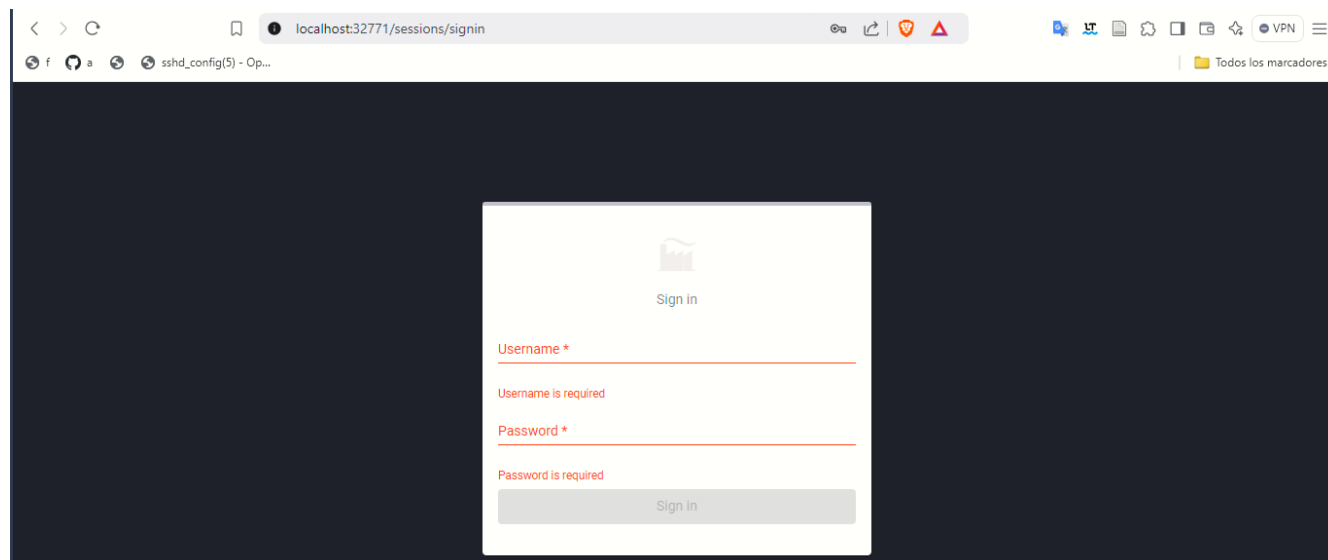


Figura 4: Inicio de sesión en interfaz HMI.

Esta interfaz permite autenticarse con diferentes roles. El rol 'Admin' permite tanto crear como editar usuarios y diagramas dentro del aplicativo, es el máximo nivel de permiso; luego está 'Engineer' que puede crear diagramas, pero no tiene control sobre la creación de usuarios y dispositivos nuevos, finalmente está 'Viewer' que únicamente puede visualizar los diagramas en ejecución.

Por defecto se puede ingresar con el usuario ‘admin’ y la contraseña ‘hm1admin’. Luego de ingresar con este usuario al HMI, se pasa a la opción ‘DIAGRAMS’ del menú de la izquierda y se da clic en la opción ‘Run’ para ejecutar el diagrama. Esta secuencia de pasos se ilustra en las figuras 5 y 6.

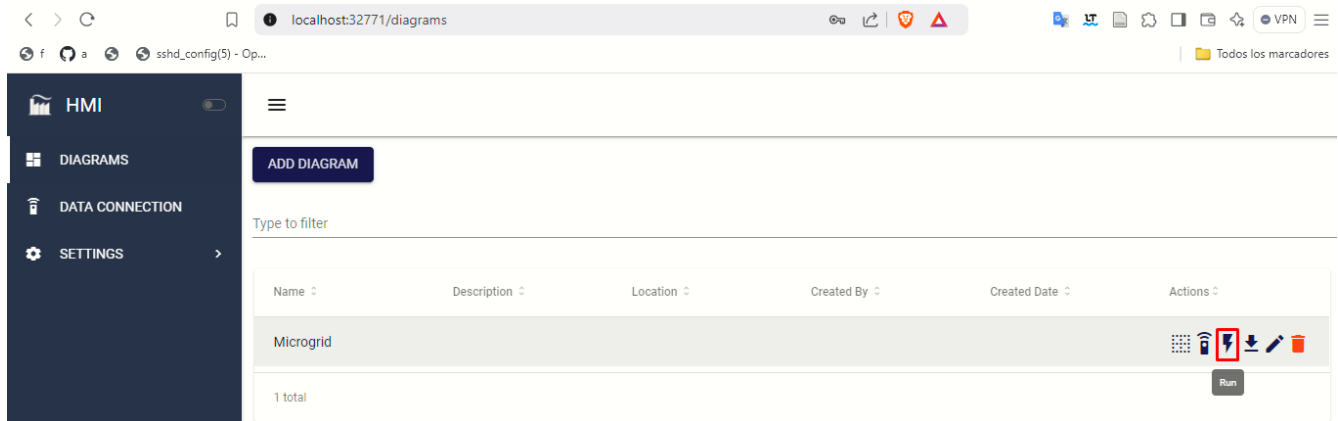


Figura 5: Vista de diagramas en HMI.

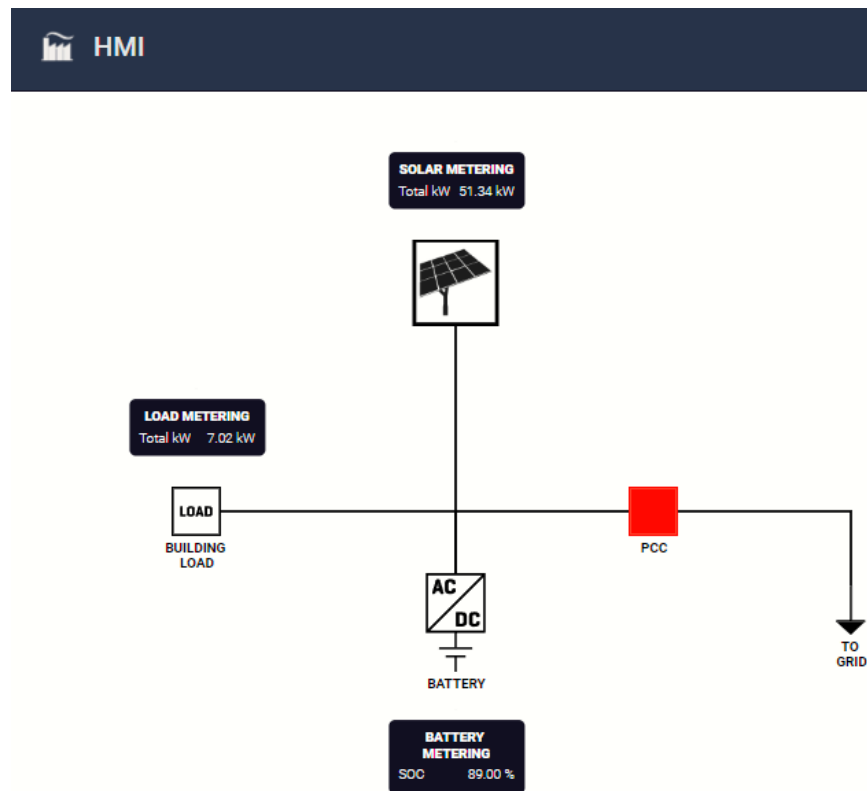


Figura 6: Ejecución de diagrama del proyecto de referencia de OpenFMB.

En este diagrama se observa que la topología del proyecto de referencia tiene un panel solar, un inversor AC/DC, la carga de un edificio, una batería y un punto de acople común con la red principal. Junto a cada componente, se tienen recuadros que muestran información de cada elemento según su tipo. En esta demostración, las mediciones provienen del fichero de texto que el adaptador reenvía a la interfaz HMI. Estos y otros elementos de la interfaz gráfica se pueden usar para crear paneles de control propios, esto se profundiza en secciones posteriores.

1.5. Detener la ejecución de OpenFMB.

Para detener la ejecución de OpenFMB, se puede hacer manualmente por medio del siguiente comando en Linux o Windows o deteniendo la ejecución por la fuerza con la combinación de teclas **ctrl + c**.

- docker-compose down

2. Conexión del Medidor Modbus A9MEM3155 SCHNEIDER ELECTRIC con OpenFMB.

Los medidores Modbus A9MEM3155 utilizan una interfaz serial RS-485 para la transmisión de sus datos mediante el protocolo Modbus RTU, sin embargo, dado que la topología principal de la red es IP sobre LAN Ethernet, se emplea un adaptador que actúa como puente entre ambas tecnologías.

En esta sección se presentan los pasos para conectar uno de estos medidores a la plataforma OpenFMB, desde la conexión física de este equipo con el adaptador RS485 a Ethernet hasta la lectura de sus mediciones con la interfaz gráfica HMI de la plataforma OpenFMB.

2.1. Conexión física del medidor A9MEM3155 con adaptador RS485 a Ethernet.

Para conectar el medidor con el adaptador RS485 a Ethernet se conectan sus terminales A+, B- y GND a los bornes D1, D0 y 0V del medidor respectivamente, como se muestra en la Figura 7.

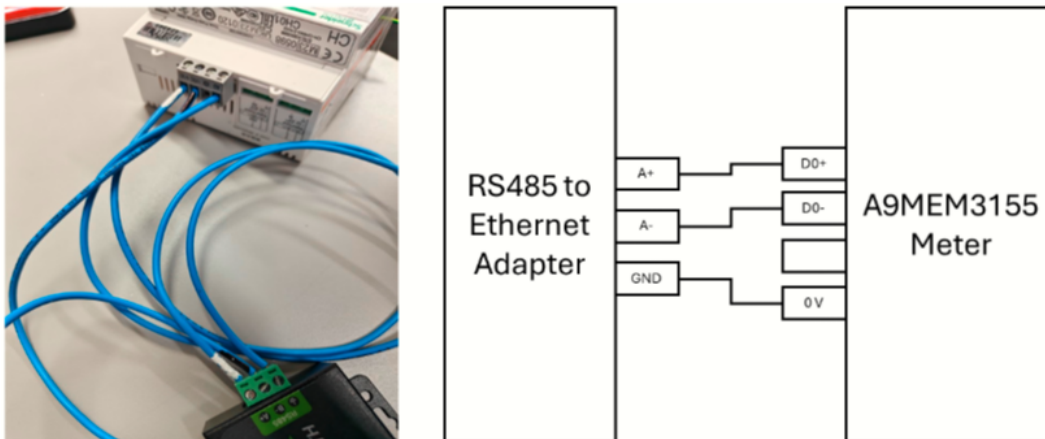


Figura 7: Conexión física entre Medidor A9MEM315 y adaptador RS485 a Ethernet.

2.2. Configuración del medidor Modbus A9MEM3155.

El medidor A9MEM3155 de Schneider Electric se ajusta según el sistema donde se va a instalar. La paridad debe configurarse en 'NONE', la frecuencia de operación en '60 Hz', y el número de esclavo debe ajustarse según el número de medidores conectados en cascada.

Para acceder al modo de configuración del medidor, se deben presionar simultáneamente los botones 'OK' y 'ESC'. Luego, se pueden modificar los valores de configuración utilizando las flechas y el botón 'OK'. La contraseña por defecto para ingresar a este modo es '0010', que puede ser ingresada con los botones de flecha y 'OK'.

2.3. Configuración del adaptador RS485 a Ethernet.

Este adaptador tiene una interfaz web para su configuración, que es accesible por medio de cualquier navegador escribiendo su dirección IP, que por defecto es 192.168.0.7 (esta misma IP se puede usar para verificar la conectividad con el adaptador con el comando 'ping + IP'). Al conectarse por primera vez se debe escribir el usuario y contraseña que por defecto es 'admin' en ambos casos, luego se debe observar la interfaz mostrada en la Figura 8.

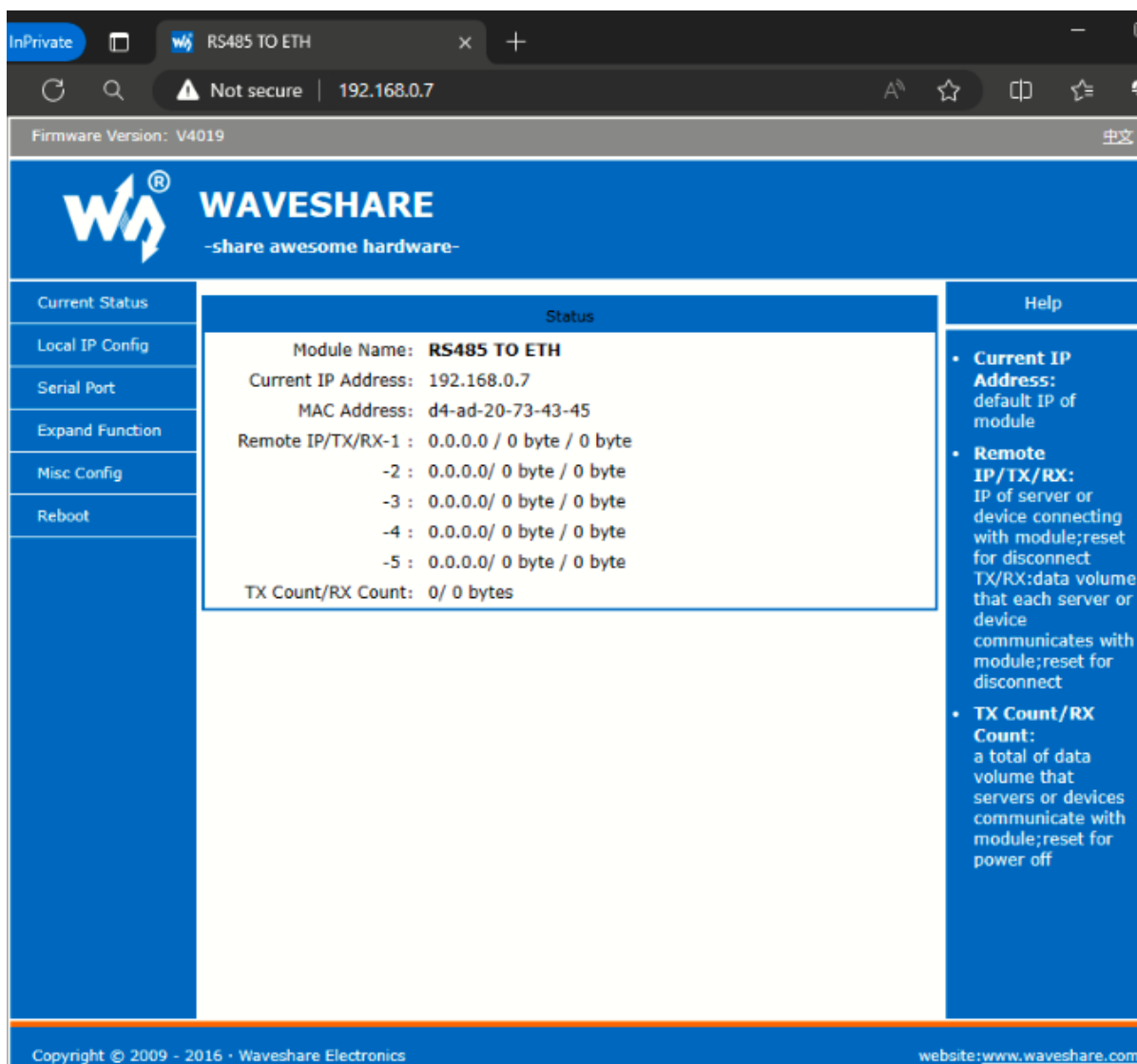


Figura 8: Interfaz web del adaptador RS485 a Ethernet.

Luego de tener acceso a la interfaz web se pasa a configurar la comunicación serial del adaptador con el medidor A9MEM3155, para esto se da clic en la opción 'Serial Port' en el menú de la izquierda mostrado en la Figura 8, luego se establecen los parámetros tal como se observa en la Figura 9.

The screenshot shows a web browser window with the address bar displaying '192.168.0.7'. The page title is 'RS485 TO ETH'. The interface is for a Waveshare module, with the firmware version 'V4019' and a language selector for '中文'. The main content area is titled 'parameter' and contains the following configuration options:

- Baud Rate: 19200 bps
- Data Size: 8 bit
- Parity: None
- Stop Bits: 1 bit
- Local Port Number: 0 (0~65535)
- Remote Port Number: 8234 (1~65535)
- Work Mode: TCP Client
- Remote Server Addr: 192.168.0.201 [192.168.0.201]
- RESET: ☐
- LINK: ☒
- INDEX: ☐
- Similar RFC2217: ☒

At the bottom of the configuration area are 'Save' and 'Cancel' buttons. On the right side, there is a 'Help' section with two bullet points:

- **HTTPD URL:** Module add GET/POST and HTTP/1.1 in URL automatically according to user's setting.
- **HTTPD Packet Header:** Module add HOST automatically according to user's setting. Add "Content Length" automatically in POST mode.

The footer of the page includes 'Copyright © 2009 - 2016 · Waveshare Electronics' and the website 'www.waveshare.com'.

Figura 9: Configuración de comunicación serial del adaptador RS485 a Ethernet con el medidor A9MEM3155.

2.4. Estructura del proyecto de OpenFMB para leer el medidor A9MEM3155.

Luego de realizar esta configuración de la comunicación serial entre el adaptador y el medidor, se puede proceder con la creación de carpetas y ficheros para una nueva aplicación de OpenFMB que permita la lectura de datos de este dispositivo.

En esta sección se recomienda seguir algunos pasos para definir una estructura de manera cercana a la que se muestra en la Figura 1, incluyendo la creación de archivos de configuración de cada componente de la plataforma. Lo primero es crear una carpeta raíz para todos los archivos de la nueva aplicación OpenFMB, dentro ella se generan 3 nuevos directorios vacíos, uno para adaptador de OpenFMB, para la interfaz HMI, y para el plugin **modbus-master**. Luego se crea el fichero denominado docker-compose.yml dentro de la carpeta raíz del proyecto. El resultado de seguir estos pasos se muestra en la Figura 10.

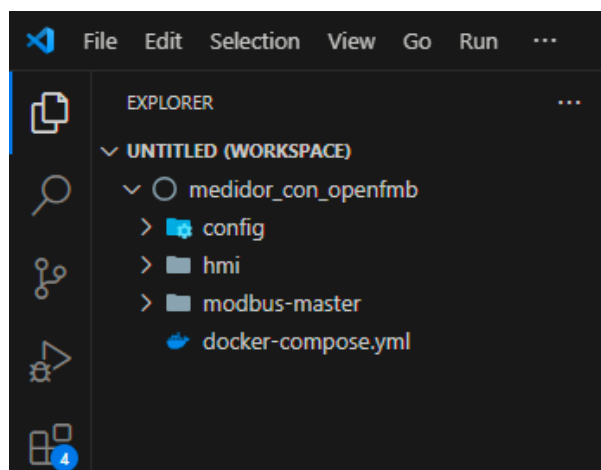


Figura 10: Estructura de ficheros recomendada para una implementación con OpenFMB.

2.5. Obtener herramienta *OpenFMB Adapter Configuration Tool* (OACT).

.....

Se procede con la generación de archivos de configuración del adaptador y el plugin **modbus-master**, para esto se puede emplear la herramienta OACT que se puede descargar desde la [documentación de OpenFMB](#) o directamente desde el repositorio de Open Energy Solutions dando clic [aquí](#). Dentro de la documentación de OpenFMB se encuentran las instrucciones para instalar esta herramienta. Luego de completar este proceso y ejecutar la herramienta OACT se debe obtener una vista similar a la Figura 11.

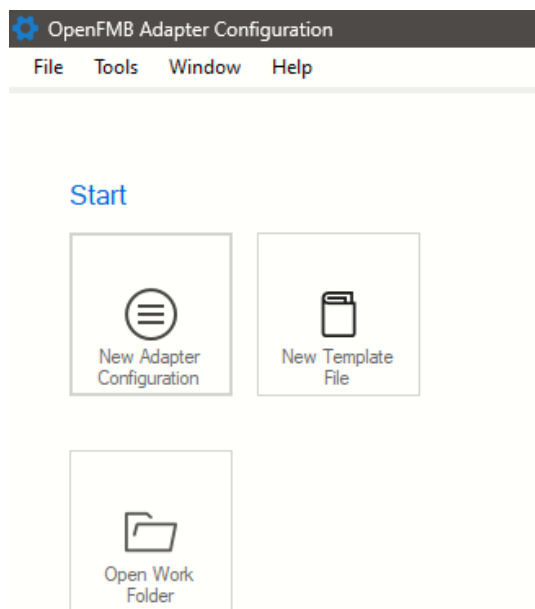


Figura 11: Inicio de herramienta OACT para crear ficheros de configuración de OpenFMB.

2.6. Creación de ficheros de configuración con Herramienta (OACT).

La herramienta OACT facilita la creación de archivos de configuración para el adaptador OpenFMB, así como para sus plugins **modbus-master** y **nats**. Estos plugins permiten al adaptador extraer información de dispositivos Modbus y traducirla al protocolo de mensajería NATS.

Primero se da clic en la opción 'New Adapter Configuration' de la Figura 11, entonces se muestra una ventana emergente donde se selecciona el directorio que va a contener los ficheros del adaptador y sus plugins, sin hacer más

cambios se da clic en 'OK'. Esta ventana emergente debe ser similar a la Figura 12.

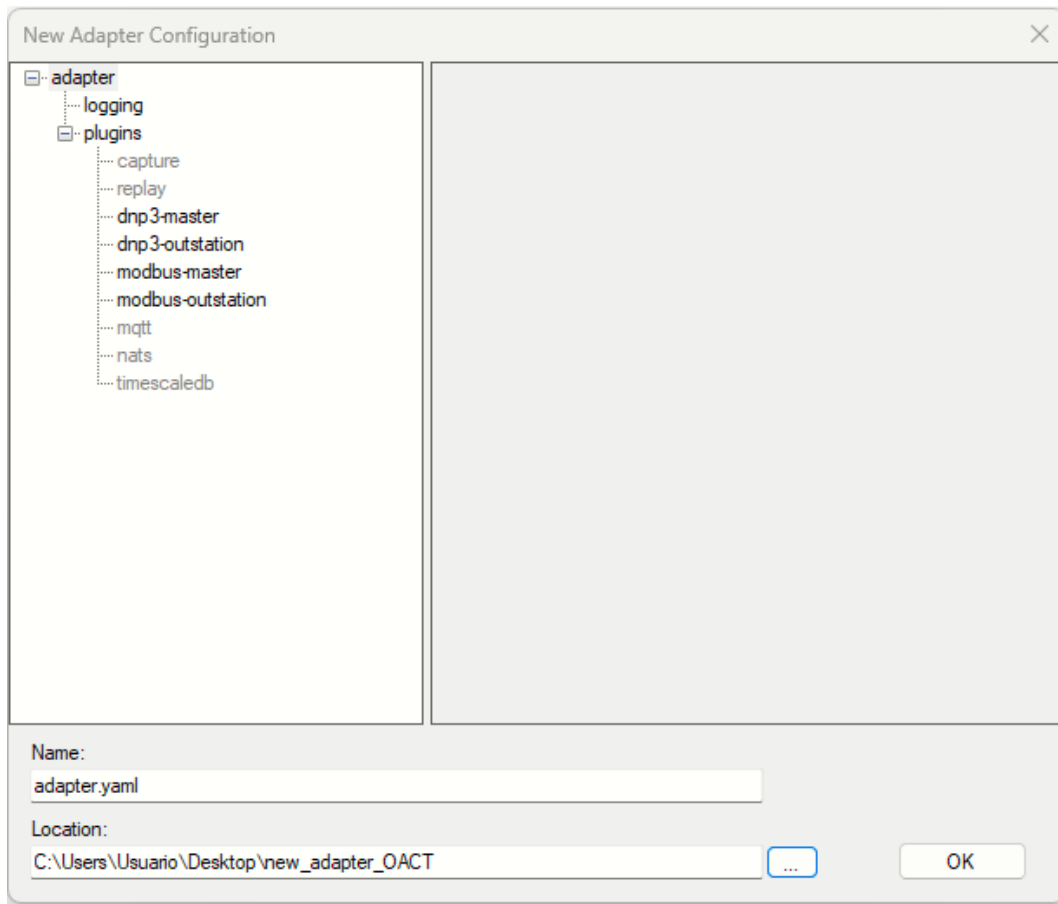


Figura 12: Creación del fichero del adaptador OpenFMB con herramienta OACT.

Ahora, en la herramienta OACT se pueden observar los elementos configurables del adaptador en el menú de la izquierda, sobre estas opciones se da clic derecho sobre el plugin **modbus-master** y luego en 'Add Session...', entonces se muestra una ventana emergente donde se da clic en 'OK'. finalmente, se debe marcar la opción 'Enabled'. Este paso se muestra en la Figura 13.

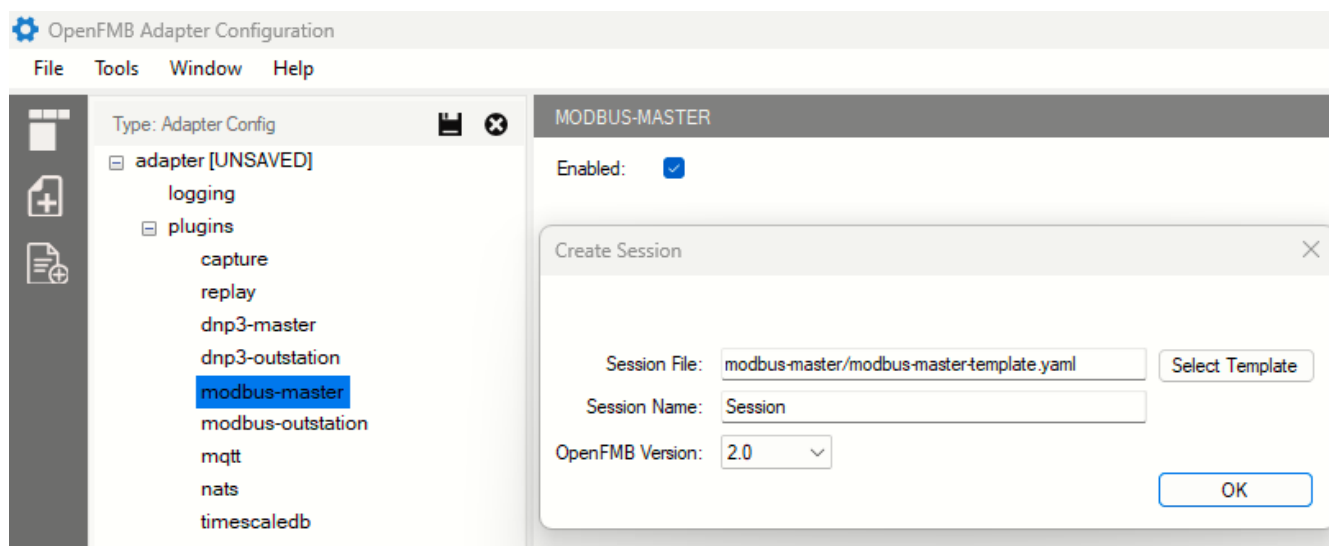


Figura 13: Creación de una sesión desde OACT para vincular dispositivos Modbus a la plataforma OpenFMB.

Luego de crear una Sesión dentro del plugin **modbus-master**, se pueden agregar diferentes dispositivos Modbus, cada uno con diferentes configuraciones. En esta sección se vincula un único medidor a la plataforma OpenFMB.

La vinculación de un dispositivo requiere de una estructura de datos llamada perfil, que mapea la información de dicho equipo para integrarla al flujo de datos de la plataforma OpenFMB. Para crear un nuevo perfil en el plugin **modbus-master** se da clic derecho en la opción 'Session' y luego en la opción 'Add profile...', entonces aparece una ventana emergente con el listado de perfiles existentes, se tienen diferentes tipos, pero se selecciona 'MeterReadingProfile' que es para vincular un dispositivo de solo lectura, finalmente, se da clic en 'OK'. En secciones posteriores se explica el uso perfiles para modificar el estado de un dispositivo.

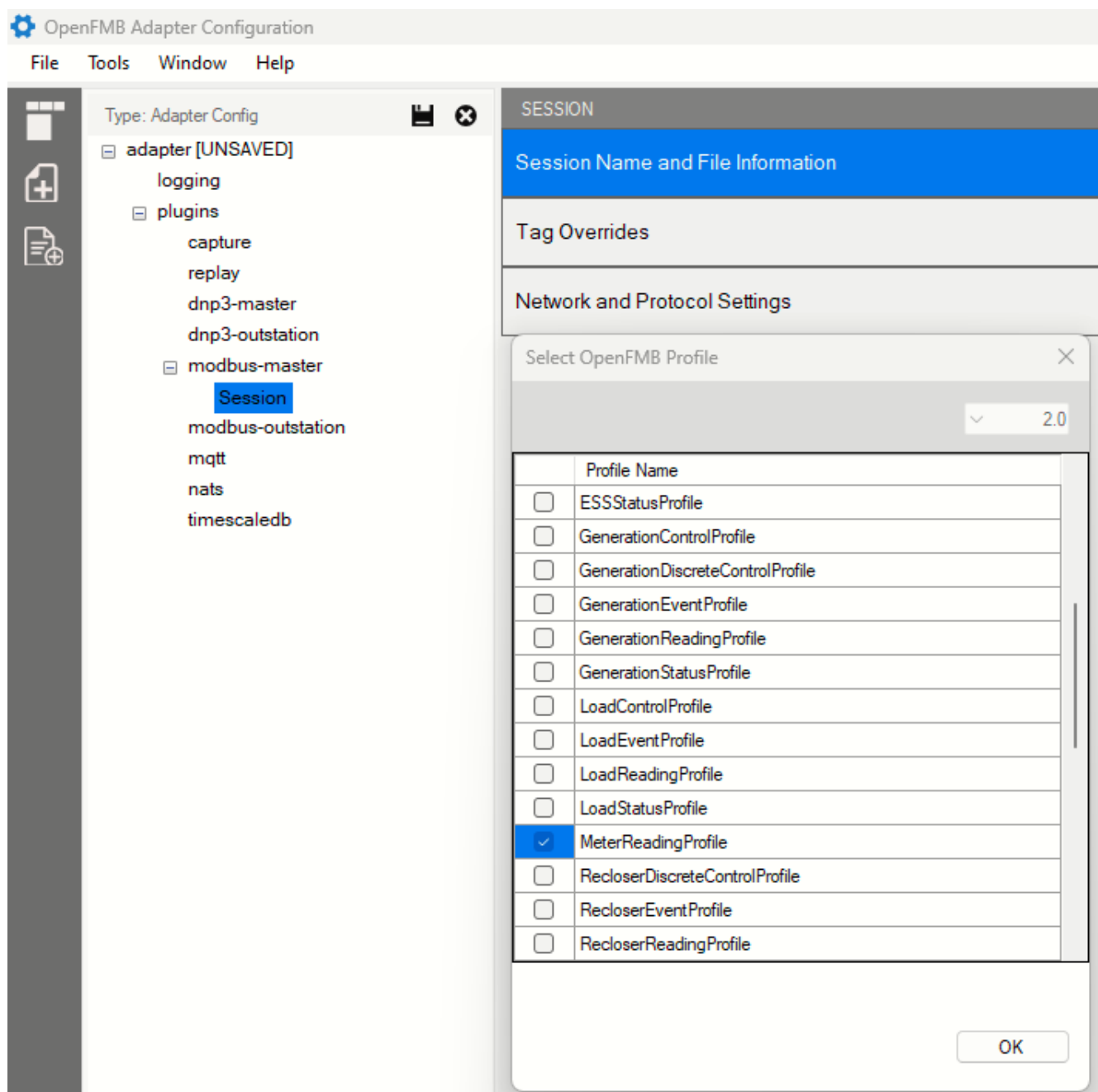


Figura 14: Agregando un perfil **MeterReadingProfile** al adaptador OpenFMB desde OACT.

Hasta este punto se tienen las configuraciones básicas para el adaptador OpenFMB y el plugin **modbus-master** que habilita la lectura de un dispositivo Modbus, sin embargo, resta habilitar y configurar el plugin **nats**, que permite establecer el protocolo de mensajes, usado para unificar el formato de las comunicaciones entre los equipos vinculados a la plataforma OpenFMB.

Para habilitar este plugin se selecciona la opción 'Enabled' dentro del plugin **nats**, luego se modifica la opción 'Connection URL' por 'nats://nats:4222' y, en 'Security' se establece el campo 'Type' en 'none'.

Por último, se da clic en el icono de guardar en la parte superior del menú izquierdo donde se encuentra la estructura del adaptador OpenFMB. Se debe llegar a una visualización similar a la mostrada en la Figura 15.

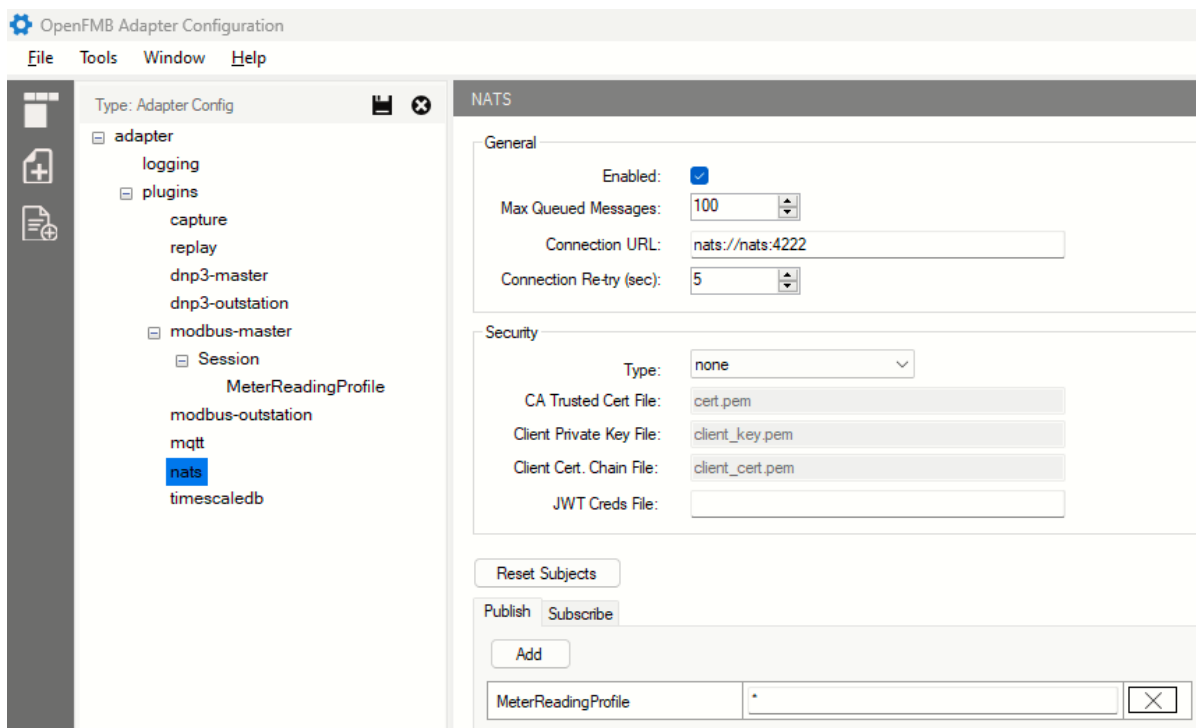


Figura 15: Habilitación y configuración de plugin **nats** desde OACT.

Luego de seguir estos pasos, el resultado son 2 ficheros en el directorio que se seleccionó en el paso de la Figura 12, estos son: 'adapter.yaml' y 'modbus-master-template.yaml', que respectivamente contienen la estructura de plugins del adaptador OpenFMB y la configuración del plugin **modbus-master** para mapear la información de un medidor Modbus a la plataforma. Esta estructura de archivos debe ser similar a la ilustrada en la Figura 16.

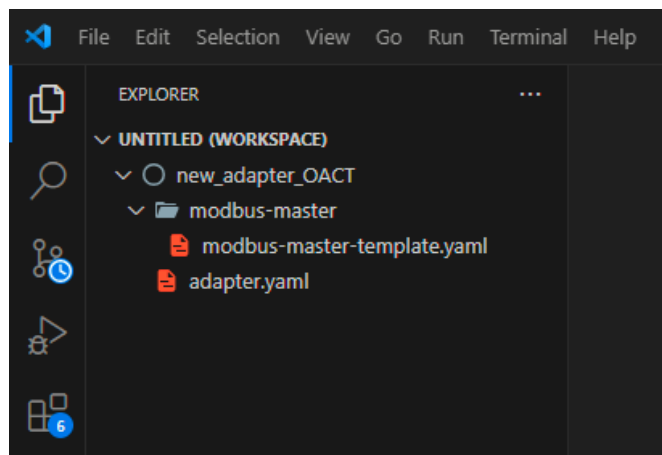


Figura 16: Estructura de ficheros de configuración generados con la herramienta OACT.

El siguiente paso es copiar los ficheros 'adapter.yaml' y 'modbus-master-template.yaml' respectivamente a los directorios 'config' y 'modbus-master' del proyecto creado en el paso de la Figura 10. La estructura del proyecto debe ser similar a la Figura 17.

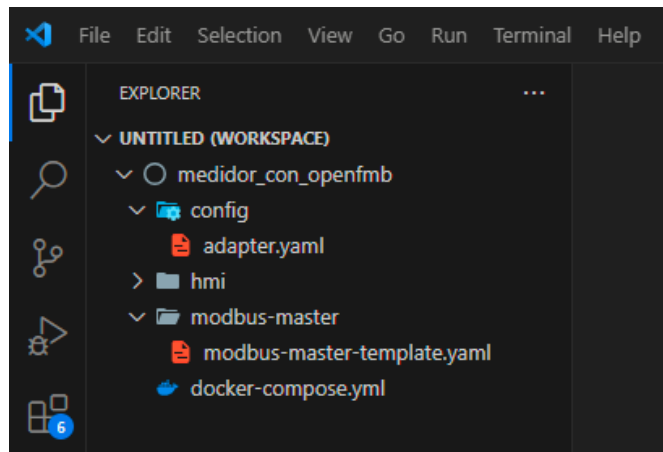


Figura 17: Estructura del proyecto OpenFMB con ficheros generados con la herramienta OACT.

2.7. Fichero del adaptador OpenFMB.

En el contexto de OpenFMB, el adaptador es el elemento encargado de traducir protocolos heredados, como Modbus, al protocolo de mensajes interno de la plataforma, por ejemplo NATS. Esto lo hace con ayuda de módulos adicionales llamados plugins, que deben ser habilitados y configurados adecuadamente. En la Figura 18 se muestra una sección del fichero del adaptador OpenFMB donde se observa la sintaxis YAML empleada para definir sus componentes, además se puede ver las configuraciones de los plugins **modbus-master** y **nats** establecidas utilizando la herramienta OACT explicada en la **Sección 2.6** (Figuras 14 y 15).

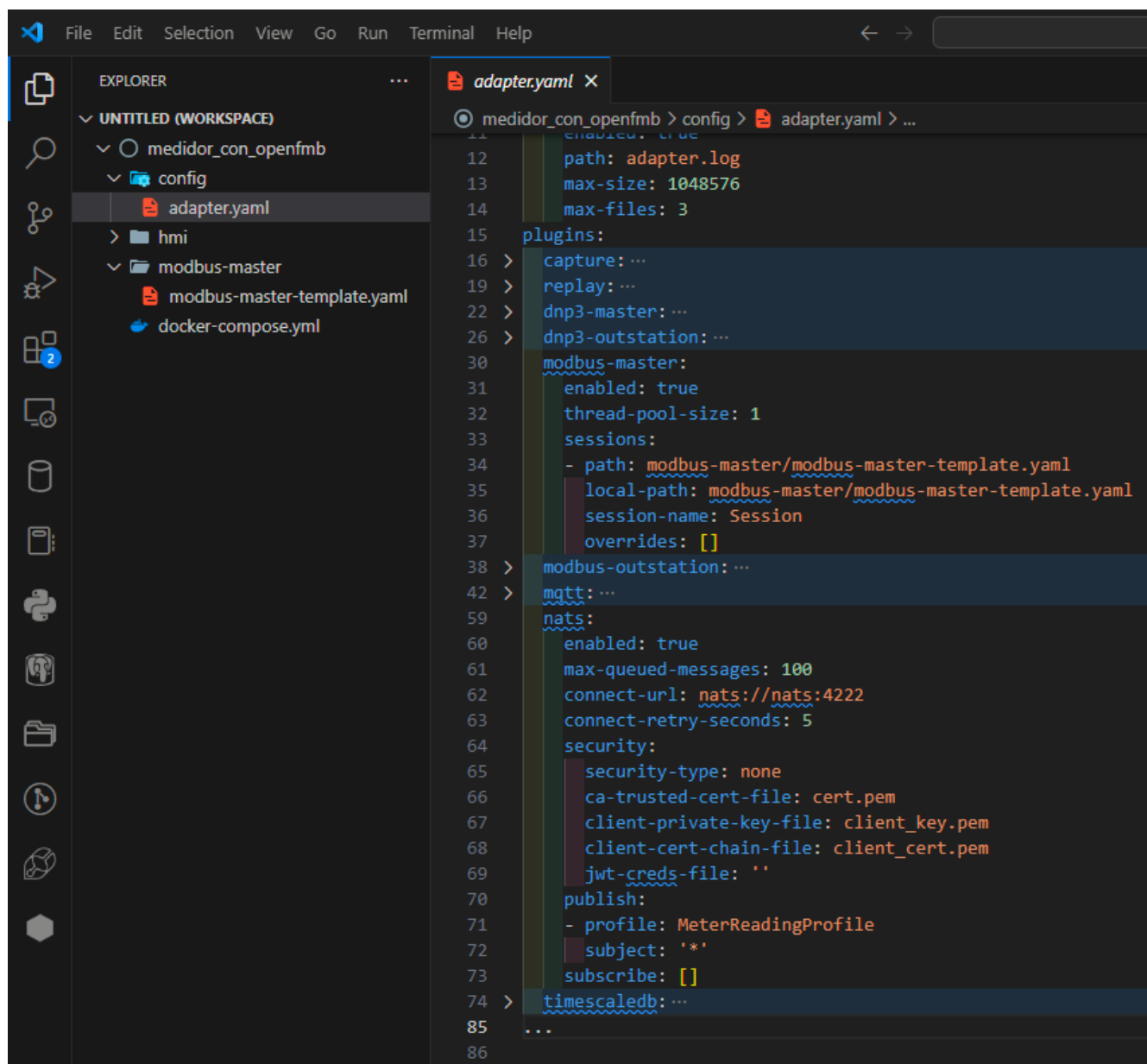


Figura 18: Sección de plugins del fichero del adaptador OpenFMB.

En la Figura 18 se observa que, además de los plugins mencionados, existen otros que permiten recopilar o reproducir información de una topología gestionada por OpenFMB. Se tienen también módulos para soportar protocolos heredados como Modbus y DNP3, así como sistemas de mensajería NATS o MQTT. Por último, el adaptador permite configurar una base de datos tipo PostgreSQL para almacenar la información que circula por la plataforma OpenFMB.

2.8. Fichero del plugin modbus-master del medidor Modbus A9MEM3155 a OpenFMB.

El medidor Modbus de interés emplea un adaptador RS485 a Ethernet para soportar Modbus TCP/IP como protocolo de comunicaciones, por lo cual, el plugin **modbus-master** del adaptador OpenFMB identifica este equipo por medio de tres parámetros principales: dirección IP, puerto, ID de esclavo. Se debe recordar que los elementos gestionados por esta plataforma deben estar en el mismo segmento de red. Además de estos parámetros, se muestra el perfil de lectura seleccionado desde OACT (Figura 14), para mapear la información del medidor Modbus, como se mencionó en la **Sección 2.6**.

La sintaxis y las propiedades del fichero de configuración del plugin **modbus-master** se pueden visualizar en

la Figura 19.

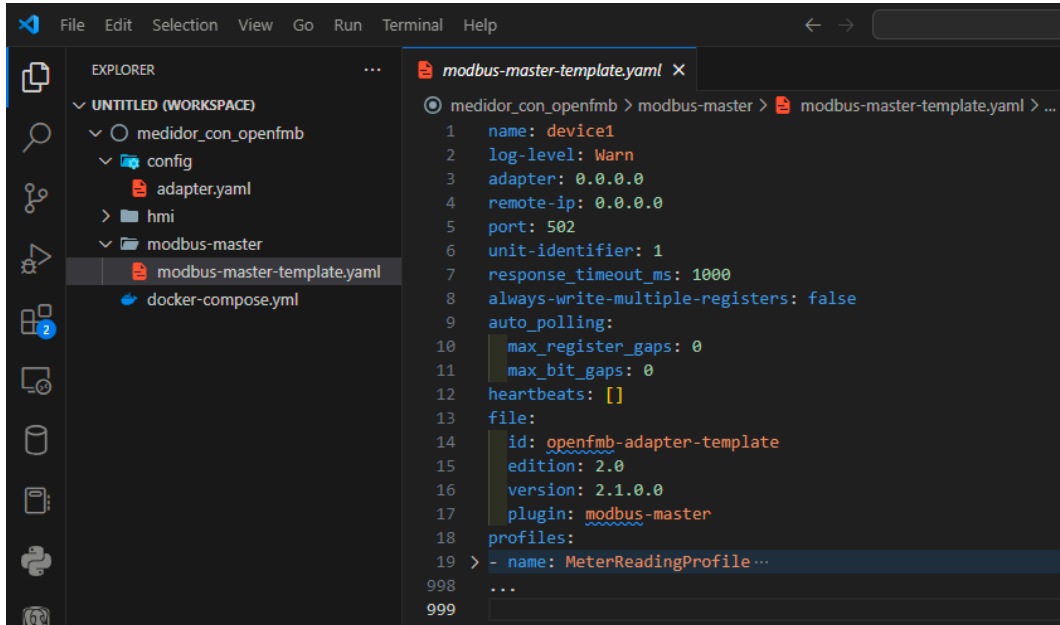


Figura 19: Fichero del plugin **modbus-master** generado con herramienta OACT para vincular un medidor Modbus.

A continuación, se debe modificar este fichero para que el adaptador OpenFMB identifique el medidor, los datos para vincular el medidor A9MEM3155 por defecto son 'remote-ip: 192.168.0.7', 'port:26' y 'unit-identifier:1', que se pueden visualizar o modificar en la interfaz WEB de configuración del adaptador RS485 a Ethernet, específicamente, en la opción 'Local IP Config' ilustrada en la Figura 9. El resultado de estas modificaciones debe ser similar a la Figura 20.

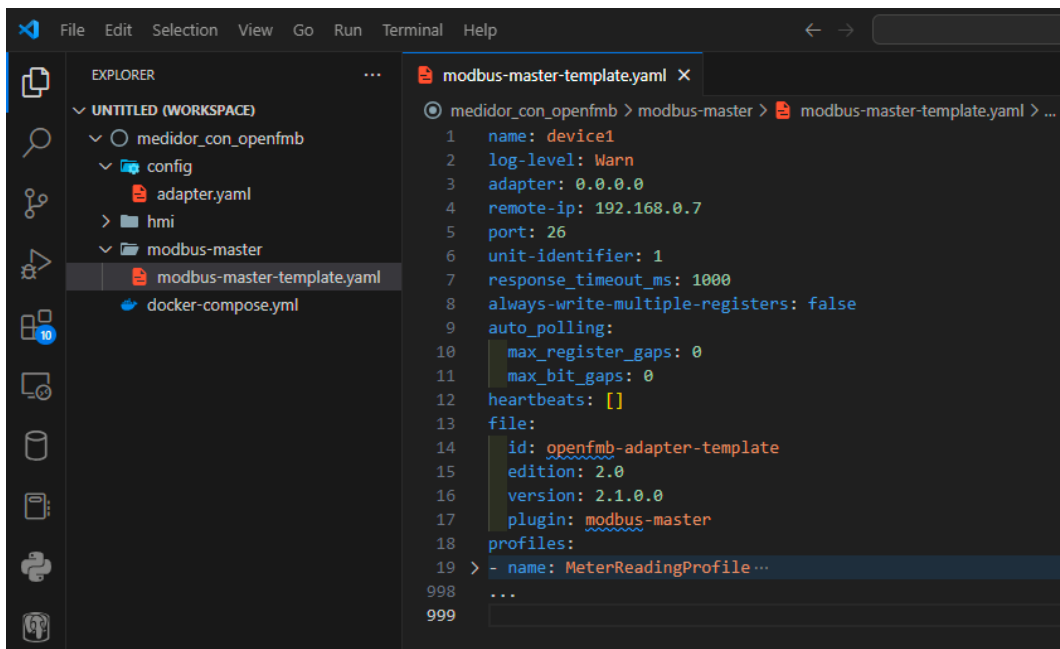


Figura 20: Fichero del plugin **modbus-master** direccionando un medidor A9MEM3155.

El perfil **MeterReadingProfile** es uno de los modelos estandarizados de la plataforma con los cuales se pueden leer datos de equipos en la topología, en este caso un medidor. El primer campo que se debe modificar es el identificador 'mRID', el cual es una secuencia de 20 dígitos numéricos en formato hexadecimal. Este parámetro

identifica un perfil dentro del flujo interno de datos en la plataforma, por esta razón debe ser diferentes para cada uno, incluso si se tienen múltiples medidores con registros iguales. El formato de este identificador y la variable a modificar dentro del perfil **MeterReadingProfile** se muestra en la Figura 21.

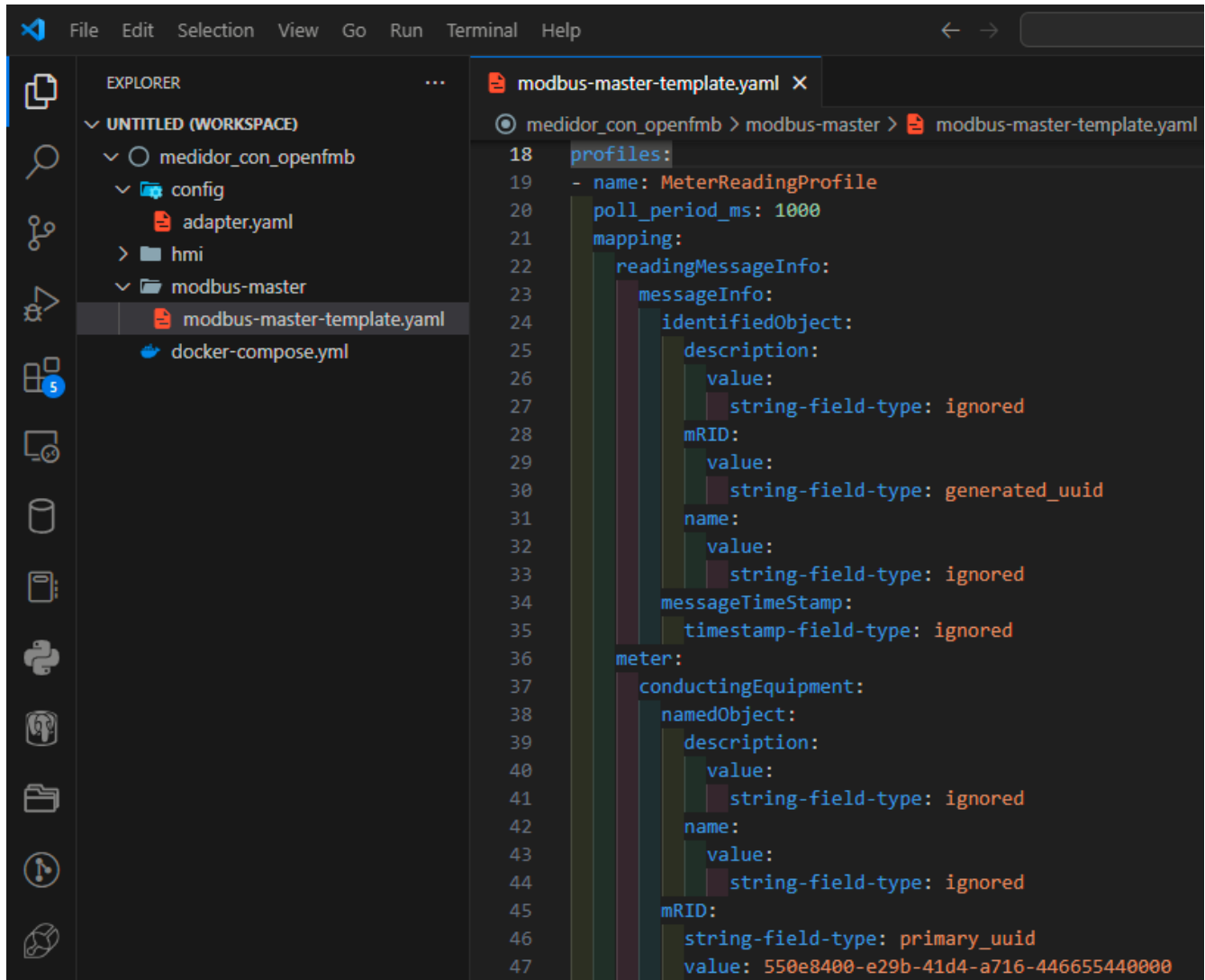


Figura 21: Identificador mRID de un perfil **MeterReadingProfile**.

Para establecer el identificador **mRID** de un perfil **MeterReadingProfile**, se debe modificar la propiedad 'conductingEquipment' en sus campos 'string-field-type' y 'value', tal como se ilustra en la Figura 21. Posteriormente, se seleccionan otros campos del perfil de lectura para hacer el mapeo de registros del medidor. La sintaxis para vincular variables a estos campos se muestra en la [documentación de OpenFMB](#). Adicionalmente, se debe conocer la tabla de registros reales del equipo de interés, en este caso del [medidor A9MEM3155](#). El resultado del mapeo del registro 3028 del medidor se ilustra en la Figura 22.

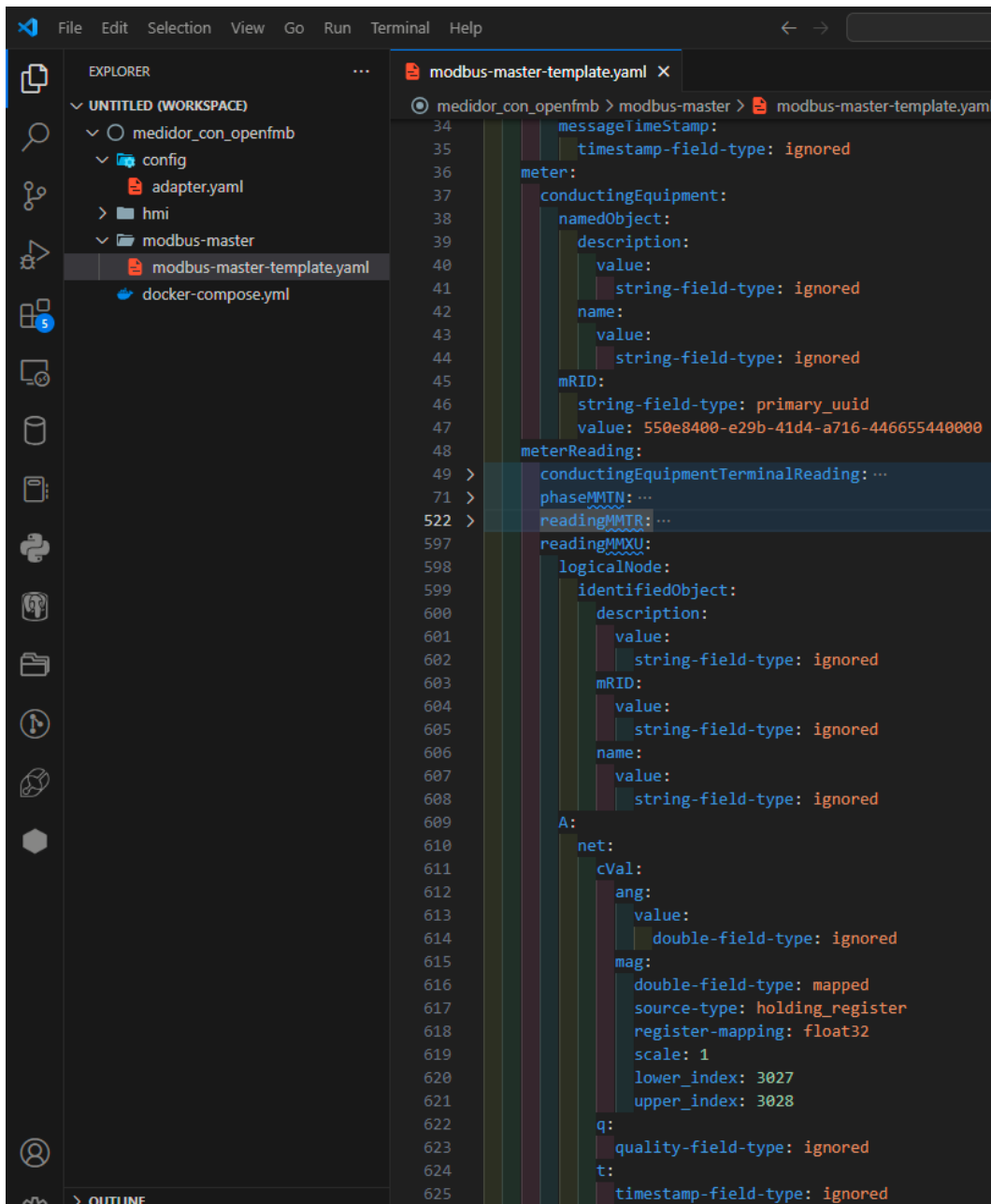


Figura 22: Mapeo de voltaje L1-N de medidor A9MEM3155 con perfil **MeterReadingProfile**.

La variable correspondiente al voltaje L1-N del medidor son los registros tipo 'float' 3028 y 3029 (según su documentación), que como se muestra en la Figura 22, está en la ruta 'mapping.meterReading.readingMMXU.A.net.cVal.mag' del perfil **MeterReadingProfile**.

Una de las consideraciones pertinentes están relacionadas a la longitud del registro y su formato, que respectivamente son: 32 bits y 'big-endian', por lo cual se emplean 2 índices, iniciando por el mayor (en el campo 'upper index') y luego el menor (en el campo 'lower index'). Finalmente, cabe resaltar que para leer los registros 3028 y 3029 del medidor, se establecieron los índices en 3027 y 3028 (que son un número menos), ya que así los interpreta la plataforma.

2.9. Configuración de la interfaz gráfica HMI de OpenFMB.

La interfaz gráfica de OpenFMB, al igual que el adaptador, son aplicativos pre construidos que requieren configuraciones y archivos adicionales para operar adecuadamente. Esto se puede observar en la Figura 1. Por

facilidad, se recomienda copiar la carpeta ‘hmi’ del proyecto de referencia (explicado en la **Sección 1.2**) dentro del directorio raíz del proyecto que se está trabajando. El resultado de este paso se muestra en la Figura 23.

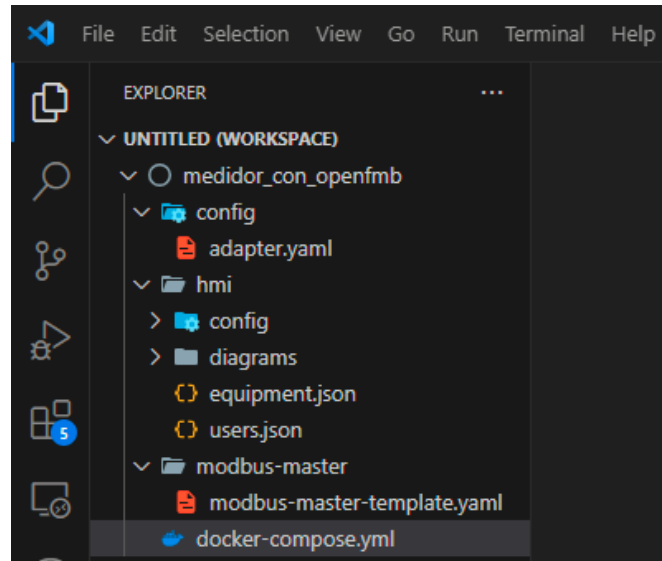


Figura 23: Proyecto de OpenFMB con el directorio de configuraciones del HMI.

En el directorio raíz del proyecto, mostrado en la Figura 23, se tienen los ficheros del adaptador OpenFMB y de su plugin **modbus-master**, los cuales se configuraron para vincular un medidor Modbus. Adicionalmente, se agregó la carpeta ‘hmi’ que proviene del proyecto de referencia. Dentro de este directorio no es necesario realizar modificaciones, ya que cuenta con las configuraciones necesarias para conectarse al servicio de mensajes NATS, además incluye usuarios registrados por defecto y tiene el diagrama del proyecto de referencia. Estas configuraciones pueden ser editadas dentro del entorno gráfico de la herramienta posteriormente.

2.10. Fichero docker-compose.yml para ejecutar plataforma OpenFMB.

Este fichero, además de definir las configuraciones de múltiples servicios de un mismo aplicativo, permite gestionar su ejecución conjunta de manera sencilla. Para ejecutar la plataforma OpenFMB vinculando el medidor Modbus se requieren tres aplicaciones inicialmente: la mensajería NATS, el adaptador OpenFMB y la interfaz gráfica HMI. Se recomienda la siguiente estructura. Las configuraciones de cada servicio, así como su sintaxis, se muestran en la Figura 24.

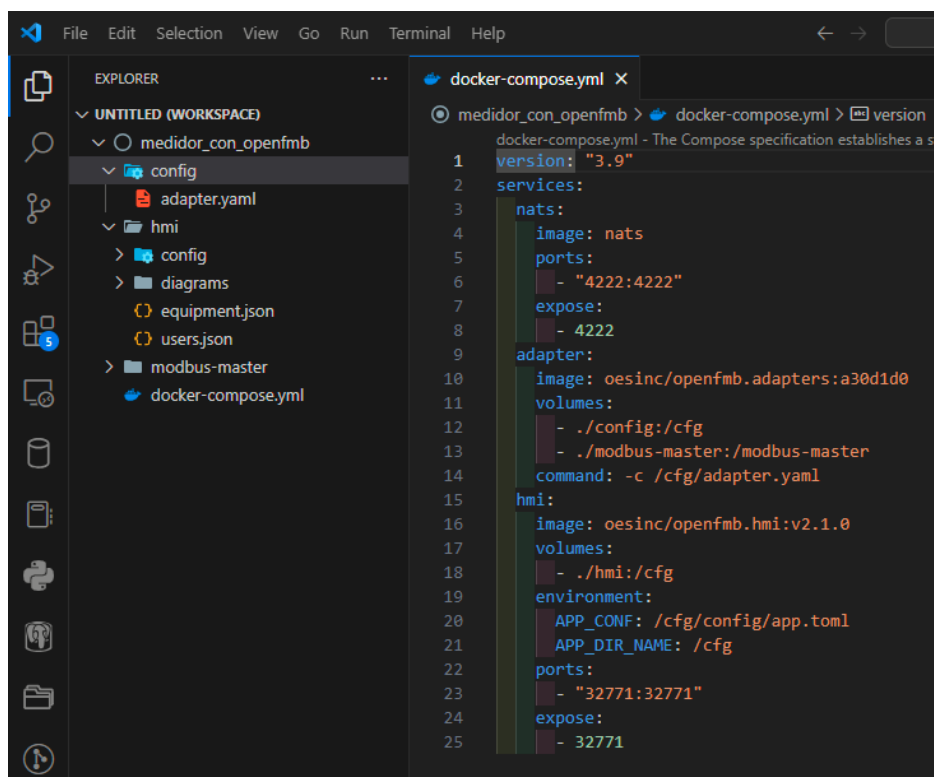


Figura 24: Fichero docker-compose.yml para ejecutar OpenFMB con el medidor A9MEM3155.

En la Figura 24 se observa la estructura del fichero docker-compose.yml. El servicio de mensajes NATS se configura con la imagen de Docker y el puerto que esta emplea para atender peticiones. El siguiente es el adaptador OpenFMB, que usa la imagen predefinida de Open Energy Solutions. Este servicio recibe las configuraciones de los plugins habilitados, los respectivos ficheros de configuración (en este caso solo el plugin **modbus-master**) y un comando con el que se pone en ejecución este aplicativo. Finalmente, en la interfaz gráfica HMI se establece la imagen, el directorio de configuraciones, las variables de entorno y el puerto por el que se accede a la interfaz web.

2.11. Ejecución de la plataforma OpenFMB con medidor A9MEM3155.

En este punto se tienen las configuraciones básicas para el adaptador OpenFMB, sus plugins y la interfaz gráfica. Adicionalmente, se tiene el fichero docker-compose.yml que define estos servicios y permite su ejecución conjunta. Para poner en marcha la plataforma OpenFMB con estos elementos, se abre una terminal de Visual Studio Code dentro del directorio con todos los archivos del proyecto, al mismo nivel que el fichero docker-compose.yml, entonces se ejecuta el comando 'docker-compose up'. Esto debe iniciar la ejecución de los diferentes servicios dentro de una red Docker, al mismo tiempo se muestran mensajes del estado de los componentes de OpenFMB que se están ejecutando, similar a como se ilustra en la Figura 25 y 26.

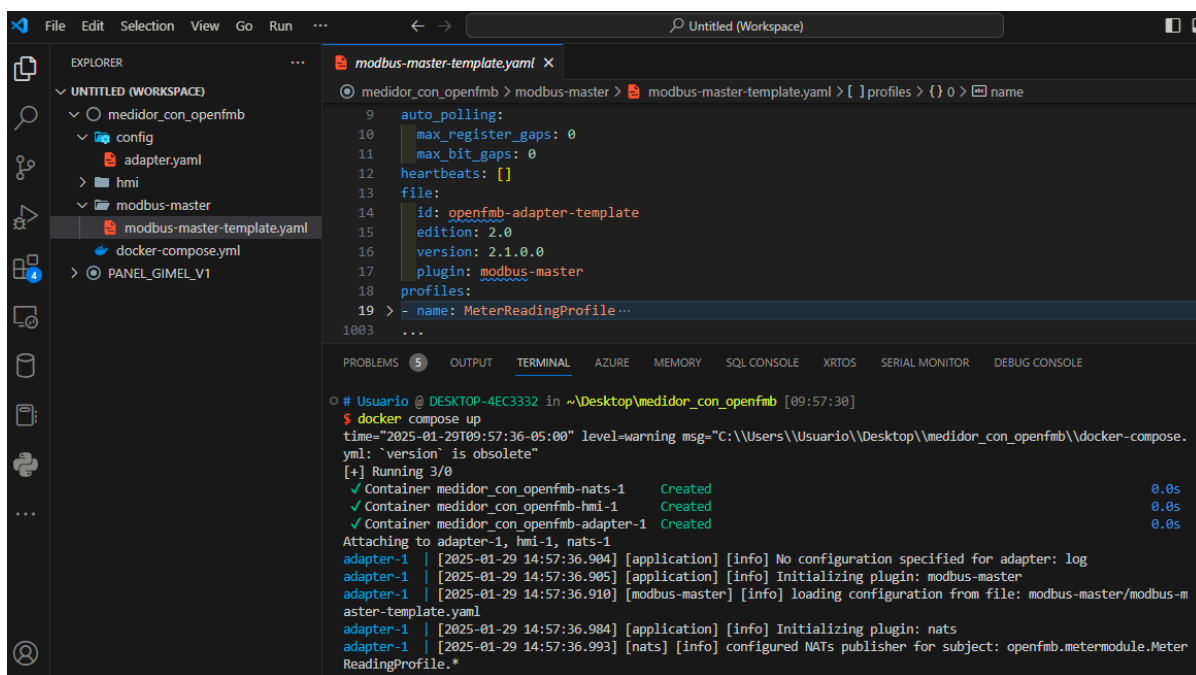


Figura 25: Inicialización de OpenFMB con medidor A9MEM3155.

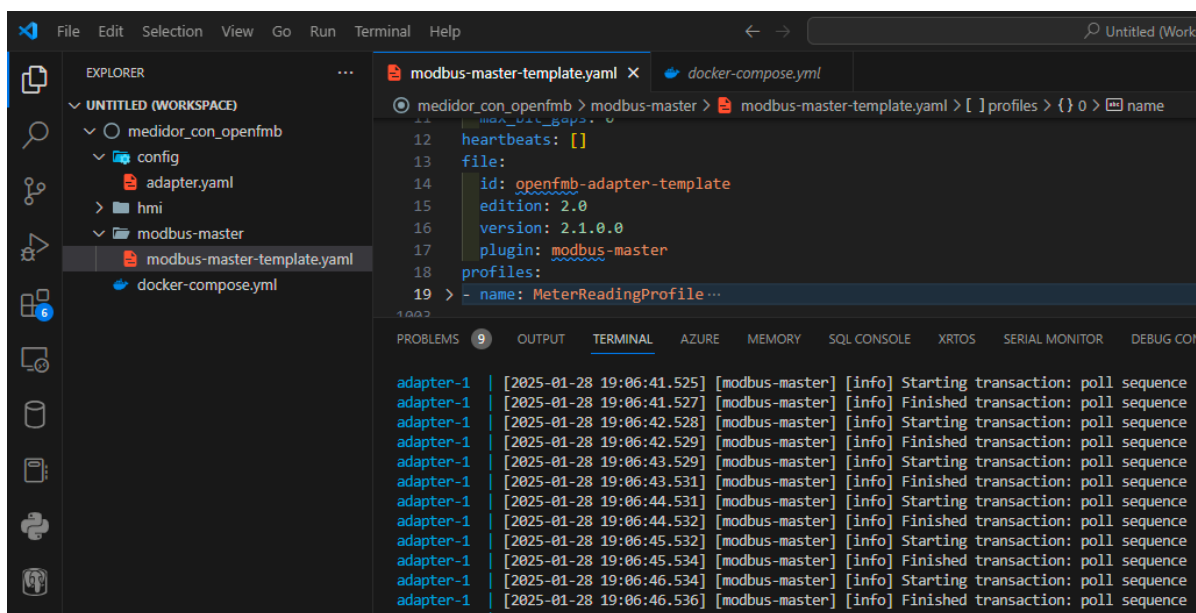


Figura 26: Plataforma OpenFMB leyendo información de un medidor A9MEM3155.

En la Figura 25, se muestran los ‘logs’ generados por el adaptador de OpenFMB luego de poner en ejecución la plataforma. En estos mensajes se muestra que el adaptador inicia una transacción con medidor cada segundo, en este caso solo se lee un registro que, como se mencionó al final de la **Sección 2.8**, corresponde al voltaje L1-N del medidor.

2.12. Visualización de diagramas interactivos con el HMI de OpenFMB.

La herramienta HMI se conecta al servicio de mensajes NATS de la plataforma, lo que le permite enviar y recibir información de los equipos vinculados a OpenFMB. Sin embargo, el adaptador es quien extrae y publica periódicamente la información del medidor en NATS por medio del perfil de OpenFMB, por esto es necesario

registrar su respectivo identificador ‘mRID’ en el HMI. De esta manera, las variables del equipo se podrán supervisar por medio de los diagramas interactivos.

Para registrar un dispositivo dentro del HMI, Lo primero es verificar que la plataforma se encuentra en ejecución (como se explica en la **Sección 2.11**), luego se escribe la dirección ‘localhost:32771’. Se deberá observar la interfaz web de la herramienta HMI, luego se da clic en la opción ‘SETTINGS’ y luego en ‘Devices’. En esta vista se deben encontrar los dispositivos del diagrama del proyecto de referencia de OpenFMB (mostrado en la Figura 6). A continuación se debe registrar el identificador ‘550e8400-e29b-41d4-a716-446655440000’ que se asignó previamente al perfil del medidor (mapeo mostrado en la Figura 22). Para registrar un nuevo dispositivo se da clic en ‘ADD DEVICE’, en el recuadro emergente se debe escribir un nombre, el tipo y el identificador ‘mRID’, finalmente, se da clic en ‘SAVE’. Estos pasos se muestran en la Figura 27.

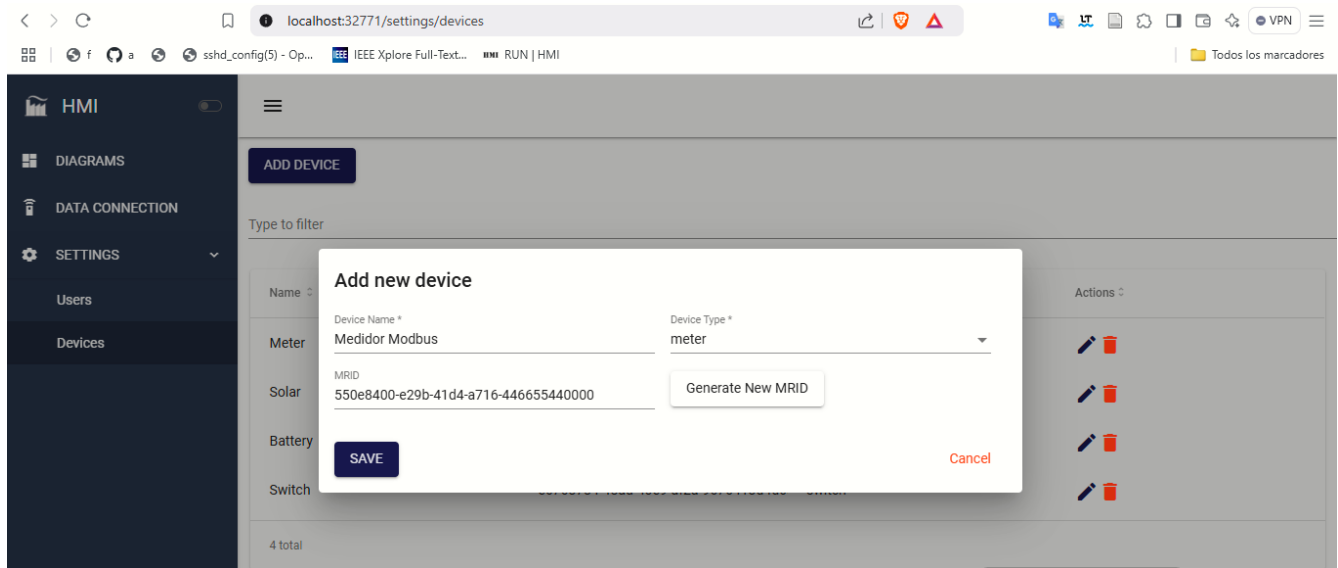


Figura 27: Registro del identificador ‘mRID’ del medidor Modbus en la herramienta HMI.

El siguiente paso es crear un diagrama que permita visualizar la información del equipo con perfil registrado en la herramienta HMI. Para esto se da clic en la opción ‘DIAGRAMS’, en el menú de la izquierda, luego en ‘ADD DIAGRAM’. Se debe obtener una vista similar a la que se muestra en la Figura 28.

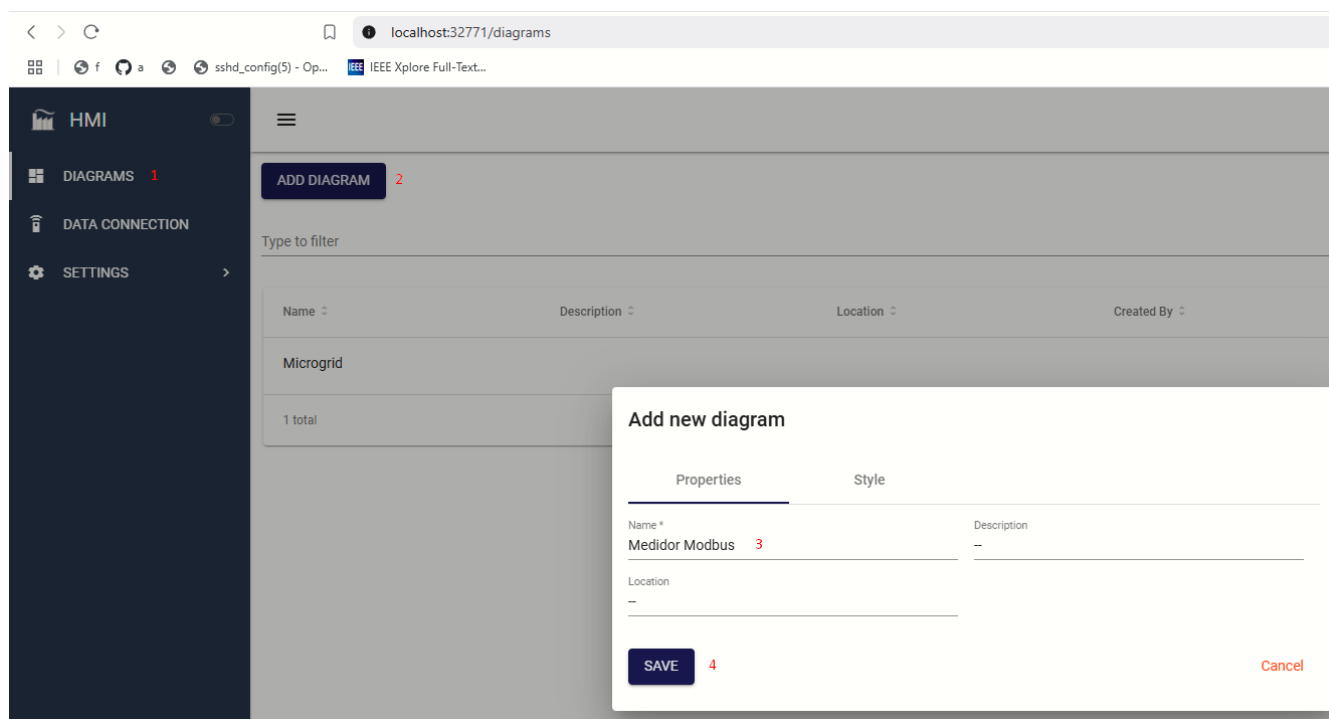


Figura 28: Creación de un diagrama nuevo en el HMI de la plataforma OpenFMB.

Una vez dentro del diagrama vacío, se agrega un nuevo elemento llamado ‘Measure Box’ y se selecciona el nombre del dispositivo que se registró previamente. Esto autocompletará el mRID del respectivo dispositivo y se confirma dando clic en ‘Done’. Opcionalmente, se puede agregar información como una etiqueta para el elemento del diagrama, este paso se muestra en la Figura 29.

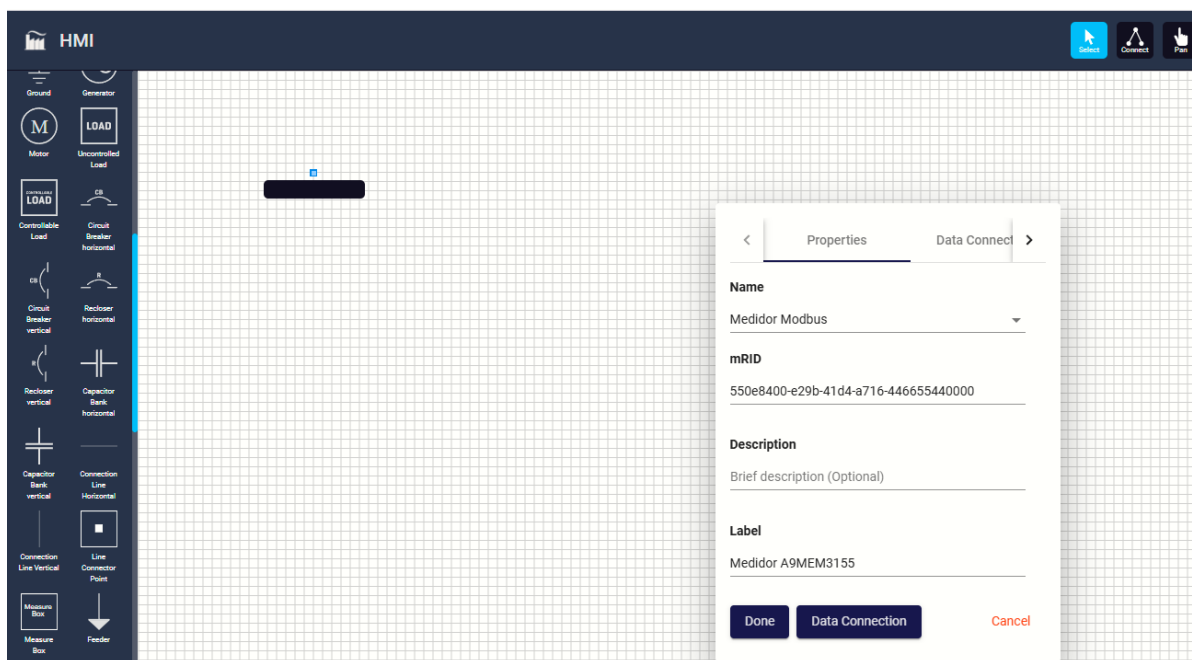


Figura 29: Vinculando un componente de un diagrama a un dispositivo registrado en la HMI.

El siguiente paso es seleccionar las variables que se mostrarán en el componente ‘Measure Box’. Para esto se da clic nuevamente sobre el elemento y dentro de sus opciones se selecciona ‘Data Connection’. Esto redirige a

una nueva página donde se asocia el componente del diagrama con las variables del respectivo perfil de interés, aquí se deben seleccionar los campos del perfil OpenFMB utilizado en el mapeo de registros del equipo real. Este procedimiento se muestra en la Figura 30.

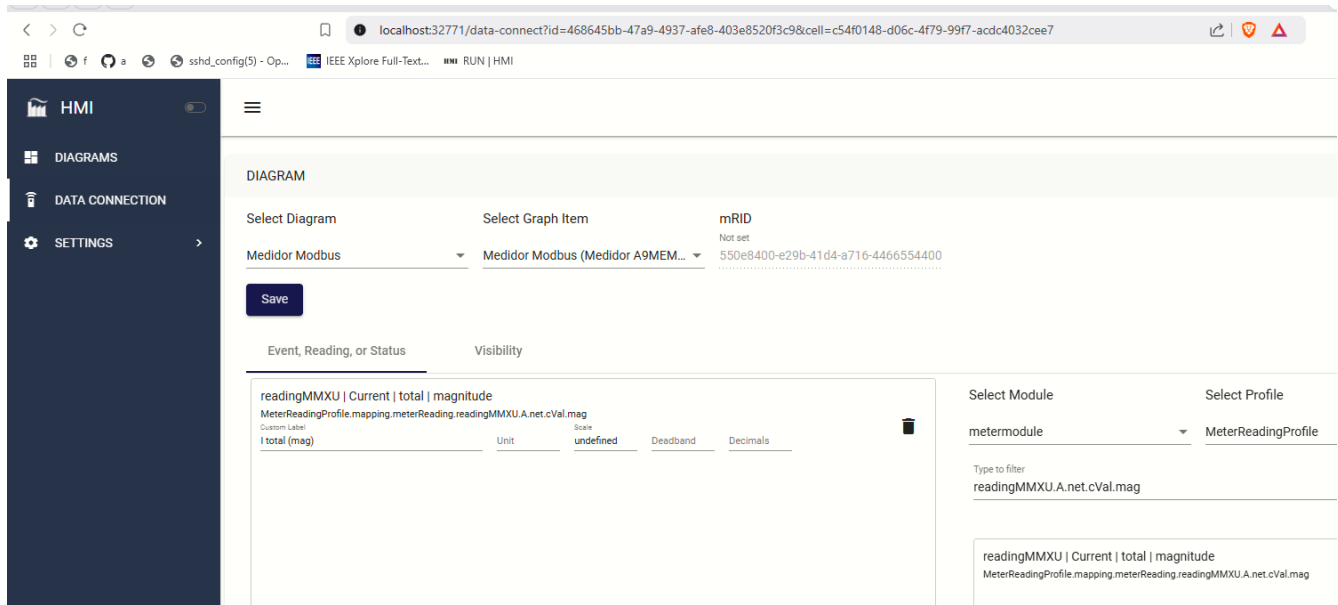


Figura 30: Vinculando variables de un perfil **MeterReadingProfile** al componente ‘Measure Box’ de un diagrama.

En la página ‘Data Connection’ del HMI, mostrada en la Figura 30, se puede seleccionar el diagrama y el componente en específico al cual se le quieren enlazar campos de un perfil específico (previamente registrado en la plataforma como en la Figura 27). También se puede observar que en la parte central baja hay 2 paneles: uno con las variables enlazadas al componente y otro con el listado de variables disponibles según el perfil seleccionado. Como se mostró al final de la **Sección 2.8**, la variable que se utilizó tiene la secuencia ‘meterReading.readingMMXU.A.net.cVal.mag’, por esto el respectivo módulo es ‘meter’ y el perfil **MeterReadingProfile**. Al finalizar se confirma selección de variables dando clic en ‘Save’.

Luego de seguir estos pasos, se puede abrir de nuevo el listado de diagramas, donde se tienen las diferentes acciones para cada diagrama, y poner en ejecución el que contiene el mapeo de variables del medidor Modbus. Estos últimos pasos se muestran en las Figuras 31 y 32.

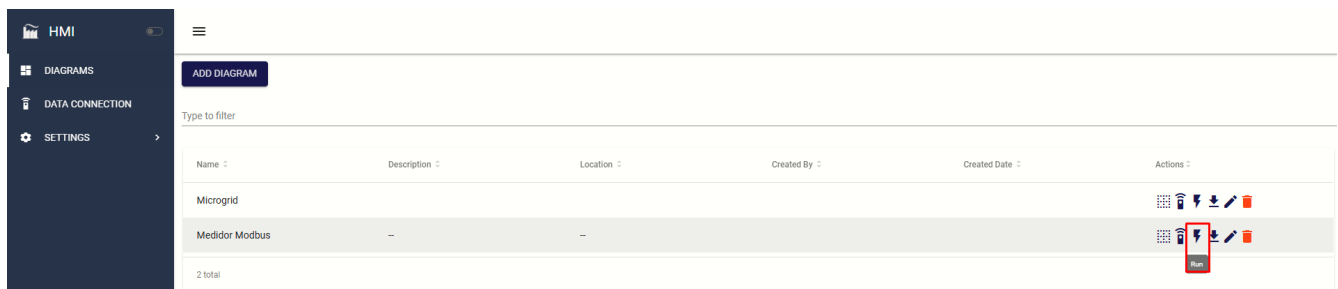


Figura 31: Ejecutando un diagrama del HMI.



Figura 32: Monitoreo de voltaje L1-N de medidor A9MEM3155 con diagrama del HMI de OpenFMB.

2.13. Conexión de carga monofásica al medidor A9MEM3155.

Hasta este punto es posible visualizar en la interfaz gráfica el voltaje L1-N del medidor, pero no las demás variables relevantes de una carga como corriente o potencia. Para esto, primero se conecta físicamente una carga al medidor, el esquema de conexión se muestra en la Figura 33.

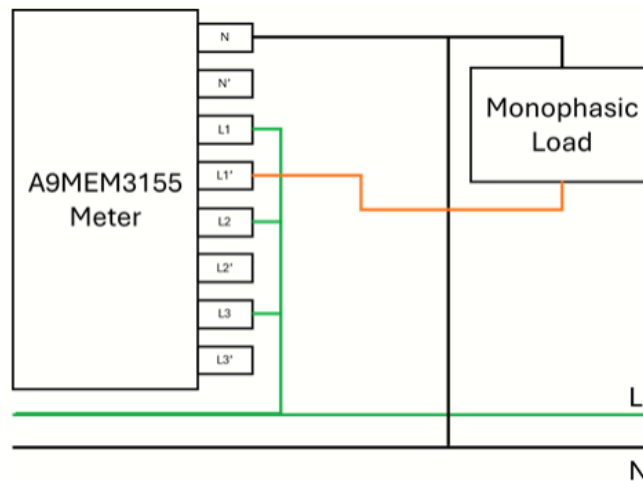


Figura 33: Conexión de carga monofásica con medidor A9MEM3155.

Para realizar las conexiones mostradas en la Figura 33, se pueden seguir estos pasos.

- Conectar L1, L2 y L3 a un mismo bus.
- Conectar una carga monofásica entre los bornes de N y L1'.
- Energizar el medidor conectando L al bus de L1, L2 y L3, y N del medidor al N de la red eléctrica.

2.14. Ajustes en OpenFMB para la lectura de nuevas variables.

Luego de conectar la carga monofásica al medidor A9MEM3155, se recomienda seguir este orden para ajustar las configuraciones en la plataforma y visualizar nueva información del medidor como potencia o corriente de la carga.

Inicialmente, con la plataforma detenida, se deben abordar las configuraciones del plugin **modbus-master** que mapea la información del medidor.

Como se muestra en la Figura 34, se emplean otros campos del perfil **MeterReadingProfile** de tipo 'double-field-type' para mapear nuevos registros del medidor. Similar al mapeo del voltaje L1-N, se deben establecer algunas propiedades entre las que están: el tipo de registro, el tipo de dato y los índices correspondientes a los registros del medidor que se desean mapear, por ejemplo para este medidor, los registros 3010 y 3076 son respectivamente la corriente media y la potencia aparente total.

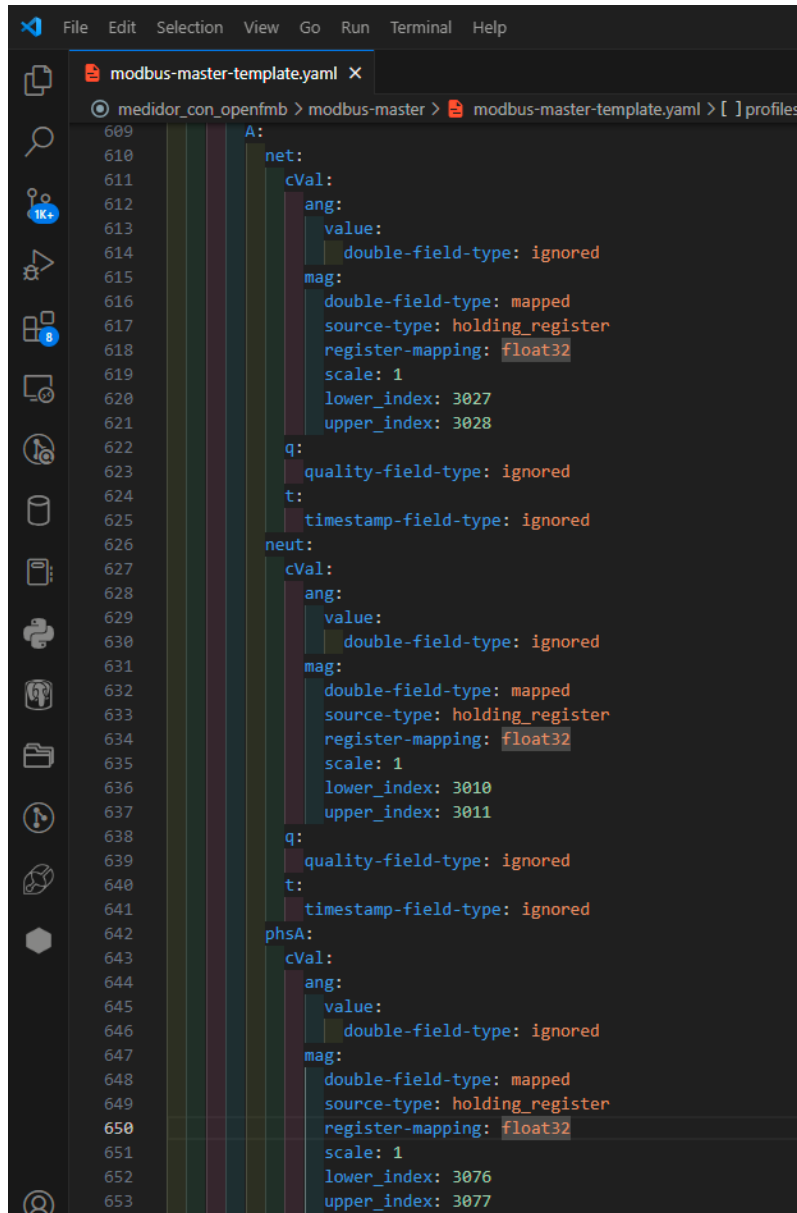


Figura 34: Mapeo de múltiples registros del medidor con perfil **MeterReadingProfile**.

Posteriormente, se ejecuta la plataforma OpenFMB y se accede a la interfaz de edición de diagramas del HMI, como se muestra en la Figura 29 de la **Sección 2.12**. A continuación se ingresa a la opción 'Data Connection' dentro del componente que muestra el voltaje L1-N del medidor. En esta vista se buscan las variables nuevas del perfil **MeterReadingProfile** que mapean los registros de corriente media y potencia aparente total, como se muestra en la Figura 35. Por último, se da clic en 'Save'.

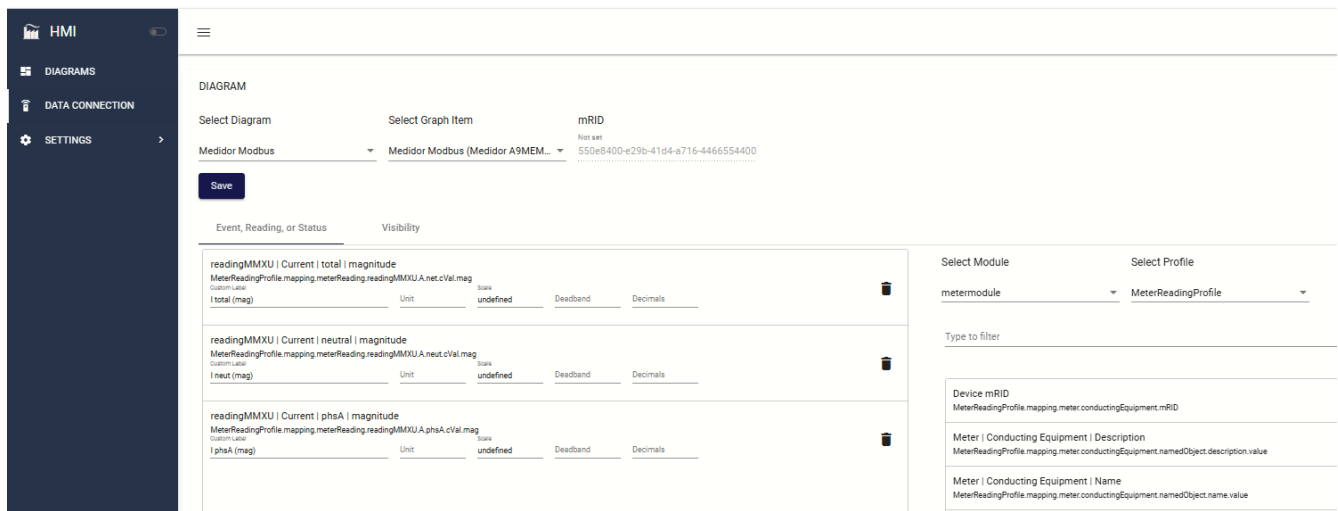


Figura 35: Selección de variables del perfil **MeterReadingProfile** que se muestran en diagrama para el medidor.

Luego de seguir estos pasos y ejecutar el respectivo diagrama, se podrá visualizar, el componente gráfico con las lecturas de voltaje, corriente y potencia del medidor. Esto se muestra en la Figura 36.



Figura 36: Visualización del diagrama con múltiples datos del medidor A9MEM3155.

Este conjunto de pasos para vincular dispositivos a OpenFMB y visualizar sus datos con la herramienta HMI se puede complementar con [la documentación del plugin modbus-master](#) del adaptador de OpenFMB.

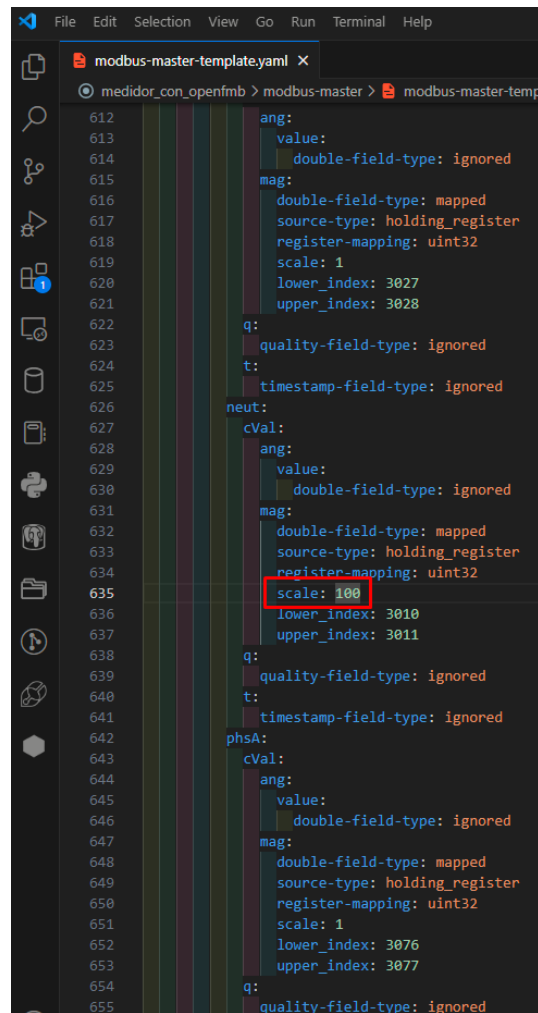
3. Aplicación de factores de escala sobre los datos.

Los datos extraídos de los dispositivos que se vinculan a OpenFMB pueden ser escalados por un determinado factor. Esto permite, por ejemplo, visualizar las cantidades de los diagramas en diferente escala de unidades. Este procedimiento se puede hacer de dos maneras.

3.1. Escalamiento de datos desde el plugin modbus-master del adaptador de OpenFMB.

Como se menciona en la **Sección 2.8**, el adaptador emplea perfiles de datos de la plataforma para mapear registros de dispositivos Modbus. Dentro de las propiedades que se definen para este mapeo se encuentra 'scale', que permite aplicar un factor de incremento (mayor a 1) o de decremento (menor a 1). En la Figura 37 se muestra el uso de esta configuración para escalar por 100 el valor leído del registro 3010 de un esclavo Modbus, mientras que

en la Figura 38 se observa el mismo diagrama empleado con el medidor, pero se observan los cambios producidos por el escalamiento del registro 3010.



```
modbus-master-template.yaml
medidor_con_openfmb > modbus-master > modbus-master-temp
612   ang:
613     value:
614       double-field-type: ignored
615   mag:
616     double-field-type: mapped
617     source-type: holding_register
618     register-mapping: uint32
619     scale: 1
620     lower_index: 3027
621     upper_index: 3028
622   q:
623     quality-field-type: ignored
624   t:
625     timestamp-field-type: ignored
626   neut:
627     cVal:
628       ang:
629         value:
630           double-field-type: ignored
631       mag:
632         double-field-type: mapped
633         source-type: holding_register
634         register-mapping: uint32
635         scale: 100
636         lower_index: 3010
637         upper_index: 3011
638     q:
639       quality-field-type: ignored
640     t:
641       timestamp-field-type: ignored
642   phsA:
643     cVal:
644       ang:
645         value:
646           double-field-type: ignored
647       mag:
648         double-field-type: mapped
649         source-type: holding_register
650         register-mapping: uint32
651         scale: 1
652         lower_index: 3076
653         upper_index: 3077
654     q:
655       quality-field-type: ignored
```

Figura 37: Escalamiento de cantidades en la configuración del plugin **modbus-master**.



Figura 38: Escalamiento de cantidades en la configuración del plugin **modbus-master**.

En la Figura 37 se observa la forma de escalar por un factor de 100 el valor contenido en el registro 3010. Luego de ejecutar la plataforma con estas configuraciones en el plugin **modbus-master**, se ve el efecto en el diagrama de la Figura 38, donde se muestra un valor de 200 leído del registro con valor de 2 escalado previamente.

3.2. Escalamiento de datos desde la herramienta HMI de OpenFMB.

Otra forma de aplicar escalamiento a las cantidades de los dispositivos, es desde la vista 'Data Connection' de la interfaz web del HMI.

Para esto se puede entrar a la edición del diagrama y se selecciona el elemento 'Measure Box' que tiene asociado el dato que se desea escalar. Sobre este elemento se da clic en 'Data Connection', entonces se tiene una vista similar a la Figura 39 y se muestra la forma de establecer el escalamiento. Se da clic en 'Save' para confirmar los cambios.

Select Diagram	Select Graph Item	mRID
Medidor Modbus	Medidor Modbus (Medidor A9MEM...	Not set 550e8400-e29b-41d4-a716-4466554400

Save

Event, Reading, or Status	Visibility
readingMMXU Current total magnitude MeterReadingProfile.mapping.meterReading.readingMMXU.A.net.cVal.mag Custom Label: I total (mag) Unit: undefined Scale: undefined Deadband: Decimals: [icon]	
readingMMXU Current neutral magnitude MeterReadingProfile.mapping.meterReading.readingMMXU.A.neut.cVal.mag Custom Label: I neut (mag) Unit: Scale: 100 Deadband: Decimals: [icon]	
readingMMXU Current phsA magnitude MeterReadingProfile.mapping.meterReading.readingMMXU.A.phsA.cVal.mag Custom Label: I phsA (mag) Unit: undefined Scale: undefined Deadband: Decimals: [icon]	

Figura 39: Escalamiento de datos desde la interfaz web del HMI.

Luego de ejecutar el diagrama se llega a una vista exactamente igual a la Figura 38, dado que el efecto del escalamiento desde cualquiera de estas dos maneras es igual.

4. Acceso remoto a la interfaz HMI de OpenFMB.

En esta sección se explican los pasos para ejecutar un proyecto de OpenFMB y visualizar la interfaz gráfica con sus diagramas, de manera remota desde otro PC dentro del mismo segmento de red.

Es posible lanzar un proyecto OpenFMB ejecutando únicamente el adaptador principal y el plugin nats con el que se transfieren los datos al interfaz humano máquina. La interfaz gráfica se puede ejecutar, en otra máquina dentro del mismo segmento de red, en un contenedor de docker con sus respectivos parámetros de imagen, puertos, volumen de datos, etc.

Para evidenciar tal esquema de operación (la interfaz HMI ejecutándose separadamente del contenedor del adaptador OpenFMB), a continuación, se describe un entorno con una máquina virtual linux sin interfaz gráfica ejecutándose en un host con sistema operativo Windows. En este caso el adaptador OpenFMB corre en la máquina virtual con sistema operativo linux (Ubuntu Server). Este entorno no provee interfaz gráfica para destinar mayor capacidad de cómputo a los procesos del adaptador principal, por esta razón se detalla cómo lograr la visualización desde otro PC (con interfaz gráfica), por ejemplo, el PC anfitrión, que en este caso es Windows.

4.1. Conexión en red de Ubuntu Server y el anfitrión Windows.

primero, con la máquina virtual apagada, se configura el adaptador de red de la máquina virtual en ‘sólo anfitrión’, luego de este paso el anfitrión y la máquina virtual estarán en red por medio del adaptador elegido, en este caso ‘VirtualBox Host-Only Ethernet Adapter’ (ver Figura 40).

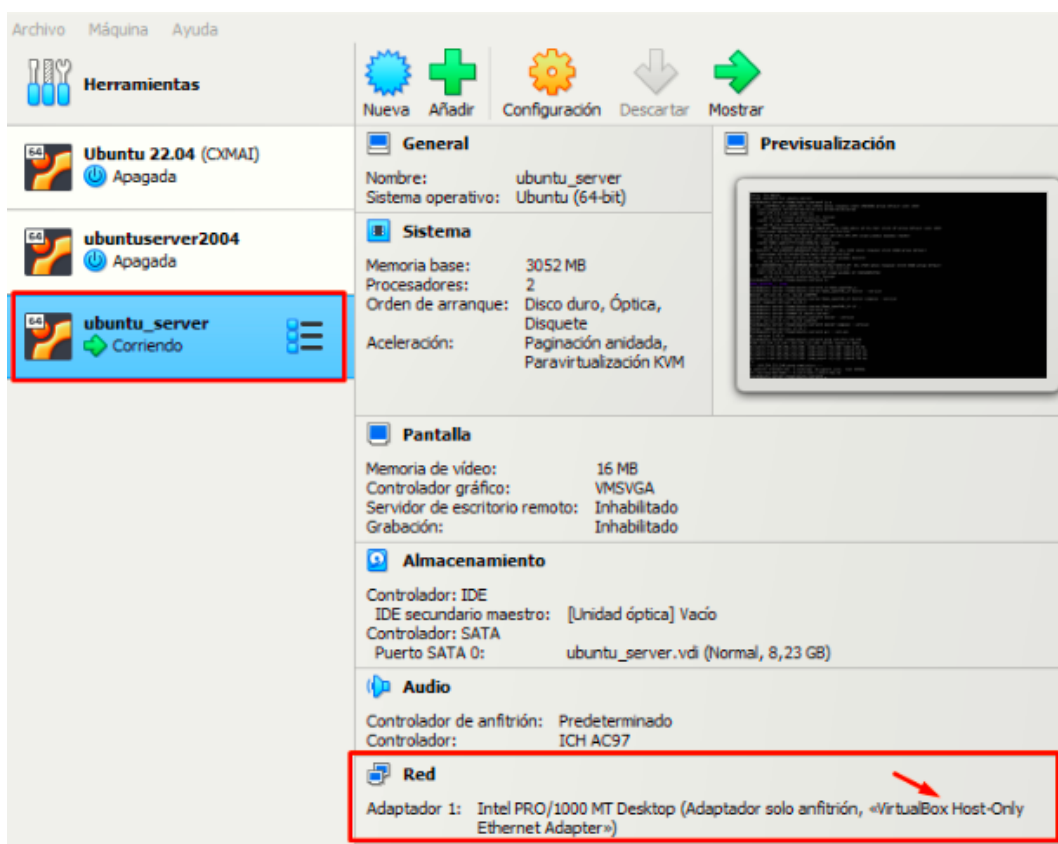


Figura 40: Adaptador ‘sólo anfitrión’ para máquina virtual de Ubuntu Server.

Es posible verificar la conectividad entre el anfitrión y la máquina virtual ya que la interfaz virtual ‘VirtualBox Host-Only Ethernet Adapter’ permite que sus direcciones IPv4 estén en el mismo segmento de red. Para ello en la

máquina virtual, se usa el comando ‘ip address’ o ‘ip a’ para identificar su IP. Luego se identifica la IP del anfitrión, que en Windows se puede ver usando el comando ‘ipconfig /all’ (ver Figuras 41 y 42).

```
root@ubuntu-server:/home/ubuntu-server# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:43:d5:3b brd ff:ff:ff:ff:ff:ff
    inet 169.254.213.250/16 metric 100 brd 169.254.255.255 scope global dynamic enp0s3
        valid_lft 510sec preferred_lft 510sec
    inet6 fe80::a00:27ff:fe43:d53b/64 scope link
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:34:8d:23:0a brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
4: br-34460db5376d: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:d4:68:a7:26 brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.1/16 brd 172.18.255.255 scope global br-34460db5376d
        valid_lft forever preferred_lft forever
```

Figura 41: Identificación de IP en máquina virtual de Ubuntu Server.

```
# % Usuario @ DESKTOP-4EC3332 in ~ [11:00:56] C:1
$ ipconfig /all

Configuración IP de Windows

Nombre de host. . . . . : DESKTOP-4EC3332
Sufijo DNS principal . . . . . :
Tipo de nodo. . . . . : híbrido
Enrutamiento IP habilitado. . . : no
Proxy WINS habilitado . . . . . : no

Adaptador de Ethernet Ethernet:

Estado de los medios. . . . . : medios desconectados
Sufijo DNS específico para la conexión. . :
Descripción . . . . . : Realtek PCIe GbE Family Controller
Dirección física. . . . . : 0C-9D-92-31-67-C8
DHCP habilitado . . . . . : sí
Configuración automática habilitada . . . : sí

Adaptador de Ethernet Ethernet 2:

Sufijo DNS específico para la conexión. . :
Descripción . . . . . : VirtualBox Host-Only Ethernet Adapter
Dirección física. . . . . : 0A-00-27-00-00-0A
DHCP habilitado . . . . . : no
Configuración automática habilitada . . . : sí
Vínculo: dirección IPv6 local. . . : fe80::2d21:6e38:6bd5:a982%10(Preferido)
Dirección IPv4 de configuración automática: 169.254.213.248(Preferido)
Máscara de subred . . . . . : 255.255.0.0
Puerta de enlace predeterminada . . . . . :
```

Figura 42: Identificación de IP en anfitrión Windows.

Con el comando ‘ping + IP’ DESTINO se prueba la conectividad entre la máquina virtual y el anfitrión. El resultado debe mostrar que los paquetes ICMP enviados retornan correctamente entre la máquina virtual y el anfitrión.

4.2. Validar la disponibilidad de docker y docker compose en Ubuntu Sever.

Esto es necesario para poder lanzar el fichero docker-compose.yml que contiene las configuraciones de los servicios del proyecto. Para esto se pueden ejecutar los mismos comandos utilizados en la **Sección 1.1**.

4.3. Obtener los archivos del proyecto.

En este caso se lanzará el demo para visualizar sus datos pre-guardados en un entorno web del anfitrión Windows. La estrategia consiste en usar una USB con el contenido del proyecto para copiarlo dentro de una determinada carpeta en el almacenamiento del servidor sin entorno gráfico. Se recomienda seguir estos pasos para montar manualmente el contenido de un dispositivo USB externo a un directorio dentro de la máquina virtual.

- I. Configurar el controlador USB de la máquina virtual en VirtualBox, como se muestra en la Figura 43.

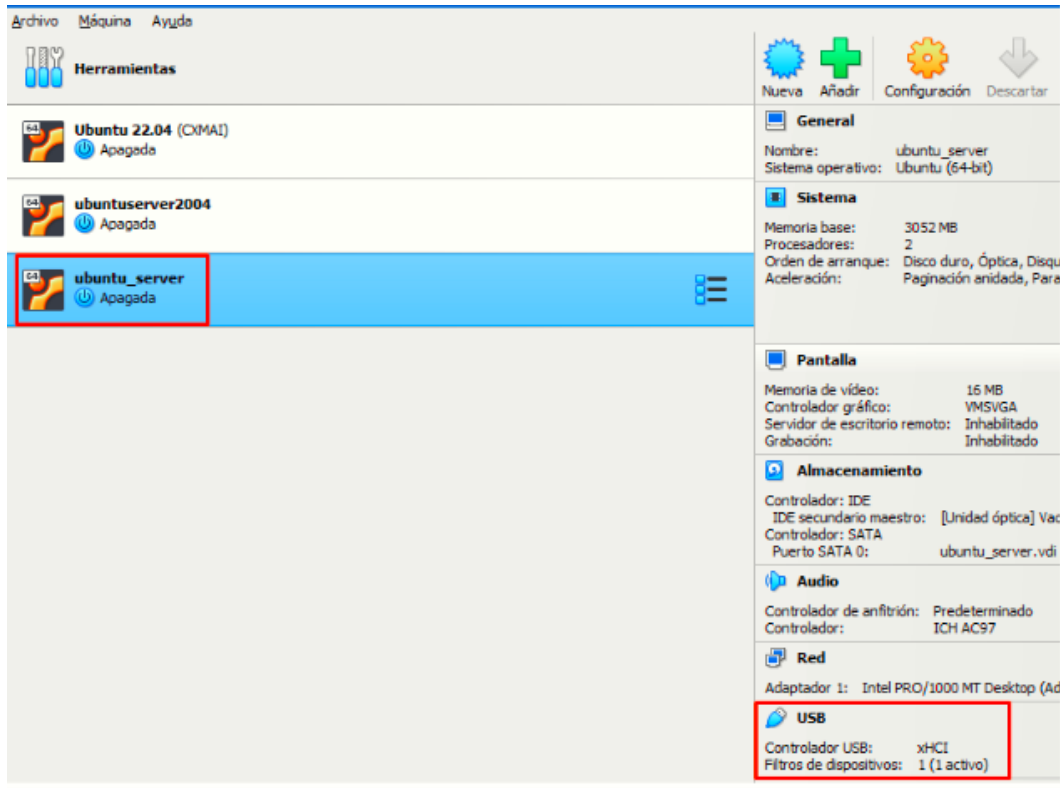


Figura 43: Configuración de controlador USB en virtualbox.

A continuación, en la ventana emergente, marcar 'Enable USB Controller' y escoger 'controlador USB 3.0', esto se observa en la Figura 44. El menú desplegable de la derecha muestra las opciones ANTES de conectar el dispositivo USB al PC anfitrión.

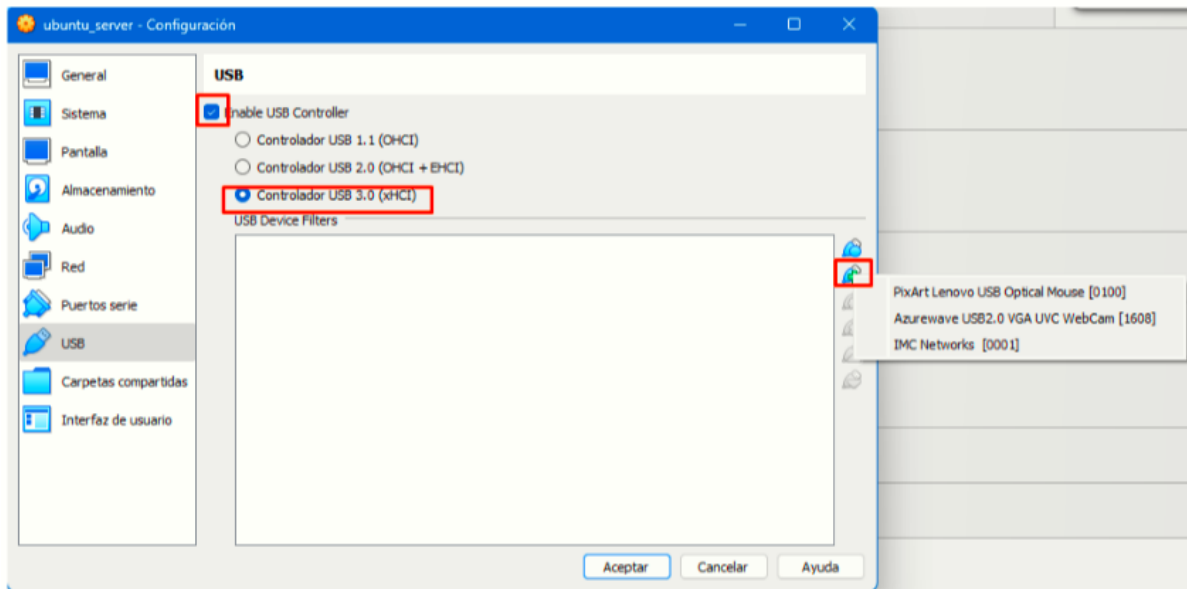


Figura 44: Opciones de controlador USB para máquina virtual Ubuntu Server.

Cuando se conecta la USB, aparece un nuevo elemento en el menú desplegable, este se debe agregar dando click en dicho elemento según se muestra en la figura 45.

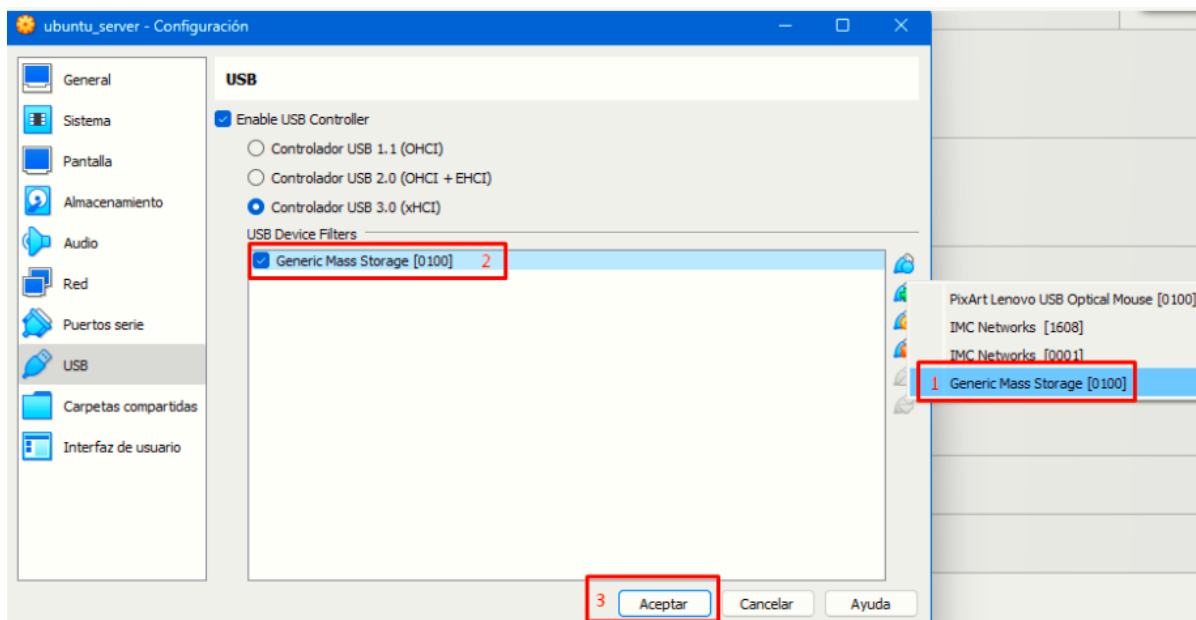


Figura 45: Agregando acceso a dispositivo USB en Virtualbox.

El siguiente paso es encender la máquina virtual y usar el comando.

- `sudo fdisk -l`

En caso de no tener disponible este comando se pueden ejecutar los comandos en Ubuntu:

- `sudo apt update`
- `sudo apt install util-linux`

Este comando permite saber la ubicación y el nombre del almacenamiento externo que se desea montar, en este caso la ruta del almacenamiento que se quiere montar es '/dev/sdb1'.

- II. Crear el directorio donde será montada la información del dispositivo USB externo. Se recomienda utilizar el comando 'mkdir' seguido de la ruta y nombre del directorio a crear.

- mkdir /media/perseo

- III. Se monta el almacenamiento USB con el comando mount con la siguiente estructura, se debe observar algo como la Figura 46.

- mount /dev/sdb1 /media/perseo

```
Disk /dev/sdb: 14.45 GiB, 15518924800 bytes, 30310400 sectors
Disk model: Flash Disk
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x99da5ad9

Device      Boot Start      End  Sectors  Size Id Type
/dev/sdb1                2048 30310399 30308352 14.5G c W95 FAT32 (LBA)
```

Figura 46: Información de dispositivo USB externo con el comando 'fdisk -l'.

- IV. Visualizar la información del dispositivo USB en el directorio creado. Se debe ver un resultado similar al de la Figura 47 con el comando:

- ls

```
root@ubuntu-server:/media#
root@ubuntu-server:/media#
root@ubuntu-server:/media#
root@ubuntu-server:/media#
root@ubuntu-server:/media# ls /media/perseo/
0_Fundallianza_2024_1  Consentimiento_multimodal.pdf  PD  demo_openfmb_1
20610001-4002-2024-Solicitud de cambio PT-PID-Tilak Purohit.pdf  Fundallianza_2024_1  'System Volume Information'  perseo-openfmb
```

Figura 47: Listado de directorios y ficheros del almacenamiento USB montado.

- V. A continuación, se deben copiar los archivos montados a un directorio de trabajo. En este caso el origen es '/media/perseo' y se copia la carpeta perseo-openfmb en el directorio de destino que es '/home/ubuntu-server/'. El resultado se puede verificar con el comando ls como se muestra en la Figura 48.

- cp -r /media/perseo/perseo-openfmb/ /home/ubuntu-server


```
root@ubuntu-server:/home/ubuntu-server#  
root@ubuntu-server:/home/ubuntu-server#  
root@ubuntu-server:/home/ubuntu-server#  
root@ubuntu-server:/home/ubuntu-server#  
root@ubuntu-server:/home/ubuntu-server#  
root@ubuntu-server:/home/ubuntu-server#  
root@ubuntu-server:/home/ubuntu-server#  
root@ubuntu-server:/home/ubuntu-server#  
root@ubuntu-server:/home/ubuntu-server#  
root@ubuntu-server:/home/ubuntu-server# ls perseo-openfmb/  
README.md config docker-compose.yml emulators hmi manuals meter_modbusTCP modbus-master schemas timescaledb.sql  
root@ubuntu-server:/home/ubuntu-server#  
root@ubuntu-server:/home/ubuntu-server#  
root@ubuntu-server:/home/ubuntu-server#  
root@ubuntu-server:/home/ubuntu-server#  
root@ubuntu-server:/home/ubuntu-server#  
root@ubuntu-server:/home/ubuntu-server#  
root@ubuntu-server:/home/ubuntu-server#  
root@ubuntu-server:/home/ubuntu-server#  
root@ubuntu-server:/home/ubuntu-server#  
root@ubuntu-server:/home/ubuntu-server#  
root@ubuntu-server:/home/ubuntu-server#
```

Figura 48: Listado de directorios y ficheros copiados en la carpeta ‘/home/ubuntu-server/perseo-openfmb’.

Se debe tener en cuenta que, si se desea extraer el dispositivo, debe desmontarse antes para evitar daños en el hardware o pérdida de información. El comando es el siguiente:

- `sudo umount /dev/sdb1`

VI. Ejecutar docker compose como se explicó en la **Sección 1.3**, se debe observar el despliegue de la plataforma como en la Figura 49.

```
root@ubuntu-server:/home/ubuntu-server/demo_openfmb_j/openfmb.demo-develop# ls  
LICENSE README.md docker-compose.yml replay start-all.sh stop-all.sh  
root@ubuntu-server:/home/ubuntu-server/demo_openfmb_j/openfmb.demo-develop# docker-compose up  
WARN[0001] Found orphan containers ([openfmbdemo-develop-hmi-1]) for this project. If you removed or renamed this service  
command with the --remove-orphans flag to clean it up.  
[+] Running 2/0  
✔ Container openfmbdemo-develop-nats-1 Created  
✔ Container openfmbdemo-develop-replay-1 Created  
Attaching to openfmbdemo-develop-nats-1, openfmbdemo-develop-replay-1  
openfmbdemo-develop-nats-1 | [1] 2024/07/03 20:31:25.448494 [INF] Starting nats-server  
openfmbdemo-develop-nats-1 | [1] 2024/07/03 20:31:25.463074 [INF] Version: 2.10.16  
openfmbdemo-develop-nats-1 | [1] 2024/07/03 20:31:25.463079 [INF] Git: [80e29794]  
openfmbdemo-develop-nats-1 | [1] 2024/07/03 20:31:25.463081 [INF] Cluster: my_cluster  
openfmbdemo-develop-nats-1 | [1] 2024/07/03 20:31:25.463083 [INF] Name: NAAKUPPQTOXF3JYLKM4DM4UUHJAJCZDLE5QQFD  
openfmbdemo-develop-nats-1 | [1] 2024/07/03 20:31:25.463085 [INF] ID: NAAKUPPQTOXF3JYLKM4DM4UUHJAJCZDLE5QQFD  
openfmbdemo-develop-nats-1 | [1] 2024/07/03 20:31:25.463095 [INF] Using configuration file: nats-server.conf  
openfmbdemo-develop-nats-1 | [1] 2024/07/03 20:31:25.478449 [INF] Starting http monitor on 0.0.0.0:8222  
openfmbdemo-develop-nats-1 | [1] 2024/07/03 20:31:25.480184 [INF] Listening for client connections on 0.0.0.0:4222  
openfmbdemo-develop-nats-1 | [1] 2024/07/03 20:31:25.480302 [INF] Server is ready  
openfmbdemo-develop-nats-1 | [1] 2024/07/03 20:31:25.481108 [INF] Cluster name is my_cluster  
openfmbdemo-develop-nats-1 | [1] 2024/07/03 20:31:25.481133 [INF] Listening for route connections on 0.0.0.0:6222  
openfmbdemo-develop-replay-1 | [2024-07-03 20:31:26.041] [application] [info] No configuration specified for adapter:  
openfmbdemo-develop-replay-1 | [2024-07-03 20:31:26.045] [application] [info] No configuration specified for adapter:  
openfmbdemo-develop-replay-1 | [2024-07-03 20:31:26.047] [application] [info] No configuration specified for adapter:  
openfmbdemo-develop-replay-1 | [2024-07-03 20:31:26.048] [application] [info] No configuration specified for adapter:
```

Figura 49: Ejecución de servidor nats y adaptador principal desde Ubuntu Server.

No es necesario definir el servicio hmi del fichero docker-compose configurado, el contenido de este fichero para esta instancia se muestra en la Figura 50.

```

GNU nano 7.2
version: "3.9"
services:
  nats:
    image: nats
    ports:
      - "4222:4222"
    expose:
      - 4222
  replay:
    image: oesinc/openfmb.adapters:a30d1d0
    depends_on:
      - nats
    volumes:
      - ./replay:/cfg
    command: -c /cfg/adapter.yaml

```

Figura 50: Fichero docker-compose.yml lanzado en Ubuntu Server.

4.4. Ejecutar la interfaz humano máquina (HMI) desde el anfitrión.

Esto permitirá visualizar los datos precargados del demo se debe establecer conexión con el servidor nats lanzado desde Ubuntu Server. Para eso se debe crear un directorio hmi como el que trae el demo por defecto, sin embargo, se requiere de algunas configuraciones adicionales que permitan direccionar el servicio nats que corre por la máquina virtual. Se recomienda seguir estos pasos:

- I. Crear un directorio con el nombre 'hmi' en una carpeta del anfitrión, en Windows, en este caso se ubica en '/C/Users/Usuario/Documents/hmi'.
- II. Se genera dentro del directorio hmi un fichero llamado 'app.toml' con el siguiente [contenido provisto por OpenFMB](#):

```

[hmi]
app_name = "OpenFMB HMI"           # name of the application
environment = "dev"                # dev or prod referring to nats.dev_uri or nats.prod_uri
log-level = "Debug"                # Off, Error, Info, Debug, Trace
# server_host = "0.0.0.0"           # For docker run, this should be "0.0.0.0", for local dev
# this needs to match the host computer's ip address
# server_port = 80                  # default is 443 for https and 80 for http
# comment out these two ssl related items if TLS is needed
# ssl_cert = "/server/certs/server/server-cert.pem"
# ssl_key = "/server/certs/server/server-key.pem"

[nats]
prod_uri = "10.0.0.1:4222"         # NATS broker connection URL. IF environment = "prod", t
dev_uri = "192.168.86.1:4222"      # NATS broker connection URL. IF environment = "dev", t
# authentication_type = "creds"    # comment out to enable NATS credential authentication ty
# creds = "meter.creds"            # comment out to specify NATS credentials file

```

Figura 51: Fichero 'app.toml' propuesto por la documentación OpenFMB.

El siguiente paso es modificar la línea 'dev uri' indicando la IP y el puerto de la máquina virtual que tiene en ejecución el proceso nats de donde provienen los datos que se desean mostrar en el HMI.

```

app.toml x
C:\Users\Usuario\Documents> hmi > app.toml
1 [hmi]
2 app_name = "OpenFMB HMI"           # name of the application
3 environment = "dev"                # dev or prod referring to nats.dev_uri or nats.prod_uri accordingly
4 log-level = "Debug"                # Off, Error, Info, Debug, Trace
5 # server_host = "0.0.0.0"           # For docker run, this should be "0.0.0.0", for local debugging,
6                                     # this needs to match the host computer's ip address
7 # server_port = 80                  # default is 443 for https and 80 for http
8 # comment out these two ssl related items if TLS is needed
9 # ssl_cert = "/server/certs/server/server-cert.pem"
10 # ssl_key = "/server/certs/server/server-key.pem"
11
12 [nats]
13 prod_uri = "10.0.0.1:4222"         # NATS broker connection URL. IF environment = "prod", this URL shall be use
14 dev_uri = "169.254.213.250:4222"  # NATS broker connection URL. IF environment = "dev", this URL shall be u
15 # authentication_type = "creds"    # comment out to enable NATS credential authentication type
16 # creds = "meter.creds"            # comment out to specify NATS credentials file

```

Figura 52: Fichero ‘app.toml’ editado para recibir la datos del servicio nats en la máquina virtual.

El siguiente paso es agregar los otros componentes que se necesitan en el HMI para su funcionamiento. Estos son: el directorio diagrams y los ficheros equipment y users. Estos se pueden tomar de la carpeta hmi que viene con los archivos del demo, ya que se quiere visualizar en ejecución el diagrama del demo que por defecto ya trae creados y vinculados los elementos en dicho diagrama. Esta estructura se muestra en la Figura 53.

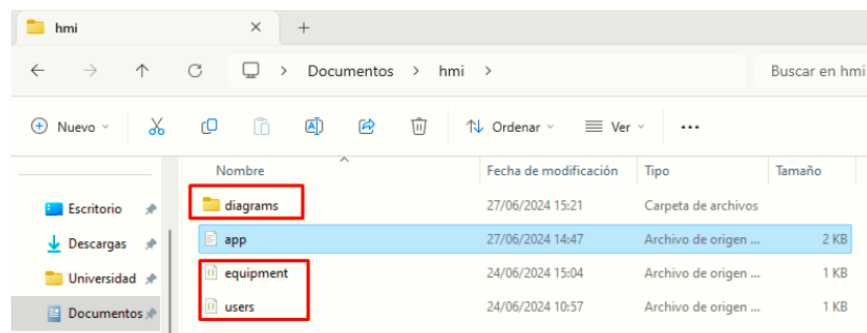


Figura 53: Estructura del directorio ‘hmi’ para visualizar datos precargados del demo.

4.5. Ejecución de contenedor HMI desde host Windows.

Esto permite acceder a la interfaz web de trabajo con los diagramas interactivos de OpenFMB. Para ejecutar el HMI se ejecuta el siguiente comando:

- `docker run -d -p 80:80 -e APP_CONF=/server/app.toml -e APP_DIRNAME=/server -v /C:/Users/Usuario/Documents/oesinc/openfmb.hmi`

En este comando se especifican variables de entorno como el archivo de configuración y el directorio de trabajo del contenedor HMI, dicionalmente se utiliza la bandera `-v` para establecer como volumen del contenedor el directorio hmi que creó en la figura 52.

En Windows se inicia primero docker desktop para poder ejecutar el comando. El resultado es como se ve en las figuras 47, 48, 49 y 50.

```
# Usuario @ DESKTOP-4EC3332 in ~\Desktop [15:41:57]
$ docker run -d -p 80:80 -e APP_CONF=/server/app.toml -e APP_DIR_NAME=/server
oesinc/openfmb.hmi
e1dafee24f6eb251e7f13847856c66d1b351af9b089bd45561c61b4aab32374e
# Usuario @ DESKTOP-4EC3332 in ~\Desktop [15:45:02]
$
```

Figura 54: Ejecución del HMI desde host Windows dentro del mismo segmento de red.

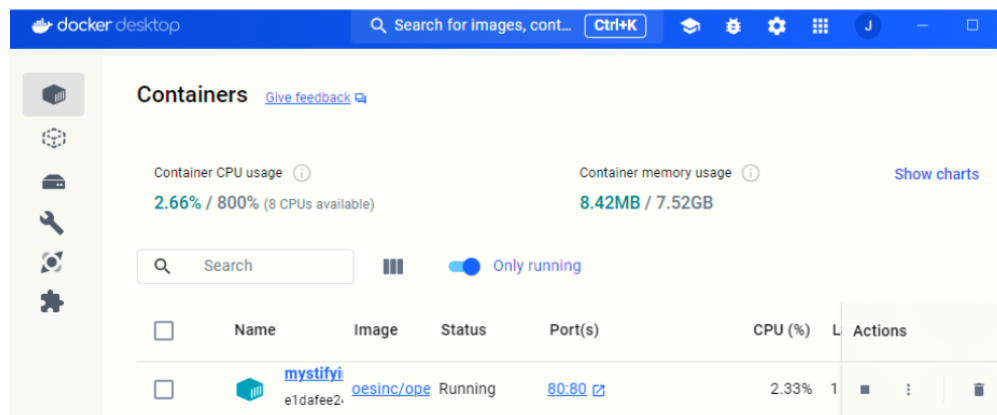


Figura 55: Contenedor HMI en ejecución.

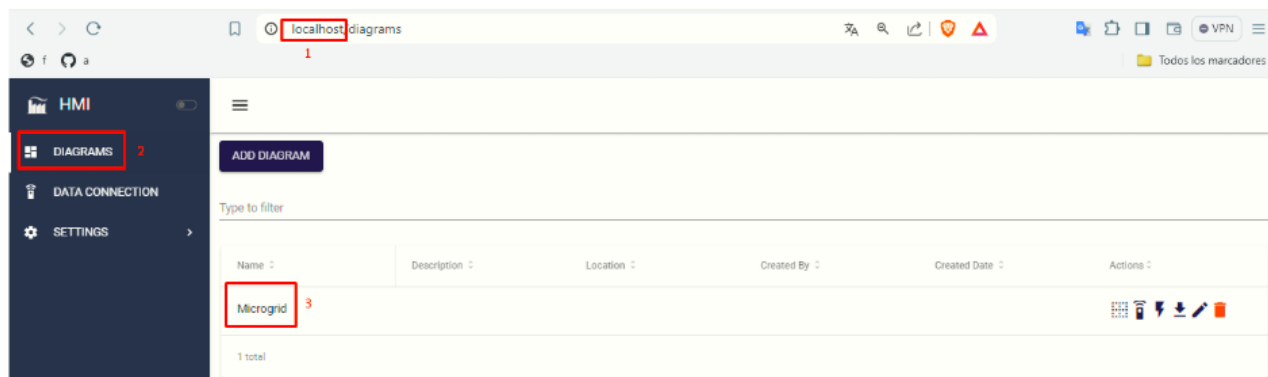


Figura 56: Entorno web visible desde navegador escribiendo 'local host'.

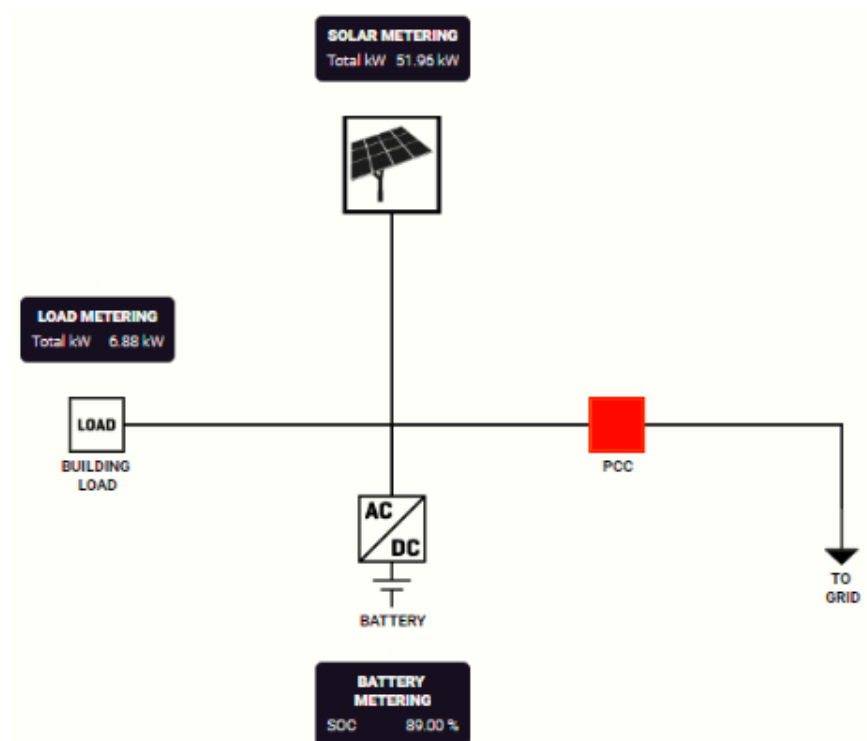


Figura 57: Ejecución del diagrama del demo OpenFMB desde otro nodo de red.

5. Medidores A9MEM3155 en cascada con OpenFMB.

5.1. Conexión física de medidores en cascada a un mismo adaptador RS485 a Ethernet.

El protocolo modbus permite utilizar un mismo adaptador RS485 TO ETH para hacer lectura de los registros de dos medidores conectados al mismo bus RS485 con diferente identificador o ID esclavo. A esto se refiere la conexión de medidores en cascada. Estos comparten una misma IP y puerto, además, el diferenciador de los dispositivos es el número de esclavo. Esto se puede hacer de manera manual como se explica en las **Secciones 2.1 a 2.3** o escribiendo en el registro 6501 del medidor. Para configurar el número de esclavo manualmente desde el medidor A9MEM3155 se usan sus botones físicos ‘OK’ y ‘ESC’, como se indica en estas mismas secciones.

Luego de estas configuraciones físicas en cada medidor por separado, siguen las conexiones al mismo adaptador RS485 a Ethernet. Para esto se conectan en paralelo de los medidores. Se conectan los bornes D1+, D1- y 0V de cada medidor a los bornes A+, A- y GND respectivamente, como se muestra en la figura 58.

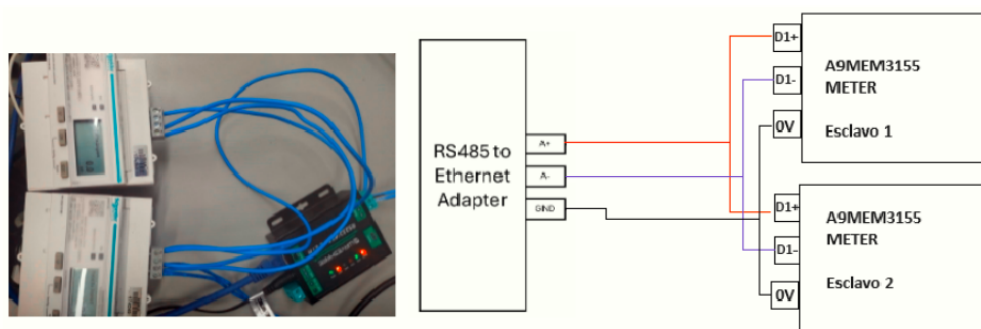


Figura 58: Conexión de 2 medidores A9MEM3155 a un adaptador RS485 to ETH.

5.2. Configurar multi-esclavo en adaptador RS485 a Ethernet.

El adaptador RS485 TO ETH presenta algunas limitas limitaciones para hacer lectura de registros de ambos esclavos al ‘mismo tiempo’. Esto se refiere al mismo tiempo de muestreo en los archivos de configuración de ambos esclavos, lo que se traduce en paquetes ethernet siendo enviados con muy poca separación temporal entre ellos. Esto no permite que por defecto el proceso de envío de peticiones desde el adaptador y las correspondientes respuestas desde los esclavos se hagan correctamente. Para corregir este comportamiento se debe agregar una configuración adicional en la interfaz wev del adaptador RS485 a Ethernet, tal como se muestra en la figura 59.

Figura 59: Configuración de modo Modbus Poll en adaptador RS485 a Ethernet.

Se debe ingresar en la interfaz web que provee el adaptador RS485 to ETH por medio de un navegador, y escribiendo la dirección IP por efecto 192.168.0.7. En la opción ‘RS485’se habilita el modo ‘Modbus Poll’ y se establece el tiempo mínimo de respuesta en 21 milisegundos ya que para tiempos inferiores no se completa correctamente la lectura de registros del(los) esclavo(s).

5.3. ‘Templating’ en OpenFMB para escalar el número de medidores A9MEM3155.

Como se explica en las **Secciones 2.7 y 2.8**, los ficheros de configuración para el adaptador y el plugin **modbus-master**, permiten definir los dispositivos físicos que se quieren vincular a la plataforma, la configuración por defecto de estos elementos de OpenFMB se pueden observar en las Figuras 18 y 19 de las mismas secciones. Para reutilizar los ficheros de configuración e incluir un nuevo medidor (que se ha conectado en cascada con el primero), se puede utilizar el ‘Templating’ que, en este caso, aprovecha que el nuevo dispositivo es un medidor, cuyas variables ya han sido mapeadas a un perfil de lectura en la plataforma, como se muestra en la Figura 22, en la **Sección 2.8**. A continuación se explica como modificar estos ficheros para usar ‘Templating’ y vincular un nuevo medidor A9MEM3155 en cascada con el anterior.

- I. Modificar el fichero de configuración del plugin **modbus-master**: se deben establecer las propiedades que serán diferentes para cada esclavo en el valor “?” y mantener iguales las variables que no cambian en ambos medidores, por ejemplo la dirección IP, el mapeo de registros, etc. Estas modificaciones se muestran en la siguiente ilustración.

```

! modbus-master-template.yaml X
perseo-openfmb-develop > modbus-master > ! modbus-master-template.yaml > [ ] pr
1  name: device1
2  log-level: Warn
3  adapter: 0.0.0.0
4  remote-ip: 192.168.0.7
5  port: 26
6  # unit-identifier: 1
7  unit-identifier: "?"
8  response_timeout_ms: 1000
9  always-write-multiple-registers: false
10 auto_polling:
11   max_register_gaps: 0
12   max_bit_gaps: 0
13 heartbeats: []
14 file:
15   id: openfmb-adapter-template
16   edition: 2.1
17   version: 2.1.0.0
18   plugin: modbus-master
19 profiles:
20 - name: MeterReadingProfile
21   # poll_period_ms: 1800
22   poll_period_ms: "?"
23 mapping:
24   readingMessageInfo:
25     messageInfo:
26     identifiedObject:
27       description:
28         value:
29           string-field-type: ignored
30       mRID:
31         value:
32           string-field-type: generated_uuid
33       name:
34         value:
35           string-field-type: ignored
36       messageTimeStamp:
37         timestamp-field-type: ignored
38 meter:
26   file:
18   plugin: modbus-master
19 profiles:
20 - name: MeterReadingProfile
21   # poll_period_ms: 1800
22   poll_period_ms: "?"
23 mapping:
24   readingMessageInfo:
25     messageInfo:
26     identifiedObject:
27       description:
28         value:
29           string-field-type: ignored
30       mRID:
31         value:
32           string-field-type: generated_uuid
33       name:
34         value:
35           string-field-type: ignored
36       messageTimeStamp:
37         timestamp-field-type: ignored
38 meter:
39   conductingEquipment:
40     namedObject:
41       description:
42         value:
43           string-field-type: ignored
44       name:
45         value:
46           string-field-type: ignored
47       mRID:
48         string-field-type: primary_uuid
49         # value: 38a23a75-d331-412d-a2da-690e14671fa5
50         value: "?"
51 meterReading:
52   conductingEquipmentTerminalReading:
53     terminal:

```

Figura 60: Parametrización del fichero de configuración del plugin **Modbus-master**.

Las propiedades modificadas con el valor “?” son: ‘unit-identifier’, ‘Poll period ms’ y ‘mRID’. Se debe tener en cuenta que estos son respectivamente el número de esclavo, el tiempo entre encuestas al dispositivo físico y la identificación interna en la plataforma, esto dado que los perfiles de datos de estos equipos tendrán la misma información de registros pero se deben identificar con ‘mRID’. Por su parte, la IP y el mapeo de variables será igual ya que ambos esclavos son accesibles con un mismo adaptador RS485 a Ethernet y tienen la misma estructura de registros internamente.

- II. Modificar el fichero de configuración del adaptador OpenFMB: En este fichero se instancian los dispositivos a vincular a la plataforma (previamente representados con un fichero **modbus-master**), para ello se modifica la propiedad **sessions** declarando dispositivos con la ruta relativa a su fichero de configuración y, estableciendo el valor de las respectivas variables declaradas con el valor “?” según el ‘Templating’ empleado en el caso anterior. Esto se muestra en la Figura 61.


```

! adapter.yaml x
perseo-openfmb-develop > config > ! adapter.yaml > {} plugins > {} modbus-master > @ enabled
15 plugins:
26   dnp3-outstation:
27     enabled: false
28     thread-pool-size: 1
29     outstations: []
30   modbus-master:
31     enabled: true
32     thread-pool-size: 10
33     # sessions:
34     # - path: /modbus-master/modbus-master-template.yaml
35     #   local-path: /modbus-master/modbus-master-template.yaml
36     #   session-name: Session
37     #   overrides: []
38     sessions:
39     - path: /modbus-master/modbus-master-template.yaml
40       local-path: /modbus-master/modbus-master-template.yaml
41       session-name: Session1
42       overrides:
43       - key: profiles[0].mapping.meter.conductingEquipment.mRID.value
44         value: 56a23a75-d331-412d-a2da-690e14671fa5
45       - key: unit-identifier
46         value: 1
47       - key: profiles[0].poll_period_ms
48         value: 1000
49     - path: /modbus-master/modbus-master-template.yaml
50       local-path: /modbus-master/modbus-master-template.yaml
51       session-name: Session2
52       overrides:
53       - key: profiles[0].mapping.meter.conductingEquipment.mRID.value
54         value: 81a91e5a-d880-4981-92f4-e5dcdb467fb4
55       - key: unit-identifier
56         value: 2
57       - key: profiles[0].poll_period_ms
58         value: 1000
59   modbus-outstation:
60     enabled: false

```

Figura 61: Uso de ‘Templating’ para instanciar 2 medidores A9MEM3155 en el adaptador OpenFMB.

En esta Figura se muestran dos instancias de elementos Modbus (Session1 y Session2). Cada una está compuesta por rutas para direccionar el respectivo fichero de configuración o ‘Template’, seguido de las propiedades ‘session-name’ y ‘overrides’. Cabe resaltar que ‘overrides’ es una lista que debe tener un ‘key’ y ‘value’ para todas las propiedades del template establecidas en “?”, donde el ‘key’ se construye partiendo del respectivo perfil con la variable en “?” y, cada nivel de indentación se representa con un punto(.) de esta manera se indica cual es la variable a la que se hace referencia, por su parte, el ‘value’ lleva el respectivo valor de dicha propiedad.

5.4. Ejecución de la plataforma y visualización de los 2 medidores en HMI de OpenFMB.

Luego de editar los ficheros de configuración, se pasa a trabajar en el HMI, donde se debe registrar el nuevo medidor indicando su respectivo ‘mRID’ y su tipo ‘meter’, adicionalmente se deben agregar nuevos componentes gráficos al diagrama de lectura anterior para visualizar la información del nuevo medidor. Estos pasos se han explicado previamente en la **Sección 2.12**, allí se detalla el registro del nuevo dispositivo en el HMI al igual que el mapeo de variables de un perfil específico a componentes de un diagrama por medio del ‘mRID’. Se deja la siguiente ilustración de este esquema de 2 medidores en cascada proporcionando datos a un diagrama interactivo del HMI de OpenFMB.



Figura 62: Visualización de datos de 2 esclavos modbus conectados en cascada.

6. Escritura en registros de emulador Modbus Pal con OpenFMB.

La plataforma OpenFMB permite también la escritura de registros Modbus mediante la configuración de otros elementos, estas configuraciones se detallan en esta sección. Para ilustrar este modo de operación con la plataforma (escritura de registros), se considera un entorno de pruebas que tiene una máquina virtual con sistema operativo Linux ejecutándose en un host Windows. Los componentes de la plataforma OpenFMB se ejecutan en el entorno virtualizado, donde además se ejecuta un script en Python para generar comandos de escritura Modbus. Estos son leídos por el plugin modbus-outstation de OpenFMB, que luego las publica al servidor NATS de OpenFMB, y este a su vez envía estas peticiones al plugin **modbus-master**. Este último finalmente traduce las peticiones a comandos de escritura para modificar el valor de un registro específico, el cual está alojado en el simulador Modbus Pal. Cabe anotar que este simulador se encuentra en ejecución en el host Windows. El entorno de pruebas descrito se ilustra en la figura 63.

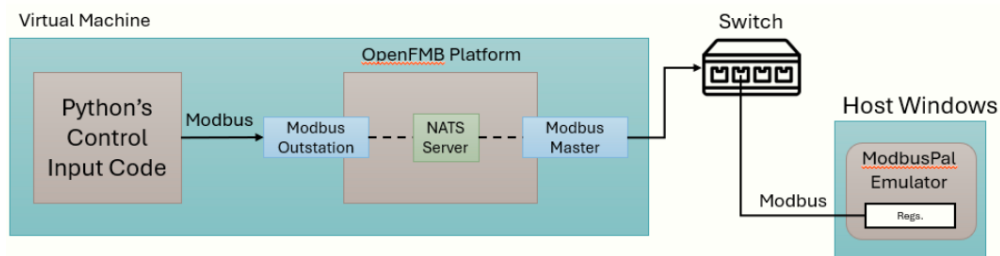


Figura 63: Esquema para pruebas de escritura de registros Modbus con OpenFMB y Modbus Pal.

6.1. Direcccionamiento IP para máquina virtual linux y host Windows.

La máquina virtual y el host Windows deben tener interfaces en el mismo segmento de red. Para esto se configura desde VirtualBox un adaptador de red en modo puente, como se muestra en las figuras 64 y 65. El segmento de red utilizado en este caso es 192.168.0.0/24 y las direcciones IP para la máquina virtual (la interfaz de red del adaptador en modo puente) y el host Windows son respectivamente 192.168.0.5 y 192.168.0.12.

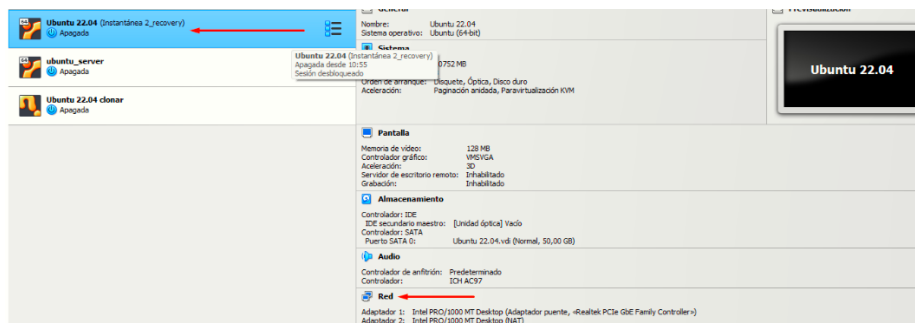


Figura 64: Configuración de adaptador de red para máquina virtual Linux.

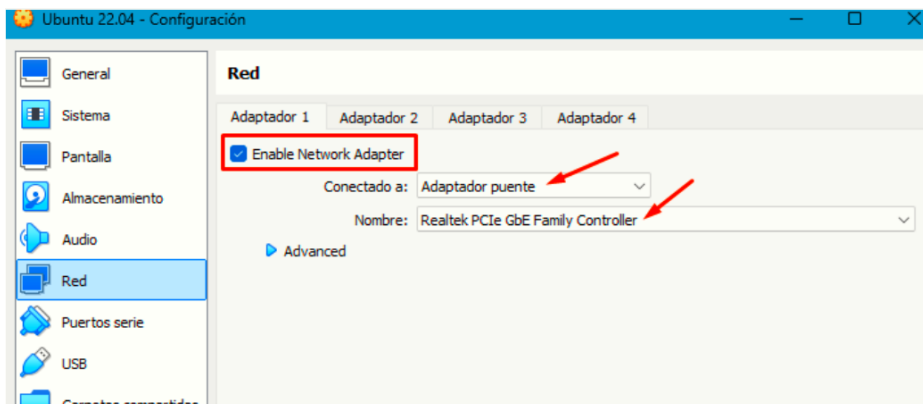


Figura 65: Selección de adaptador puente para la máquina virtual.

6.2. Obtener emulador Modbus Pal y creación de escalvos simulados.

ModbusPal emula un dispositivo Modbus con esclavos y registros reales, es un aplicativo sencillo que se ejecuta fácilmente en Linux y Windows. Este programa se puede obtener gratuitamente en [este enlace](#), y se descarga y ejecuta el archivo como se muestra en la Figura 66, en la Figura 67 se muestra la ejecución del programa luego de descargarlo. Este simulador es una aplicación portable tipo JAR, por lo que hay que instalar JAVA en el host con Windows.

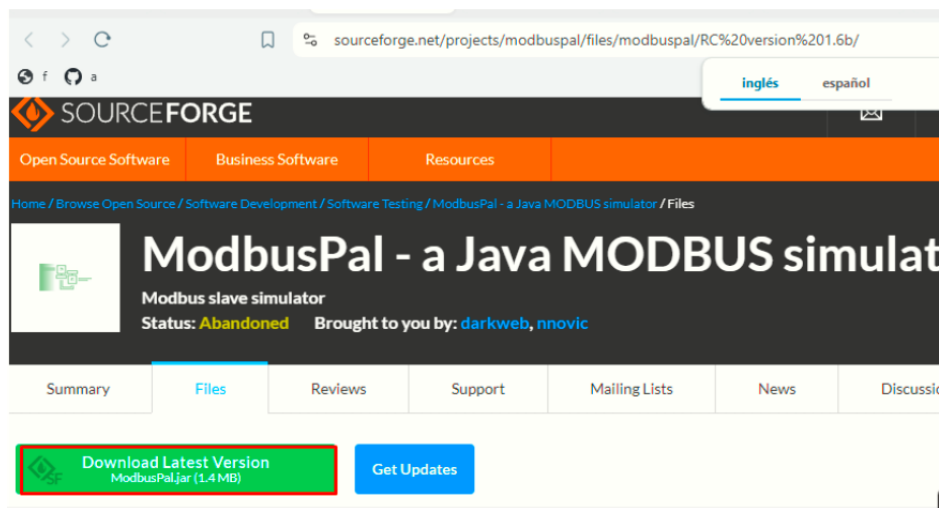


Figura 66: Descarga del simulador Modbus Pal.

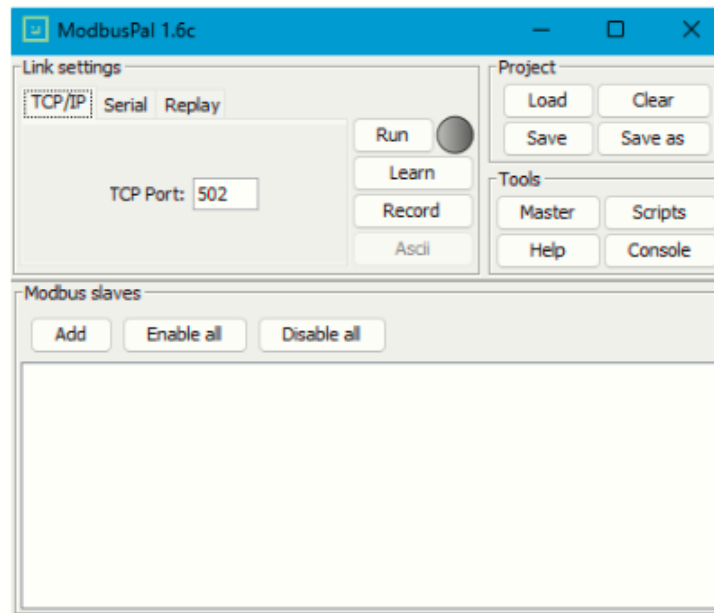


Figura 67: Interfaz gráfica del simulador Modbus Pal.

Luego de abrir Modbus Pal, se crea un nuevo esclavo dando click en la opción 'Add' de la sección 'Modbus slaves' y se recomienda llenar ambos campos en 1, como se muestra en la figura 68. El campo 'Add slave' indica el ID de esclavo de esta instancia (el número 1 en la figura 68). En las siguientes Figuras se muestra como establecer el rango de registros, habilitar y ejecutar el programa para que actúe como un esclavo real por un puerto del host.

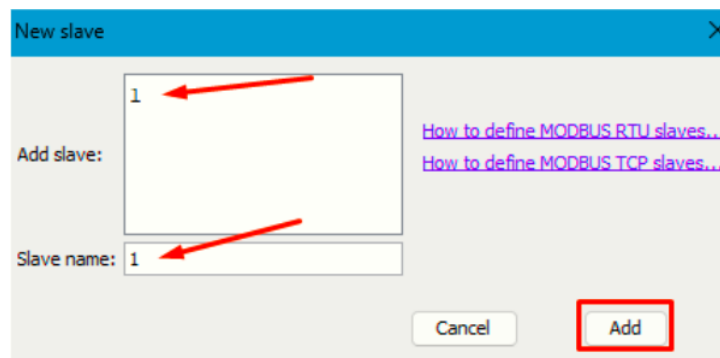


Figura 68: Creación de un esclavo en Modbus Pal.

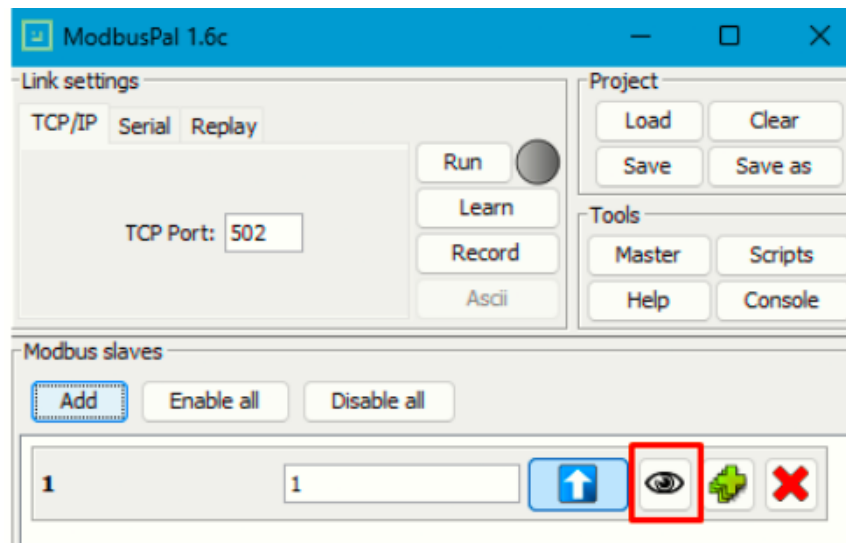


Figura 69: Opción para configurar registros de un esclavo en ModbusPal.

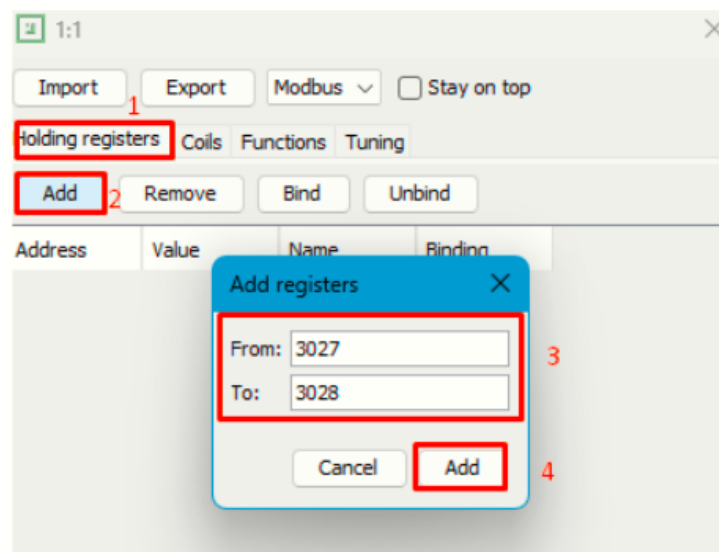


Figura 70: Creación de registros para un esclavo en Modbus Pal.

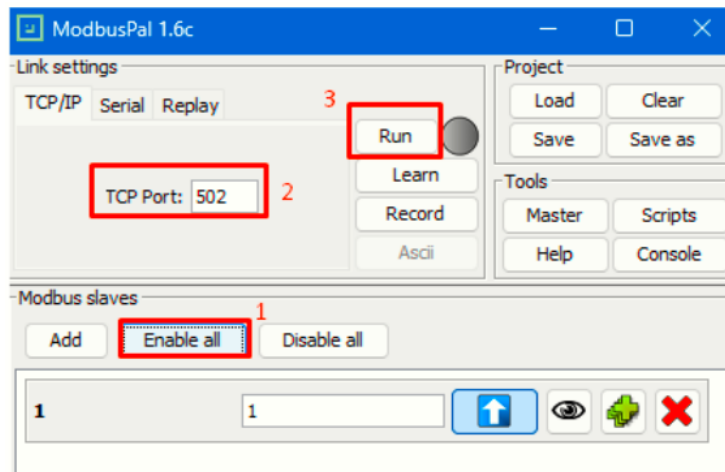


Figura 71: Ejecución del esclavo ModbusPal.

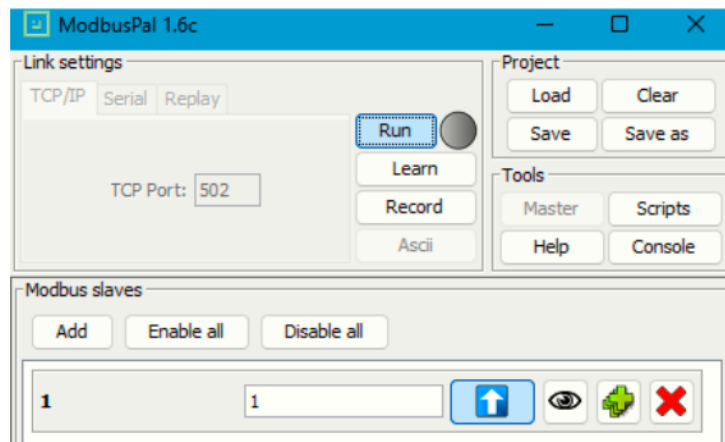


Figura 72: Visualización del esclavo ModbusPal en ejecución.

6.3. Ficheros de configuración con OACT para escritura con OpenFMB.

Anteriormente, en la **Sección 2.6**, se explicó el uso de la herramienta OACT para generar ficheros de configuración de OpenFMB y asignarles algunas propiedades en un entorno gráfico, en este caso se usará esta herramienta de nuevo para generar una estructura de ficheros nueva que permite establecer un escenario de prueba para la escritura de registros en el emulador Modbus Pal con OpenFMB.

Primero se debe generar el adaptador OpenFMB con los plugins modbus-master, modbus-outstation y nats. Luego se agregan las sesiones modbus con sus respectivos perfiles de control en cada plugin. Estos perfiles permiten que el adaptador interprete los mensajes internos de OpenFMB como solicitudes de escritura en los dispositivos físicos. Este procedimiento se ilustra en las figuras 73 a 83.

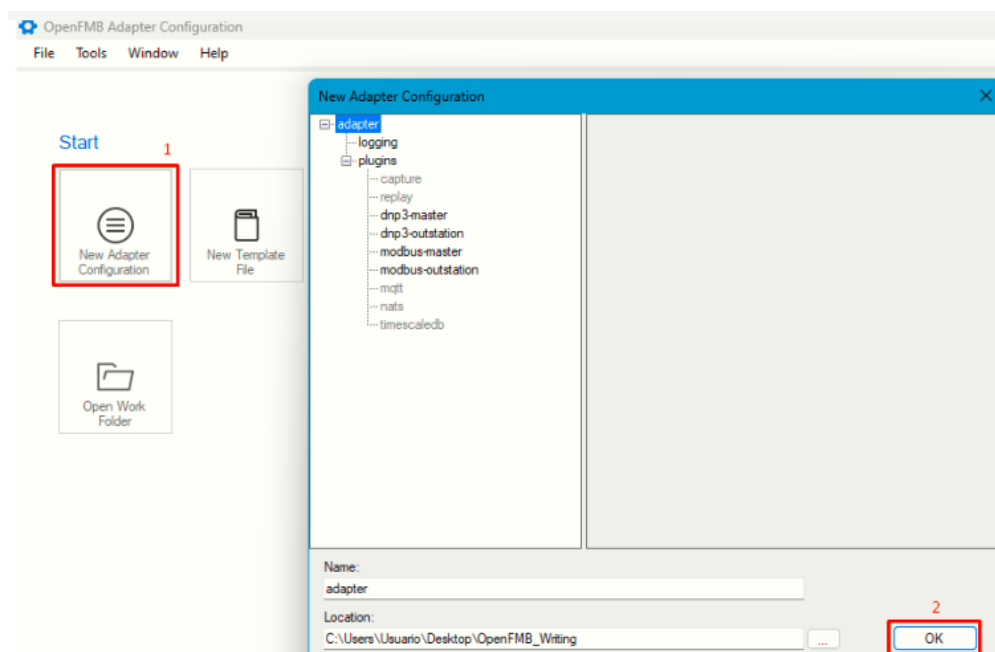


Figura 73: Creación de adaptador nuevo con herramienta OACT.

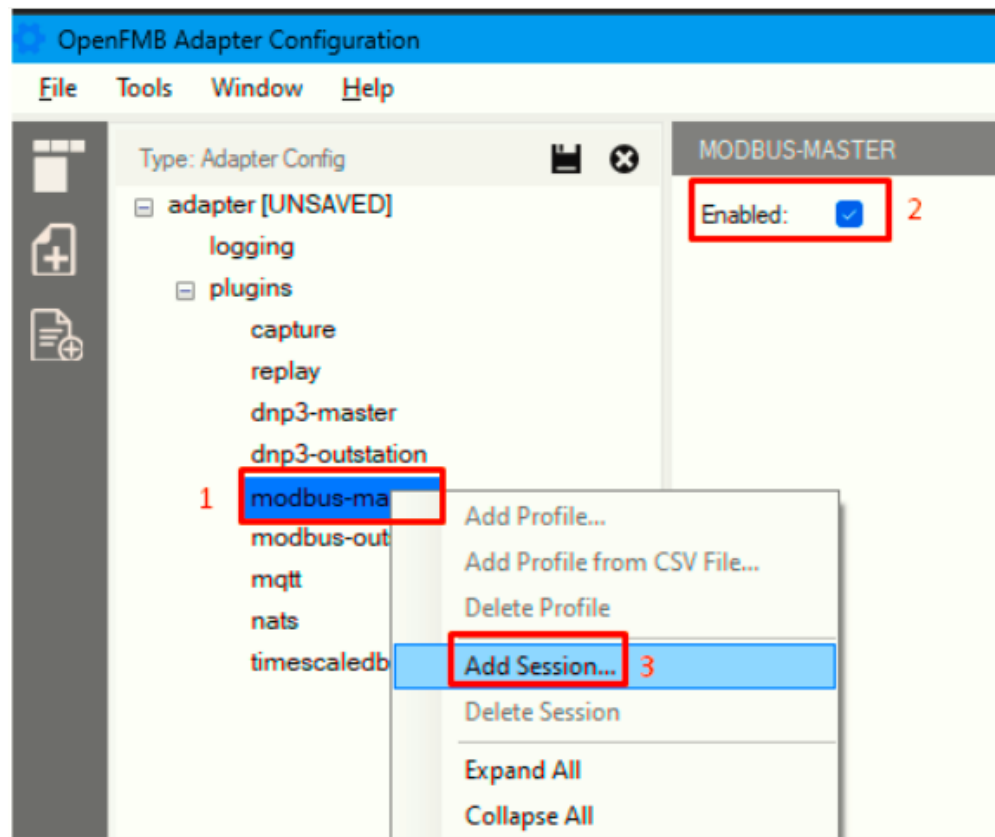


Figura 74: Habilitación del plugin **modbus-master**.

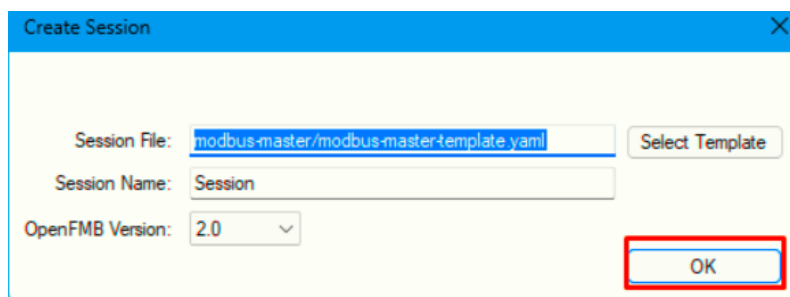


Figura 75: Creación nueva sesión para el plugin **modbus-master**.

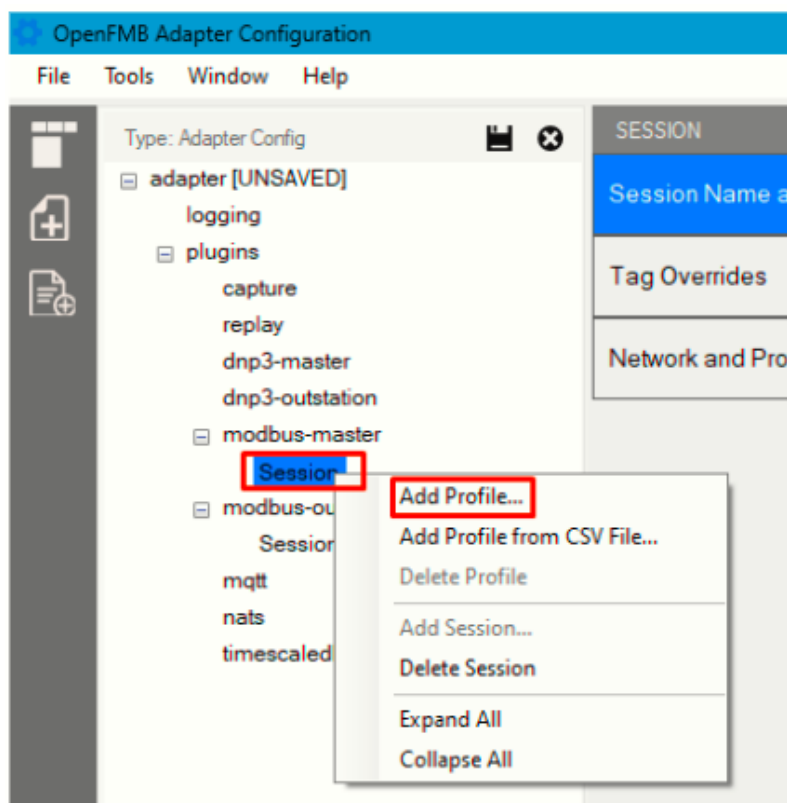


Figura 76: Agregando un perfil para el plugin **modbus-master**.

Select OpenFMB Profile ✕

2.0

	Profile Name
☐	BreakerDiscreteControlProfile
☐	BreakerEventProfile
☐	BreakerReadingProfile
☐	BreakerStatusProfile
☐	CapBankControlProfile
☒	CapBankDiscreteControlProfile
☐	CapBankEventProfile
☐	CapBankReadingProfile
☐	CapBankStatusProfile
☐	ESSControlProfile
☐	ESSEventProfile
☐	ESSReadingProfile
☐	ESSStatusProfile
☐	GenerationControlProfile

OK

Figura 77: Selección de un perfil de escritura en el plugin **modbus-master**.

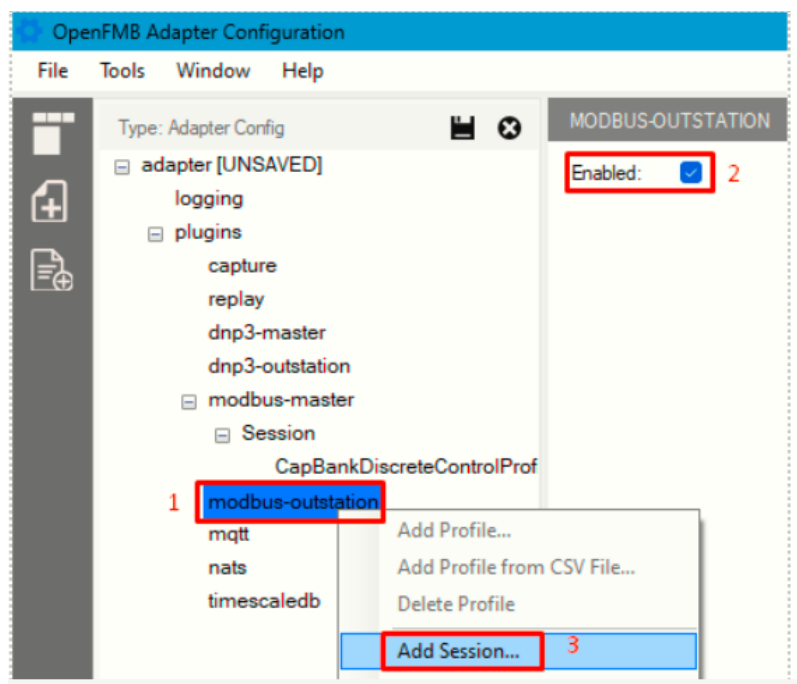


Figura 78: Habilitación de plugin **modbus-outstation**.

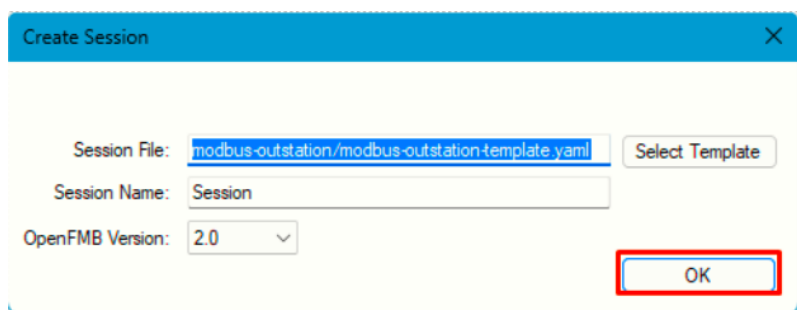


Figura 79: Creación nueva sesión para el plugin **modbus-outstation**.

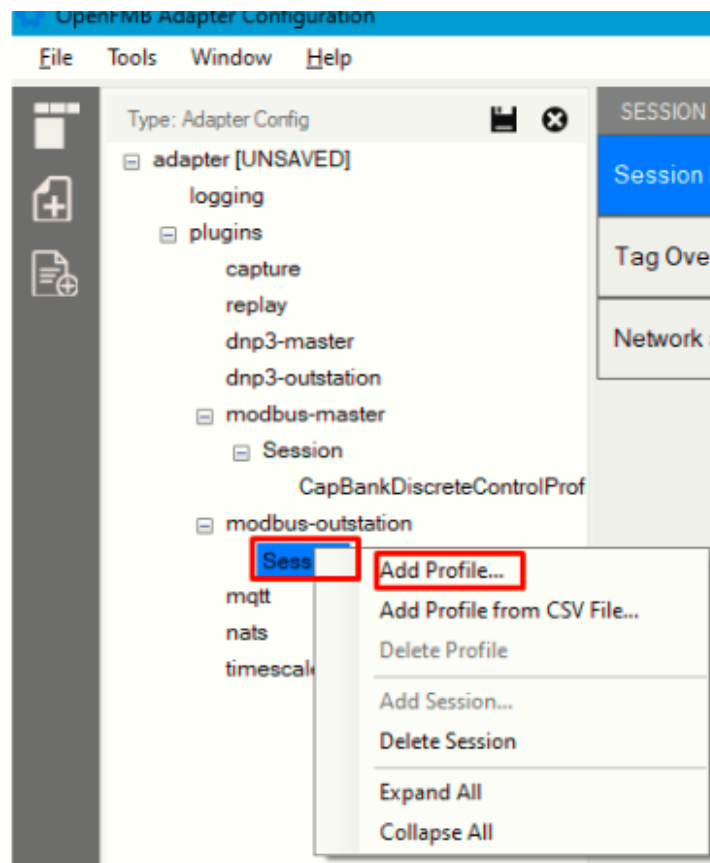


Figura 80: Agregando un perfil para el plugin **modbus-outstation**.

Select OpenFMB Profile ✕

2.0

	Profile Name
☐	BreakerDiscreteControlProfile
☐	BreakerEventProfile
☐	BreakerReadingProfile
☐	BreakerStatusProfile
☐	CapBankControlProfile
☒	CapBankDiscreteControlProfile
☐	CapBankEventProfile
☐	CapBankReadingProfile
☐	CapBankStatusProfile
☐	ESSControlProfile
☐	ESSEventProfile
☐	ESSReadingProfile
☐	ESSStatusProfile
☐	GenerationControlProfile

OK

Figura 81: Selección de un perfil de escritura para el plugin **modbus-outstation**.

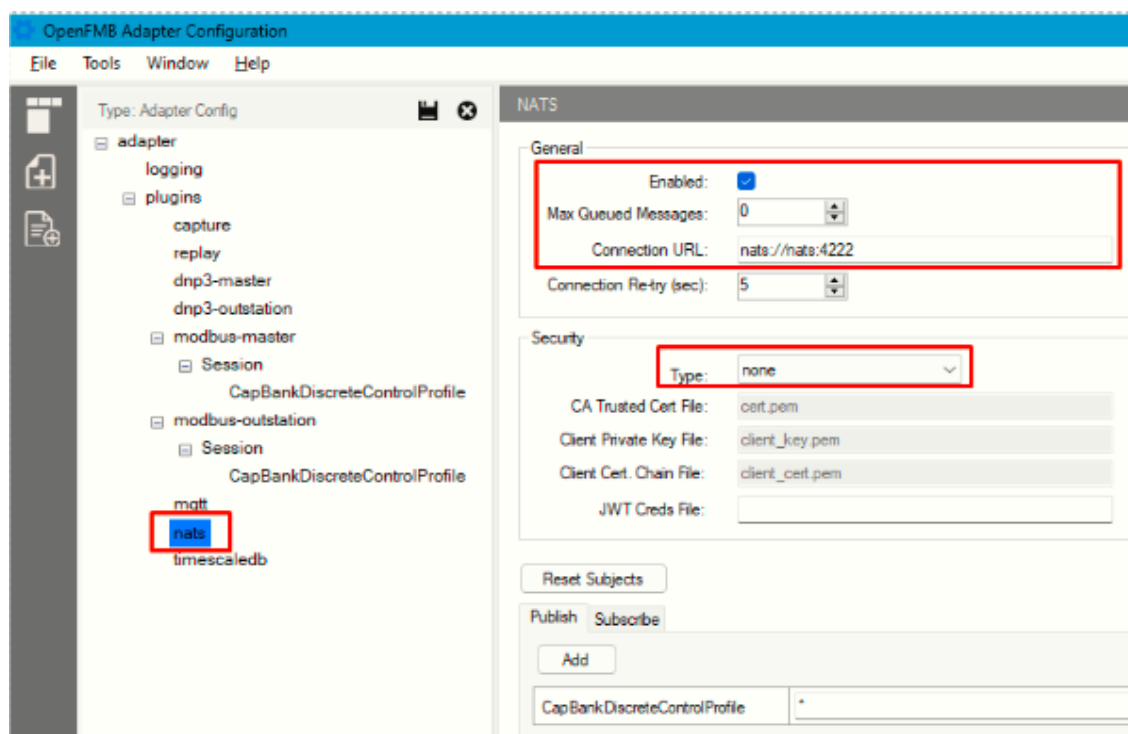


Figura 82: Habilitación plugin NATS.

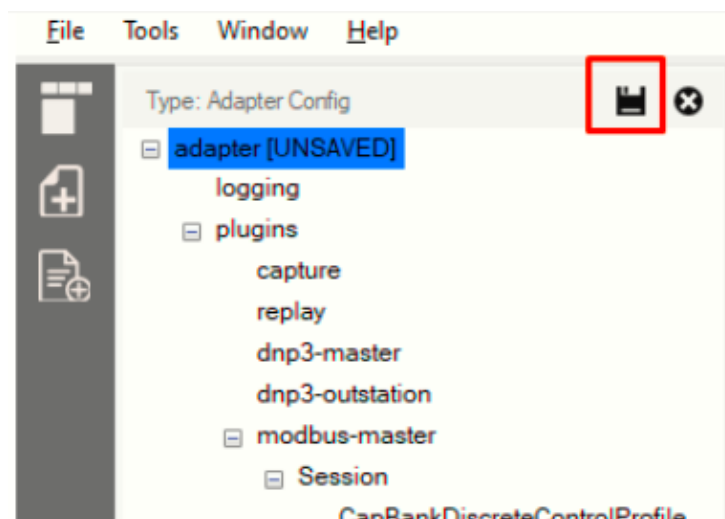


Figura 83: Guardando proyecto para generar directorio con ficheros de configuración.

6.4. Modificación de fichero del plugin modbus-outstation.

Este plugin permite al adaptador, escuchar comandos de escritura Modbus de una fuente externa y, publicarlos al servidor NATS de la plataforma OpenFMB. Las propiedades más relevantes por establecer son:

- Port: puerto por el que se escuchan comandos de escritura Modbus.
- Unit-identifier: ID del esclavo al que se dirige el comando.
- Perfil de control: modelo de datos propio de la plataforma para establecer información del mensaje de escritura que se publica en protocolo NATS.

Existen diferentes perfiles de escritura con el nombre ‘DiscreteControlProfile’. La elección del perfil depende del tipo de dato de los registros modbus que se leen o escriben. En esta prueba se establecen los valores ‘Port’ en 26 y ‘unit-identifier’ en 1, y el perfil de control escogido es ‘CapBankDiscreteControlProfile’. La configuración completa se muestra en la figura 84.

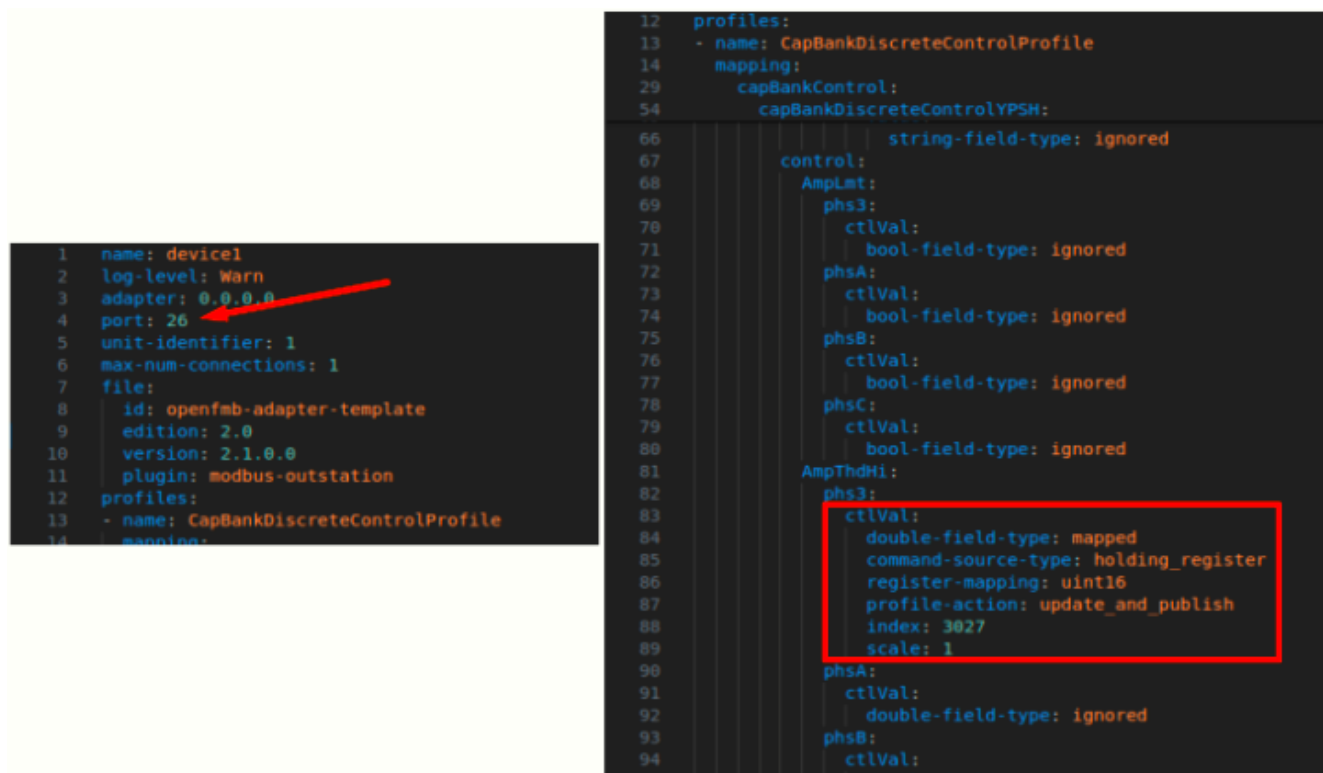


Figura 84: Configuración de plugin **modbus-outstation** con perfil de escritura.

6.5. Modificación del fichero del plugin modbus-master.

Este establece conexión con el servidor NATS para enviar o recibir mensajes de un perfil específico. Las propiedades más importantes en este fichero son:

- Remote-ip: dirección IP del dispositivo físico.
- Port: puerto por el que se escuchan comandos de escritura Modbus.
- Unit-identifier: ID del esclavo al que se dirige el comando.
- Perfil de control: modelo de dato de la plataforma que permite recibir mensajes de NATS y traducirlos a comandos de escritura modbus. La configuración de este perfil además debe incluir información del registro que se desea modificar. Algunos campos son el tipo de dato y el índice del registro y principalmente un ‘mRID’ que debe ser igual en ambos plugins, es decir **modbus-master** y **modbus-outstation**.

En este punto es importante mencionar que los perfiles ‘DiscreteControlProfile’ permiten escribir de manera directa en registros modbus, diferente a los perfiles ControlProfile que requieren configuraciones adicionales de “Scheduling”. Por esta razón se recomienda el uso de perfiles del tipo DiscreteControlProfile.

6.6. Fichero docker-compose.yml para escritura en Modbus Pal con OpenFMB.

Este fichero establece los servicios NATS y el adaptador OpenFMB que, adicionalmente, debe tener acceso a los ficheros de configuración creados en las Secciones 6.3 a 6.5, tal como se detalla en la figura 85.

```

documentation > docker-compose.yml > {} services > {} modbus >
docker-compose.yml - The Compose specification establishes a standard
1  version: "3.9"
2  services:
3    nats:
4      image: nats
5      ports:
6        - "4222:4222"
7      expose:
8        - 4222
9    modbus:
10     image: oesinc/openfmb.adapters:a30d1d0
11     ports:
12       - "26:26"
13     expose:
14       - 26
15     depends_on:
16       - nats
17     volumes:
18       - ./config:/cfg
19       - ./modbus-master:/modbus-master
20       - ./modbus-outstation:/modbus-outstation
21     command: -c /cfg/adapter.yaml

```

Figura 85: Descripción de contenedores OpenFMB en Docker Compose.

En la Figura 85 se muestra que el servicio de mensajería **NATS** opera por el puerto 4222, por su parte, el adaptador OpenFMB, con su nueva configuración de escritura, usa el puerto 26 para recibir estos comandos. La propiedad ‘volumes’, permiten copiar los ficheros de configuración generados al servicio del adaptador OpenFMB. Por último, se pone en ejecución el aplicativo del adaptador con la propiedad ‘command’.

6.7. Escritura de registros Modbus con Python.

Python cuenta con librerías para manejar el entorno Modbus, por ejemplo, generar servidores Modbus, simular esclavos y generar comandos de escritura. El script utilizado para esto se muestra en la Figura 86.

```

1  from pymodbus.client import ModbusTcpClient
2  import time
3
4  client = ModbusTcpClient("192.168.0.5", port=26, debug=True)
5
6
7  slaveNum = 1
8
9  client.connect()
10
11  regg = 20
12  # Writing:
13  while True:
14      result = client.write_register(3027, regg, slave=slaveNum)
15      time.sleep(0.5)
16      regg = regg + 1
17      if regg >= 2000:
18          regg = 0
19
20  client.close()

```

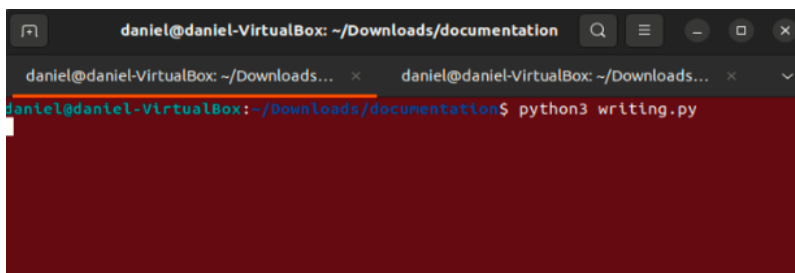
Figura 86: Script de Python para generar comandos de escritura Modbus.

En este script se define la dirección IP donde se encuentra el receptor de este comando, que en este caso, es la plataforma OpenFMB ejecutada en el PC con dirección IP ‘192.168.0.5’; También se define el puerto que como

se mencionó previamente es el 26. Por último, se emplea el método `write register` de la clase `ModbusTcpClient` que permite definir el registro a escribir y su respectivo valor.

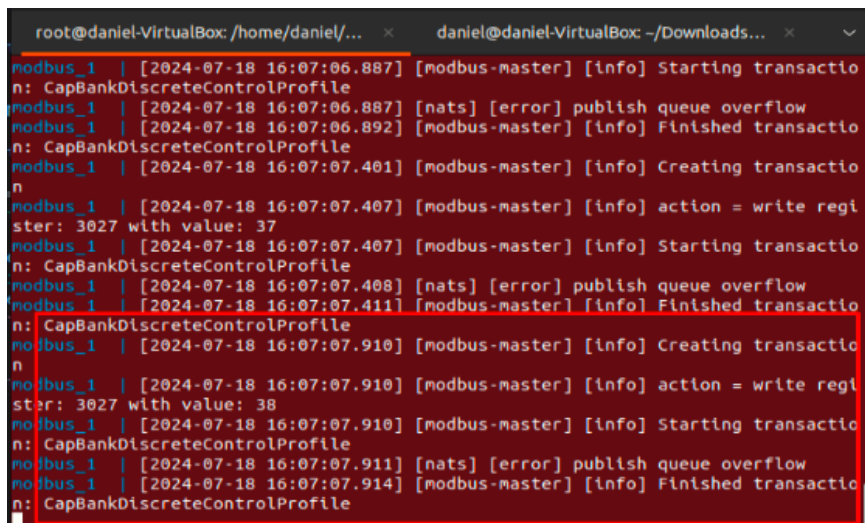
6.8. Ejecución de pruebas de escritura en Modbus Pal con OpenFMB.

Lo primero es verificar que el emulador Modbus Pal esté funcionando apropiadamente como se muestra en la Figura 72. La configuración y creación de esclavos se detalla en la **Sección 6.2**. Luego se ejecuta la plataforma OpenFMB abriendo una terminal en el directorio del proyecto, donde se encuentra el fichero `'docker-compose.yml'`, y ejecutando el comando `'docker compose up'`. Por último, se pone en ejecución el script de Python. En las Figuras 87, 88 y 89 se muestra la secuencia de eventos desde que se ejecuta el script de escritura con Python, luego la plataforma OpenFMB que recibe este comando y lo convierte en una transacción Modbus al respectivo registro.



```
daniel@daniel-VirtualBox: ~/Downloads/documentation
daniel@daniel-VirtualBox: ~/Downloads... x daniel@daniel-VirtualBox: ~/Downloads... x v
daniel@daniel-VirtualBox: ~/Downloads/documentation$ python3 writing.py
```

Figura 87: Ejecución del script Python generador de comandos Modbus.



```
root@daniel-VirtualBox: /home/daniel/... x daniel@daniel-VirtualBox: ~/Downloads... x v
modbus_1 | [2024-07-18 16:07:06.887] [modbus-master] [info] Starting transactio
n: CapBankDiscreteControlProfile
modbus_1 | [2024-07-18 16:07:06.887] [nats] [error] publish queue overflow
modbus_1 | [2024-07-18 16:07:06.892] [modbus-master] [info] Finished transactio
n: CapBankDiscreteControlProfile
modbus_1 | [2024-07-18 16:07:07.401] [modbus-master] [info] Creating transactio
n
modbus_1 | [2024-07-18 16:07:07.407] [modbus-master] [info] action = write regi
ster: 3027 with value: 37
modbus_1 | [2024-07-18 16:07:07.407] [modbus-master] [info] Starting transactio
n: CapBankDiscreteControlProfile
modbus_1 | [2024-07-18 16:07:07.408] [nats] [error] publish queue overflow
modbus_1 | [2024-07-18 16:07:07.411] [modbus-master] [info] Finished transactio
n: CapBankDiscreteControlProfile
modbus_1 | [2024-07-18 16:07:07.910] [modbus-master] [info] Creating transactio
n
modbus_1 | [2024-07-18 16:07:07.910] [modbus-master] [info] action = write regi
ster: 3027 with value: 38
modbus_1 | [2024-07-18 16:07:07.910] [modbus-master] [info] Starting transactio
n: CapBankDiscreteControlProfile
modbus_1 | [2024-07-18 16:07:07.911] [nats] [error] publish queue overflow
modbus_1 | [2024-07-18 16:07:07.914] [modbus-master] [info] Finished transactio
n: CapBankDiscreteControlProfile
```

Figura 88: Logs generados con los comandos de escritura Modbus desde Python.

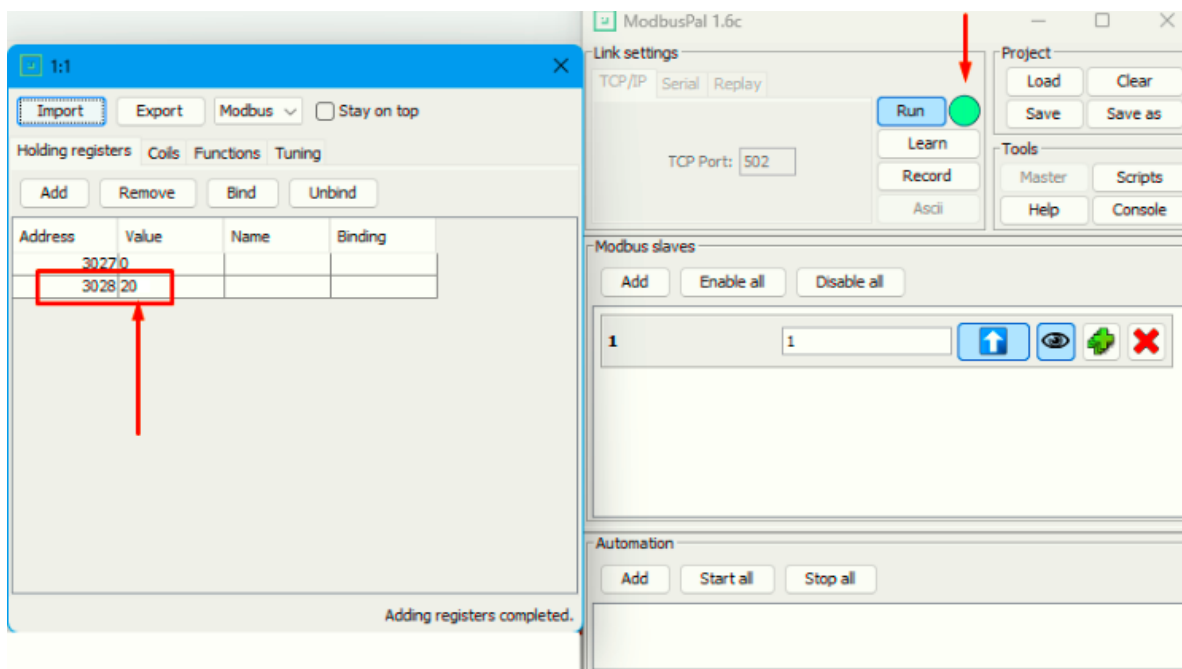


Figura 89: Visualización de escritura de registros Modbus en simulador Modbus Pal.

7. Control ON/OFF con OpenFMB + Raspberry PI 4 MODEL B.

La Raspberry PI 4 se puede programar como esclavo Modbus para recibir comandos de lectura y escritura de valores de tipo coil (true o false), esto con el fin de procesar peticiones que llegan desde la plataforma OpenFMB y luego ejecutar acciones de control por medio de sus GPIO's (por ejemplo, abrir o cerrar un relé). El entorno de pruebas descrito en esta sección configura la interacción de un script en Python, OpenFMB, Raspberry PI y circuito de relé.

7.1. Escritura de coils con Python.

Python permite generar comandos de escritura de coils por medio de la librería pymodbus. Los parámetros a ajustar en este script son: la dirección IP, puerto, índice del coil y el valor booleano que se desea escribir; los detalles de esta implementación se muestran en la siguiente Figura.


```

1  from pymodbus.client import ModbusTcpClient
2
3  def write_coil(ip, port, address, value):
4      client = ModbusTcpClient(ip, port)
5      client.connect()
6
7      # Escribir en el registro
8      client.write_coil(address, value, slave=1)
9
10     client.close()
11
12 def read_coil(ip, port, address):
13     client = ModbusTcpClient(ip, port)
14     client.connect()
15
16     # Leer el registro
17     result = client.read_coils(address, count=1, slave=1)
18     value = result.bits[0] if result.bits else None
19
20     client.close()
21     return value
22
23 # Dirección IP y puerto del servidor Modbus TCP
24 ip_address = '192.168.0.5'
25 port = 26 # Puerto Modbus TCP
26
27 # Dirección del registro que deseas escribir y leer
28 coil_address1 = 1 # Dirección de la primera bobina (coil)
29 coil_address2 = 2 # Dirección de la primera bobina (coil)
30 coil_address3 = 3 # Dirección de la primera bobina (coil)
31 coil_address4 = 4 # Dirección de la primera bobina (coil)
32
33 # Escribir en la bobina (encender el relé)
34 write_coil(ip_address, port, coil_address1, 1)
35 # Escribir en la bobina (encender el relé)
36 write_coil(ip_address, port, coil_address2, 1)
37 # Escribir en la bobina (encender el relé)
38 write_coil(ip_address, port, coil_address3, 1)
39 write_coil(ip_address, port, coil_address4, 1)
40
41 # Leer el estado de la bobina
42 state1 = read_coil(ip_address, port, coil_address1)
43 print("Estado del relé1:", state1)
44 # Leer el estado de la bobina
45 state2 = read_coil(ip_address, port, coil_address2)
46 print("Estado del relé2:", state2)
47 # Leer el estado de la bobina
48 state3 = read_coil(ip_address, port, coil_address3)
49 print("Estado del relé3:", state3)
50 state4 = read_coil(ip_address, port, coil_address4)
51 print("Estado del relé4:", state4)

```

Figura 90: Script de Python para escritura de coils de esclavo Modbus.

7.2. Configurar de Raspberry PI.

Lo primero es que la Raspberry tenga una interfaz de red en el mismo segmento que el PC que cuenta con la plataforma OpenFMB, en este caso se emplea el segmento 192.168.0.0/24. Posteriormente, se debe ejecutar en la Raspberry un script en Python con diferentes funciones: la inicialización de pines, el establecimiento del esclavo Modbus (con sus coils) y el cambio de estado de GPIOs con base en comandos Modbus. Los detalles de este código se muestran en las figuras 84, 85 y 86.

```

1  import pymodbus
2  import asyncio
3  import RPi.GPIO as GPIO
4
5  from pymodbus.datastore import ModbusSequentialDataBlock, ModbusServerContext, ModbusSlaveContext
6  from pymodbus.device import ModbusDeviceIdentification
7  from pymodbus.server.async_io import StartAsyncTcpServer
8
9  # Definición de los pines GPIO para los relés
10 rele1 = 3 # GPIO17
11 rele2 = 4 # GPIO27
12 rele3 = 14 # GPIO22
13 rele4 = 15 # GPIO14
14 reles = [rele1, rele2, rele3, rele4]
15
16 # Configuración inicial de los pines GPIO
17 GPIO.setwarnings(False)
18 GPIO.setmode(GPIO.BCM)
19 for pin in reles:
20     GPIO.setup(pin, GPIO.OUT, initial=GPIO.LOW)
21

```

Figura 91: Script de Python para Raspberry PI 4: configuración de librerías y pines.

```

22 # Funcion para controlar los relees
23 async def control_gpio(context):
24     while True:
25         # Leer los valores de los registros Modbus de las bobinas (coils)
26         coil_values = context[0].getValues(1, 1, count=4)
27         for i, value in enumerate(coil_values):
28             GPIO.output(reles[i], value)
29         await asyncio.sleep(1)
30
31 # Configuración del servidor Modbus
32 async def run_modbus_server():
33     # Configuración de los bloques de datos Modbus
34     store = ModbusSlaveContext(
35         di=ModbusSequentialDataBlock(0, [0]*100),
36         co=ModbusSequentialDataBlock(0, [0]*6), # Bobinas para controlar los relees
37         hr=ModbusSequentialDataBlock(0, [0]*100),
38         ir=ModbusSequentialDataBlock(0, [0]*100)
39     )
40     context = ModbusServerContext(slaves=store, single=True)
41

```

Figura 92: Script de Python para Raspberry PI 4: control de GPIOs.

```

42 # Configuración de la identificación del dispositivo
43 identity = ModbusDeviceIdentification()
44 identity.VendorName = 'Pymodbus'
45 identity.ProductCode = 'PM'
46 identity.VendorUrl = 'http://github.com/riptideio/pymodbus/'
47 identity.ProductName = 'Pymodbus Server'
48 identity.ModelName = 'Pymodbus Server'
49 identity.MajorMinorRevision = '1.0'
50
51 # Iniciar el controlador de GPIO en un hilo asincrono
52 gpio_task = asyncio.create_task(control_gpio(context))
53
54 # Iniciar el servidor Modbus TCP asincrono en el puerto 1502
55 await StartAsyncTcpServer(context, identity=identity, address=("0.0.0.0", 1504))
56
57 await gpio_task
58
59 if __name__ == "__main__":
60     loop = asyncio.get_event_loop()
61     loop.run_until_complete(run_modbus_server())
62

```

Figura 93: Script de Python para Raspberry PI 4: descripción de los elementos del esclavo Modbus.

En esta prueba se utilizó el software [RealVNC](#) en Windows que permite establecer conexión remota y configurar la Raspberry PI 4.

7.3. Crear ficheros de configuración de OpenFMB para control ON/OFF.

Al igual que en el caso anterior, donde se usaron los plugins **modbus-master** y **modbus-outstation**, y el perfil de escritura 'CapBankDiscreteControlProfile' para modificar el valor de un registro con OpenFMB, en esta sección se busca modificar el estado de una variable binaria o un relé. Para esto se pueden generar los mismos ficheros que en el caso de pruebas anterior, entonces se pueden seguir los pasos mostrados en las Figuras 73 a 83 para obtener dichos ficheros de configuración iniciales. La estructura de ficheros resultante debe ser como se muestra a continuación en las Figuras.

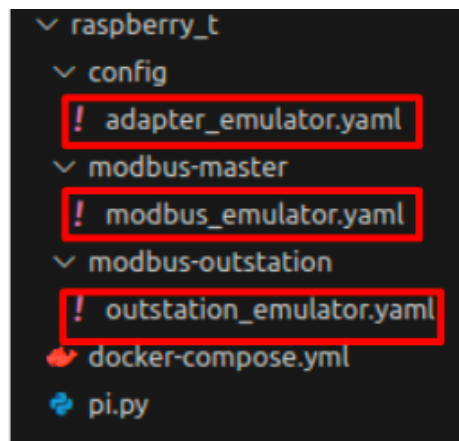


Figura 94: Estructura de ficheros de configuración OpenFMB.

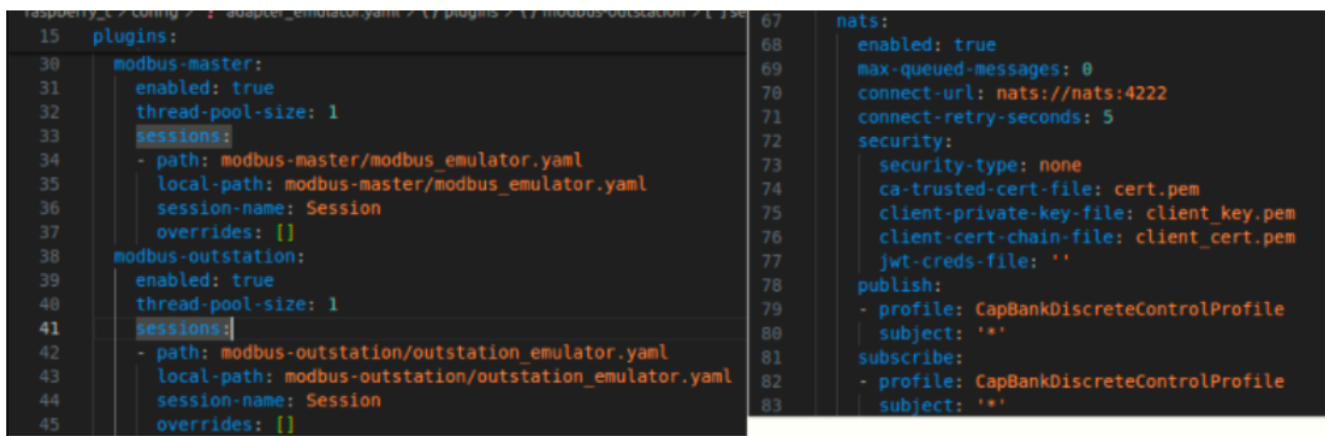


Figura 95: Configuración de plugins del adaptador OpenFMB

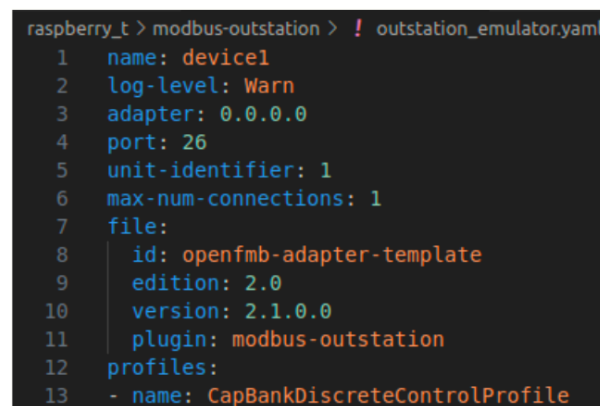


Figura 96: Configuración de plugin modbus-outstation.

```

14 mapping:
29   capBankControl:
54     capBankDiscreteControlYPSH:
109       bool-field-type: ignored
110       CtlModeOvrRd:
111         ctlVal:
112           bool-field-type: ignored
113       CtlModeRem:
114         ctlVal:
115           bool-field-type: ignored
116       DirMode:
117         value:
118           enum-field-type: ignored
119       Pos:
120         phs3:
121           ctlVal:
122             bool-field-type: mapped
123             command-source-type: coil
124             profile-action: update_and_publish
125             index: 1
126             negate: false
127         phsA:
128           ctlVal:
129             bool-field-type: mapped
130             command-source-type: coil
131             profile-action: update_and_publish
132             index: 2
133             negate: false
12 profiles:
13   - name: CapBankDiscreteControlProfile
14     mapping:
29       capBankControl:
54         capBankDiscreteControlYPSH:
133           negate: false
134         phsB:
135           ctlVal:
136             bool-field-type: mapped
137             command-source-type: coil
138             profile-action: update_and_publish
139             index: 3
140             negate: false
141         phsC:
142           ctlVal:
143             bool-field-type: mapped
144             command-source-type: coil
145             profile-action: update_and_publish
146             index: 4
147             negate: false
148         TempLmt:
149           ctlVal:
150             bool-field-type: ignored
151         TempThdHi:
152           ctlVal:
153             double-field-type: ignored
154         TempThdLo:

```

Figura 97: Configuración del perfil de escritura en plugin **modbus-outstation**.

```

modbus-master > ! modbus_emulator.yaml > [ ] profiles > {} 0
1  name: device1
2  log-level: Warn
3  adapter: 0.0.0.0
4  remote-ip: 192.168.0.50
5  port: 1504
6  unit-identifier: 1
7  response_timeout_ms: 1000
8  always-write-multiple-registers: false
9  auto_polling:
10   max_register_gaps: 0
11   max_bit_gaps: 0
12  heartbeats: []
13  file:
14   id: openfmb-adapter-template
15   edition: 2.0
16   version: 2.1.0.0
17   plugin: modbus-master
18  profiles:
19   - name: CapBankDiscreteControlProfile

```

Figura 98: Configuración del plugin **modbus-master**.

<pre> 18 profiles: 19 - name: CapBankDiscreteControlProfile 24 mapping: 39 capBankControl: 64 capBankDiscreteControlYPSH: 130 phs3: 131 ctlVal: 132 bool-field-type: mapped 133 when-true: 134 - output-type: write_single_coil 135 command-id: first 136 index: 1 137 value: true 138 when-false: 139 - output-type: write_single_coil 140 command-id: first 141 index: 1 142 value: false 143 phsA: 144 ctlVal: 145 bool-field-type: mapped 146 when-true: 147 - output-type: write_single_coil 148 command-id: first 149 index: 2 150 value: true 151 when-false: 152 - output-type: write_single_coil 153 command-id: first 154 index: 2 155 value: false </pre>	<pre> 18 profiles: 19 - name: CapBankDiscreteControlProfile 24 mapping: 39 capBankControl: 64 capBankDiscreteControlYPSH: 156 phsB: 157 ctlVal: 158 bool-field-type: mapped 159 when-true: 160 - output-type: write_single_coil 161 command-id: first 162 index: 3 163 value: true 164 when-false: 165 - output-type: write_single_coil 166 command-id: first 167 index: 3 168 value: false 169 phsC: 170 ctlVal: 171 bool-field-type: mapped 172 when-true: 173 - output-type: write_single_coil 174 command-id: first 175 index: 4 176 value: true 177 when-false: 178 - output-type: write_single_coil 179 command-id: first 180 index: 4 181 value: false </pre>
---	---

Figura 99: Configuración del perfil de escritura en plugin **modbus-master**.

```

raspberrypi_t > docker-compose.yml > version
1  version: "3.9"
2  services:
3    nats:
4      image: nats
5      ports:
6        - "4222:4222"
7      expose:
8        - 4222
9    modbus:
10     image: oesinc/openfmb.adapters:a30d1d0
11     ports:
12       - "26:26"
13     expose:
14       - 26
15     depends_on:
16       - nats
17     volumes:
18       - ./config/cfg
19       - ./modbus-master:/modbus-master
20       - ./modbus-outstation:/modbus-outstation
21     command: -c /cfg/adaptador_emulador.yaml

```

Figura 100: Fichero docker-compose para ejecutar proyecto OpenFMB.

7.4. ejecución del entorno de prueba OpenFMB + Raspberry PI 4.

Se propone el siguiente orden para ejecutar este entorno de pruebas de OpenFMB + Raspberry Pi 4. Primero se ejecuta el esclavo Modbus generado con Python en las Figuras 91, 92 y 93, luego se ejecuta la plataforma OpenFMB y, por último, se usa el script de Python de la **Sección 7.1**, Figura 90, para generar los comandos de escritura que OpenFMB recibe e implementa en la Raspberry Pi 4. Esta secuencia se ilustra en las siguientes Figuras.

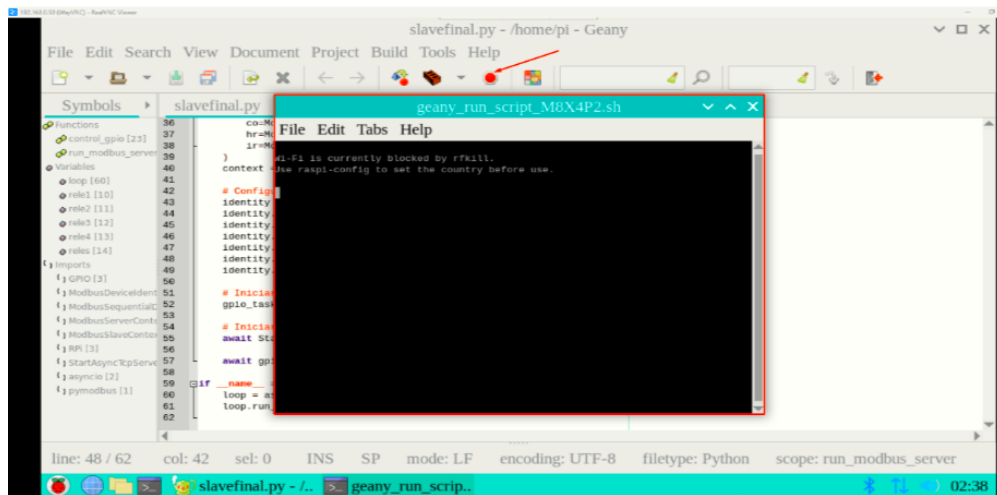


Figura 101: Ejecución de esclavo Modbus en Raspberry PI 4 con Python.

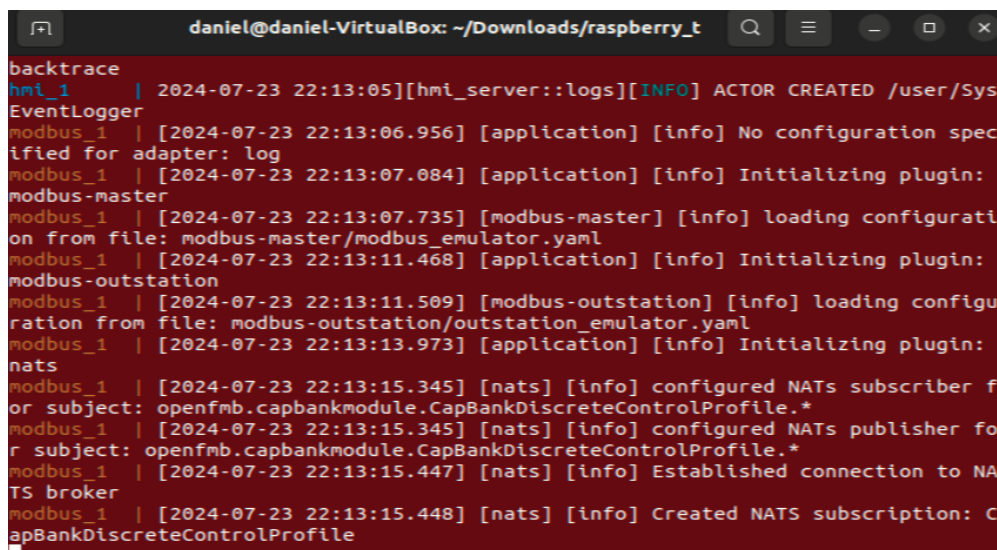


Figura 102: Ejecución de proyecto OpenFMB con control ON/OFF.


```
23 # Dirección IP y puerto del servidor Modbus TCP
24 ip_address = '192.168.0.5'
25 port = 26 # Puerto Modbus TCP
26
27 # Dirección del registro que deseas escribir y leer
28 coil_address1 = 1 # Dirección de la primera bobina (coil)
29 coil_address2 = 2 # Dirección de la primera bobina (coil)
30 coil_address3 = 3 # Dirección de la primera bobina (coil)
31 coil_address4 = 4 # Dirección de la primera bobina (coil)
32
33 # Escribir en la bobina (encender el relé)
34 write_coil(ip_address, port, coil_address1, 1)
35 # Escribir en la bobina (encender el relé)
36 write_coil(ip_address, port, coil_address2, 0)
37 # Escribir en la bobina (encender el relé)
38 write_coil(ip_address, port, coil_address3, 1)
39 write_coil(ip_address, port, coil_address4, 0)
40
41 # Leer el estado de la bobina
42 state1 = read_coil(ip_address, port, coil_address1)
43 print("Estado del relé1:", state1)
44 # Leer el estado de la bobina
45 state2 = read_coil(ip_address, port, coil_address2)
46 print("Estado del relé2:", state2)
47 # Leer el estado de la bobina
48 state3 = read_coil(ip_address, port, coil_address3)
49 print("Estado del relé3:", state3)
50 state4 = read_coil(ip_address, port, coil_address4)
51 print("Estado del relé4:", state4)
52
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
/bin/python3 /home/daniel/Downloads/raspberry_t/pi.py
daniel@daniel-Virtuoso: ~/Downloads/raspberry_t$ /bin/python3 /home/daniel/Downloads/raspberry_t/pi.py
Estado del relé1: True
Estado del relé2: False
Estado del relé3: True
Estado del relé4: False
```

Figura 103: Ejecución de escritura de Coils (1 – 0 – 1 – 0) en Python.

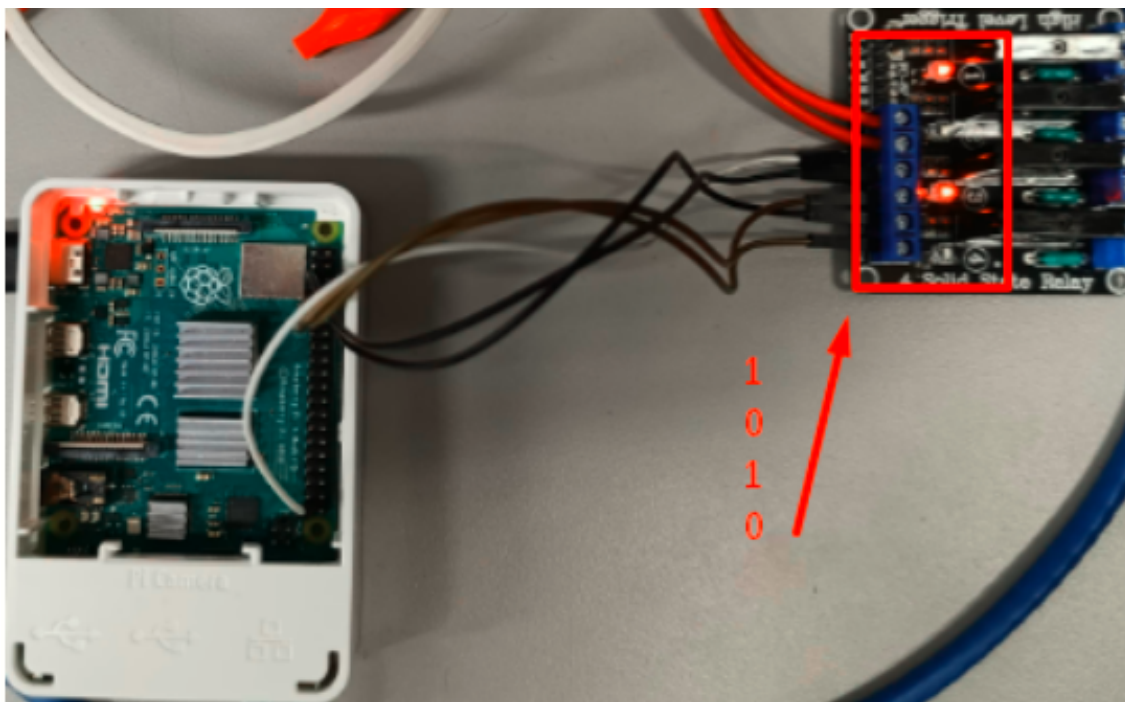


Figura 104: Control de Relés con OpenFMB y Raspberry PI 4.

8. Adaptador OpenFMB como puente entre protocolos NATS y MQTT.

El adaptador de OpenFMB permite hacer la interconexión entre redes con distintos protocolos publicador/subscriptor (por ejemplo, MQTT y NATS). En este modo, el adaptador puede subscribirse a una red NATS y republicar los mensajes recibidos con el mismo t pico en una red diferente MQTT, y viceversa. Para que el contenido de los mensajes sea interpretado correctamente por el adaptador, es necesario que el perfil a publicarse se encuentre serializado. La serializaci n consiste b sicamente en dar un formato especial a la informaci n que portan los mensajes de uno de estos protocolos. Para la serializaci n se usan archivos de extensi n .proto proporcionados por Open Energy Solutions, donde se definen estructuras de serializaci n para los perfiles y sus propiedades. En esta ilustraci n se presenta un esquema para el entorno de pruebas para esta secci n.

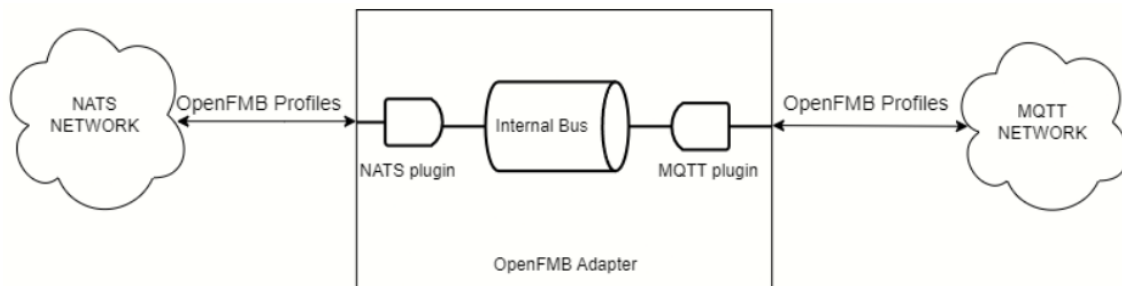


Figura 105: Esquema de OpenFMB interconectando redes con protocolos NATS y MQTT.

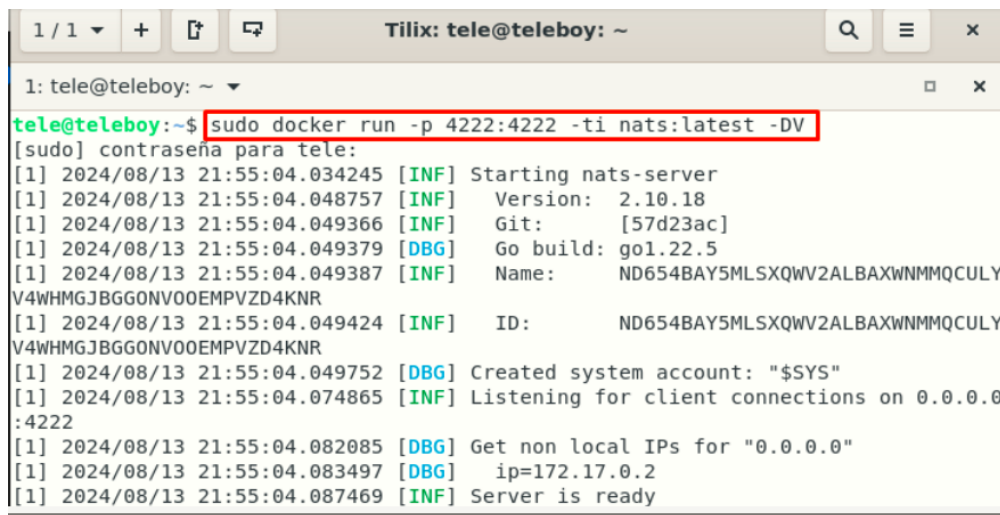
Una instancia de la configuraci n propuesta en la Figura 105 est  conformada por una m quina virtual representando la nube NATS, un sistema operativo anfitri n (Windows) ejecutando el adaptador OpenFMB y otra m quina virtual representando la nube MQTT. Se asume que estos tres elementos se encuentran en el mismo segmento de red (e.g., 192.168.0.0/24). A continuaci n se muestra la configuraci n de cada instancia de la prueba.

8.1. Configuraci n del servicio de mensajes NATS.

Para configurar NATS en la primer m quina virtual Linux, se usa un contenedor de docker ejecutando una imagen de un *broker* NATS. El comando para esa ejecuci n es el siguiente:

- `sudo docker run -p 4222:4222 -ti nats:latest -DV`

En comando anterior el n mero 4222 representa el puerto por el que el *broker* NATS escucha peticiones. La Figura 106 muestra el resultado de la ejecuci n de este comando.



```
1 / 1 + [ ] [ ] Tilix: tele@teleboy: ~
1: tele@teleboy: ~
tele@teleboy:~$ sudo docker run -p 4222:4222 -ti nats:latest -DV
[sudo] contraseña para tele:
[1] 2024/08/13 21:55:04.034245 [INF] Starting nats-server
[1] 2024/08/13 21:55:04.048757 [INF] Version: 2.10.18
[1] 2024/08/13 21:55:04.049366 [INF] Git: [57d23ac]
[1] 2024/08/13 21:55:04.049379 [DBG] Go build: go1.22.5
[1] 2024/08/13 21:55:04.049387 [INF] Name: ND654BAY5MLSXQWV2ALBAXWNMMQCULY
V4WHMGJBGGONV00EMPVZD4KNR
[1] 2024/08/13 21:55:04.049424 [INF] ID: ND654BAY5MLSXQWV2ALBAXWNMMQCULY
V4WHMGJBGGONV00EMPVZD4KNR
[1] 2024/08/13 21:55:04.049752 [DBG] Created system account: "$SYS"
[1] 2024/08/13 21:55:04.074865 [INF] Listening for client connections on 0.0.0.0
:4222
[1] 2024/08/13 21:55:04.082085 [DBG] Get non local IPs for "0.0.0.0"
[1] 2024/08/13 21:55:04.083497 [DBG] ip=172.17.0.2
[1] 2024/08/13 21:55:04.087469 [INF] Server is ready
```

Figura 106: Comando de docker para ejecutar *broker* NATS.

8.2. Configuración del servicio de mensajes MQTT.

Para ejecutar una imagen de un *broker* MQTT (en la segunda máquina virtual Linux) se puede usar Docker Compose, que facilita la inclusión de ficheros de configuración (que son requeridos para el funcionamiento del servidor MQTT, a diferencia de NATS). Estos archivos de configuración permiten establecer un método de autenticación, persistencia de datos, puertos que expone, entre otros. Se recomienda disponer de un directorio que contenga un fichero `docker-compose.yml` y un directorio con las subcarpetas `config`, `data` y `log`. La Figura 107 ilustra esta estructura para un servidor MQTT Mosquitto.

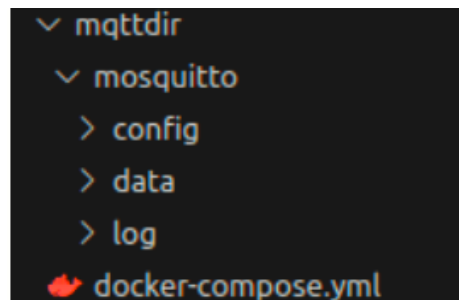
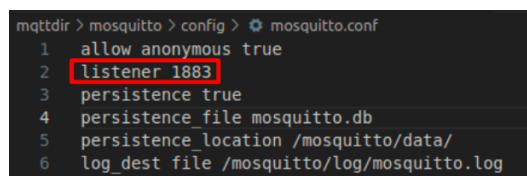


Figura 107: Estructura de directorio para configurar *broker* MQTT.

Para continuar con las configuraciones se agrega un fichero llamado `mosquitto.conf` en el subdirectorio `config` del contenido mostrado en la Figura 108. A resaltar en esta figura es el valor 1883 que representa el puerto por el que el servidor MQTT recibe peticiones nuevas. El último paso es el contenido de `docker-compose.yml` descrito en la Figura 108. En este se asocian los directorios del contenedor con los generados en la Figura 107 para la persistencia de datos.



```
mqtttdir > mosquitto > config > mosquitto.conf
1 allow_anonymous true
2 listener 1883
3 persistence true
4 persistence_file mosquitto.db
5 persistence_location /mosquitto/data/
6 log_dest file /mosquitto/log/mosquitto.log
```

Figura 108: Fichero de configuración de *broker* MQTT.



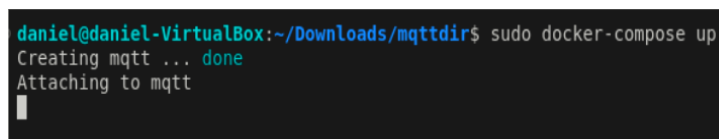
```

1 version: "3.9"
2 services:
3   mosquitto:
4     container_name: mqtt
5     image: eclipse-mosquitto:latest
6     ports:
7       - "1883:1883"
8     volumes:
9       - ./mosquitto/config/mosquitto.conf:/mosquitto/config/mosquitto.conf
10      - ./mosquitto/data:/mosquitto/data
11      - ./mosquitto/log:/mosquitto/log

```

Figura 109: Fichero docker-compose.yml para construir *broker* MQTT.

En la Figura 109 se define el nombre del contenedor con la etiqueta ‘container name’, el nombre de la imagen a ejecutar (MQTT mosquitto) con la etiqueta ‘image’, se indican los puertos con ‘ports’ y las subcarpetas con ‘volumes’. Por último, para ejecutar el *broker* MQTT se abre una terminal en el directorio generado como en la Figura 107, al mismo nivel que el fichero docker-compose.yml, y se ejecuta el comando ‘docker compose up’.



```

daniel@daniel-VirtualBox:~/Downloads/mqttdir$ sudo docker-compose up
Creating mqtt ... done
Attaching to mqtt

```

Figura 110: Comando para lanzar *broker* MQTT con fichero docker-compose.yml.

8.3. Python para implementar publicadores y suscriptores en redes NATS y MQTT.

El lenguaje Python permite el uso de librerías con métodos para generar o recibir mensajes tipo publicador/suscriptor de diferentes protocolos. En este entorno de prueba se proponen diferentes scripts según el protocolo de mensajería a utilizar y la función que se quiere simular (publicación o suscripción).

a) Publicadores.

Un publicador debe establecer conexión con el *broker* indicando el tópico de interés y el mensaje que se envía, en caso de ser publicación. El formato de este tópico está previamente definido por OpenFMB en NATS o MQTT y, está directamente relacionado a los perfiles OpenFMB que se han usado y explicado en este documento, por ejemplo, perfiles de lectura como ‘MeterReadingProfile’ o de escritura como ‘CapBankDiscreteControlProfile’. Adicionalmente, el script de un publicador debe incluir el proceso de serialización mencionado al inicio de esta sección.

* Publicador NATS.

```

nats_pub.py x
natsdir > nats_pub.py > read_data
1  from nats.aio.client import Client as NATS
2  import asyncio
3  import metermodule_pb2
4
5  NATS_SERVER = 'nats://192.168.8.15:4222'
6  NATS_SUBJECT = 'openfmb.metermodule.MeterReadingProfile.56a23a75-d331-412d-0000-690e14671fa5'
7
8  async def read_data(client, nc):
9      while True:
10         msg = input("Ingrese valor a enviar: ")
11         if msg != "salir":
12             message = metermodule_pb2.MeterReadingProfile()
13             message.meterReading.readingMMXU.A.net.cVal.nag = float(msg)
14             message.meter.conductingEquipment.mRID = "56a23a75-d331-412d-0000-690e14671fa5"
15             serialize = message.SerializeToString()
16             print(serialize)
17             await nc.publish(NATS_SUBJECT, serialize)
18         else:
19             print("good-bye")
20
21         await asyncio.sleep(1)
22
23  async def main():
24      client = 0
25
26      nc = NATS()
27      await nc.connect(servers=[NATS_SERVER])
28
29      if nc.is_connected:
30          print("Connected to the NATS server.")
31      else:
32          print("Could not connect to the NATS server.")
33          exit(1)
34
35      try:
36          await read_data(client, nc)
37      finally:
38          await nc.close()
39
40      print("Disconnected from the Modbus and NATS servers.")
41
42  if __name__ == '__main__':
43      asyncio.run(main())

```

Figura 111: Script de Python del publicador NATS.

En el código de la Figura 111 primero se define la dirección IP y el puerto del *broker* NATS. A continuación, se establece el tópico que sigue una jerarquía separada por puntos (.). Este tópico comienza con la palabra 'openfmb', seguida del módulo y uno de sus perfiles que se desea publicar. El último elemento puede ser un mRID específico o el comodín para que el mensaje sea recibido por cualquier suscriptor de ese tópico. Después se implementa una función que mapea una entrada del teclado a un perfil OpenFMB. En esta función se utiliza la librería 'metermodule pb2.py' que contiene las estructuras de datos para instanciar y serializar el perfil 'MeterReadingProfile'. Por último se publica el perfil serializado en el tópico especificado.

Para obtener la librería 'metermodule pb2.py' (necesaria para la serialización de instancias de perfiles OpenFMB) se deben compilar los ficheros 'commonmodule.proto', 'uml.proto' y 'metermodule.proto' que se obtienen en [el repositorio de Open Energy Solutions - OpenFMB](#). Para llevar a cabo este proceso se deben ejecutar estos comandos en Linux.

- apt install -y protobuf-compiler
- pip install protobuf
- protoc --python out=. uml.proto commonmodule.proto metermodule.proto

El primer comando instala el compilador ‘protoc’ con el que se generan librerías de Python a partir de los ficheros .proto. El segundo comando permite instalar la librería de Python para manejar la serialización especificada por los archivos ‘commonmodule.proto’, ‘uml.proto’ y ‘metermodule.proto’. Antes de lanzar el último comando se debe verificar que todos los ficheros .proto se encuentren en el mismo directorio, También se debe verificar que el fichero ‘metermodule.proto’ direcciona correctamente los archivos ‘uml.proto’ y ‘commonmodule.proto’ como se muestra en la Figura 105. El último comando ejecuta la compilación de los ficheros mencionados, es necesario que todos se encuentren en el mismo directorio.

```

protobuf> $ metermodule.proto
6 package metermodule;
7 option go_package = "gitlab.com/openfmb/psm/ops/protobuf/go-openfmb-ops-protobuf/v2/openfmb/metermodule";
8 option java_package = "openfmb.metermodule";
9 option java_multiple_files = true;
10 option csharp_namespace = "openfmb.metermodule";
11
12 import "uml.proto";
13 import "commonmodule.proto";
14
15 // Resource reading value
16 message MeterReading
17 {
18     // UML inherited base object
19     commonmodule.ConductingEquipmentTerminalReading conductingEquipmentTerminalReading = 1 [(uml.option_parent_message) = true];
20     // MISSING DOCUMENTATION!!!
21     commonmodule.PhaseMTN phaseMTN = 2;
22     // MISSING DOCUMENTATION!!!
23     commonmodule.ReadingMMTR readingMMTR = 3;
24     // MISSING DOCUMENTATION!!!
25     commonmodule.ReadingMMXU readingMMXU = 4;
26 }
27
28 // Resource reading profile
29 message MeterReadingProfile
30 {
31     option (uml.option_openfmb_profile) = true;
32     // UML inherited base object
33     commonmodule.ReadingMessageInfo readingMessageInfo = 1 [(uml.option_parent_message) = true];
34     // MISSING DOCUMENTATION!!!
35     commonmodule.Meter meter = 2 [(uml.option_required_field) = true, (uml.option_multiplicity_min) = 1];
36     // MISSING DOCUMENTATION!!!
37     MeterReading meterReading = 3 [(uml.option_required_field) = true, (uml.option_multiplicity_min) = 1];
38 }

```

Figura 112: Direccionamiento de ficheros ‘commonmodule.proto’ y ‘uml.proto’ dentro de ‘metermodule.proto’.

* Publicador MQTT.

```

11 client = mqtt.Client()
12
13 client.on_connect = on_connect
14
15 broker_address = "192.168.0.5"
16 topic = "openfmb/metermodule/MeterReadingProfile/56a23a75-d331-412d-a2da-690e14671fa5"
17 client.connect(broker_address, 1883, 60)
18
19 client.loop_start()
20
21 msg = ''
22 while True:
23
24     msg = input("ingrese valor a enviar: ")
25     if msg != "salir":
26         message = metermodule_pb2.MeterReadingProfile()
27         message.meterReading.readingMMXU.A.net.cVal.mag = float(msg)
28         message.meter.conductingEquipment.mRID = "56a23a75-d331-412d-a2da-690e14671fa5"
29         serialize = message.SerializeToString()
30         print(serialize)
31
32         client.publish(topic, serialize)
33     else:
34         print("good-bye")
35         break
36     time.sleep(1)
37 client.loop_stop()
38 client.disconnect()

```

Figura 113: Script de Python del publicador MQTT.

En el publicador MQTT primero se especifica la IP del servidor y el t3pico cuya jerarqu3a es como la descrita para NATS, en este protocolo el separador es '/'. Para especificar el suscriptor de destino se puede usar un 'mRID' o el comod3n '#' que significa publicaci3n para todos. El siguiente paso es crear una funci3n (similar al publicador NATS) donde se reciben entradas por teclado, esta informaci3n se incluye en una instancia del perfil OpenFMB que pasa a ser serializado y publicado.

VII. Suscriptores.

Un suscriptor se conecta al *broker* para recibir mensajes en un t3pico con el formato definido por OpenFMB en NATS o MQTT. En este script se debe realizar el proceso de deserializaci3n dado el formato con que llegan los perfiles de OpenFMB.

* Suscriptor NATS.

```

subscriberNATS_new.py > main
1  import asyncio
2  from nats.aio.client import Client as NATS
3
4  import metermodule_pb2
5
6  async def main():
7      nc = NATS()
8
9      await nc.connect("nats://192.168.0.15:4222")
10
11      async def message_handler(msg):
12          subject = msg.subject
13          reply = msg.reply
14
15          data = msg.data
16          profilemrp = metermodule_pb2.MeterReadingProfile()
17          profilemrp.ParseFromString(data)
18
19          print(f"Received a message on '{subject}': {profilemrp.meterReading.readingMMXU.A.net.cVal.mag}")
21      await nc.subscribe("openfmb.metermodule.MeterReadingProfile.*", cb=message_handler)
22
23      try:
24          while True:
25              await asyncio.sleep(1)
26      except KeyboardInterrupt:
27          print("Exiting...")
28          await nc.close()
29
30  if __name__ == "__main__":
31      asyncio.run(main())

```

Figura 114: Script de Python del subscriptor NATS.

En el script de la Figura 114 se establece la IP y el puerto del respectivo *broker* NATS, luego se define el tópic de NATS separado por puntos como se explicó anteriormente. Se declara también la función que procesa los mensajes recibidos. En esta función se debe instanciar un perfil OpenFMB con la librería 'metermodule_pb2.py', la misma que permite deserializar un mensaje, como se muestra en las líneas 16 y 18 de la Figura 114.

* Suscriptor MQTT.

```

mqttsubs.py x
mqtttdir > mqttsubs.py > on_connect
1  import paho.mqtt.client as mqtt
2  import metermodule_pb2
3
4  def on_message(client, userdata, message):
5
6      profile = metermodule_pb2.MeterReadingProfile()
7      profile.ParseFromString(message.payload)
8      print(f"Received message: {profile.meter.conductingEquipment.mRID} on topic {message.topic}")
9
10 def on_connect(client, userdata, flags, rc):
11     if rc == 0:
12         print("Connected successfully")
13         client.subscribe("openfmb/metermodule/MeterReadingProfile/56a23a75-d331-412d-0000-690e14671fa5")
14     else:
15         print(f"Failed to connect, return code {rc}")
16
17 client = mqtt.Client()
18
19 client.on_message = on_message
20 client.on_connect = on_connect
21
22 broker_address = "localhost"
23 client.connect(broker_address, 1883, 60)
24
25 client.loop_forever()

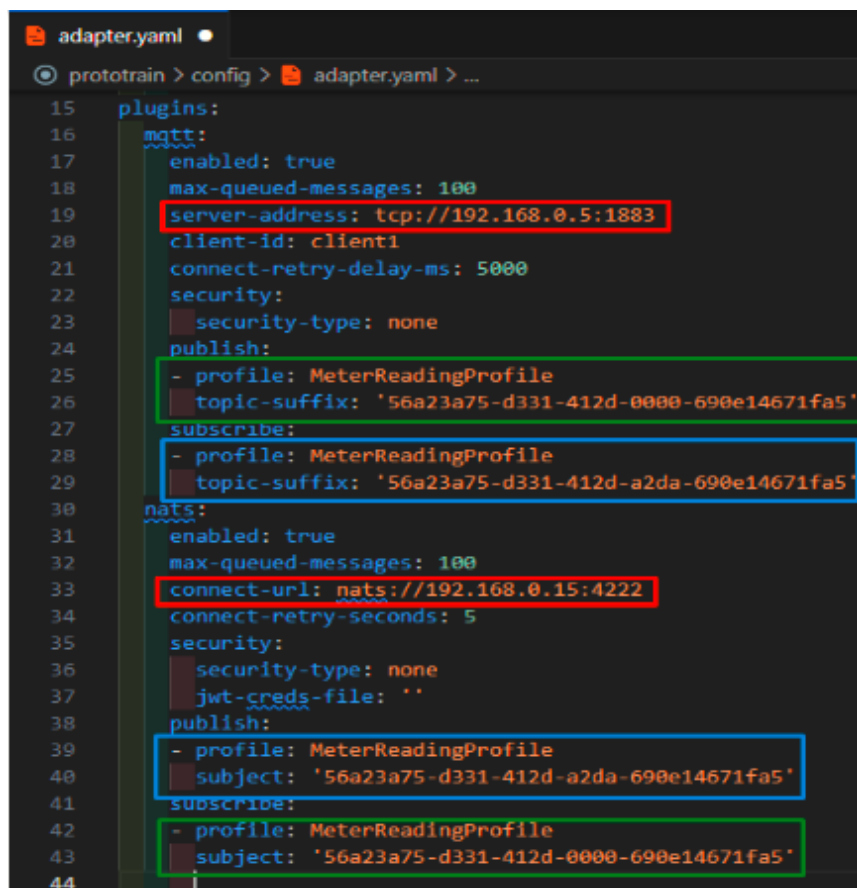
```

Figura 115: Script de Python del subscriptor MQTT.

En el script de la Figura 115 se resalta la sección de conexión con el *broker* MQTT por el puerto 1883. También se define el tópic en MQTT de interés separado por '/', como se explicó anteriormente. Por último se crea la función con la se manejan los mensajes recibidos. En esta función se realiza el proceso de deserialización, como se muestra en las líneas 6 y 7 de la Figura 115.

8.4. Configuración del Adaptador OpenFMB.

Como se mencionó al principio de esta sección, el adaptador OpenFMB puede ejecutarse como puente de interconexión entre redes NATS y MQTT (y otros protocolos publicador/suscriptor). Para esto se requiere establecer conexión con los dos *broker* (uno MQTT y uno NATS) y determinar cuáles de los perfiles OpenFMB se intercambian entre estas redes. Estas puntualidades se ilustran en las Figuras 116 y 117.



```
15 plugins:
16   mqtt:
17     enabled: true
18     max-queued-messages: 100
19     server-address: tcp://192.168.0.5:1883
20     client-id: client1
21     connect-retry-delay-ms: 5000
22     security:
23       security-type: none
24     publish:
25       - profile: MeterReadingProfile
26         topic-suffix: '56a23a75-d331-412d-0000-690e14671fa5'
27     subscribe:
28       - profile: MeterReadingProfile
29         topic-suffix: '56a23a75-d331-412d-a2da-690e14671fa5'
30   nats:
31     enabled: true
32     max-queued-messages: 100
33     connect-url: nats://192.168.0.15:4222
34     connect-retry-seconds: 5
35     security:
36       security-type: none
37       jwt-creds-file: ''
38     publish:
39       - profile: MeterReadingProfile
40         subject: '56a23a75-d331-412d-a2da-690e14671fa5'
41     subscribe:
42       - profile: MeterReadingProfile
43         subject: '56a23a75-d331-412d-0000-690e14671fa5'
44
```

Figura 116: Configuración del adaptador OpenFMB como interconexión entre redes NATS y MQTT.

En la Figura 116 se muestra que el adaptador OpenFMB solo tiene habilitados los plugins **nats** y **mqtt**, los cuales permiten direccionar los *brokers* NATS y MQTT, que permiten el acceso a las redes que se buscan interconectar. Continuando con las configuraciones, se establece la URL de los *broker* en su respectivo plugin (recuadros rojos en Figura 116). Estas direcciones IP con su puerto representan las máquinas virtuales donde se encuentran los *brokers* procesando peticiones.

El último paso es definir los perfiles que se intercambian entre las redes NATS y MQTT. Para ello se debe establecer un perfil y un 'mRID' al cual suscribirse en una de las redes, y se debe establecer este mismo identificador para republicar dicho mensaje en la otra red. En la Figura 116 se muestra que el perfil 'MeterReadingProfile' con el 'mRID' '56a23a75-d331-412d-0000-690e14671fa5' proviene de la red NATS y se republica en la red MQTT (recuadros verdes en Figura 116), mientras que el perfil de lectura con identificador '56a23a75-d331-412d-a2da-690e14671fa5' se recibe desde la red MQTT y se reenvía hacia la red NATS (recuadros azules en Figura 116).

En la Figura 117 se muestra el contenido del fichero docker-compose.yml con el que se puede ejecutar el adaptador OpenFMB configurado como elemento de conexión entre los *brokers* MQTT y NATS. Para ejecutar el adaptador se abre una terminal en el directorio que contiene el fichero adapter.yaml y docker-compose.yml, entonces ejecuta el comando 'sudo docker compose up'. Esta ejecución se puede visualizar en la Figura 118.


```

docker-compose.yml x
prototrain > docker-compose.yml > {} services > {} micontenedor
docker-compose.yml - The Compose specification establishes a standard
1  version: "3.9"
2  services:
3    micontenedor:
4      image: oesinc/openfmb.adapters:a30d1d0
5      volumes:
6        - ./config:/cfg
7      command: -c /cfg/adapter.yaml
8

```

Figura 117: Archivo docker-compose.yml usado para ejecutar adaptador OpenFMB como interconexión entre redes MQTT y NATS.

```

# Usuario @ DESKTOP-4EC3332 in ~\Desktop\prototrain [18:47:50]
$ docker-compose up
time="2024-08-19T18:52:28-05:00" level=warning msg="C:\\Users\\Usuario\\Desktop\\prototrain\\docker-compose.yml: 'version' is
obsolete"
[+] Running 1/0
✓ Container prototrain-micontenedor-1 Created 0.0s
Attaching to micontenedor-1
micontenedor-1 | [2024-08-19 23:52:29.392] [application] [info] No configuration specified for adapter: capture
micontenedor-1 | [2024-08-19 23:52:29.392] [application] [info] No configuration specified for adapter: dnp3-master
micontenedor-1 | [2024-08-19 23:52:29.392] [application] [info] No configuration specified for adapter: dnp3-outstation
micontenedor-1 | [2024-08-19 23:52:29.392] [application] [info] No configuration specified for adapter: log
micontenedor-1 | [2024-08-19 23:52:29.392] [application] [info] No configuration specified for adapter: modbus-master
micontenedor-1 | [2024-08-19 23:52:29.393] [application] [info] No configuration specified for adapter: modbus-outstation
micontenedor-1 | [2024-08-19 23:52:29.393] [application] [info] Initializing plugin: mqtt
micontenedor-1 | [2024-08-19 23:52:29.408] [application] [info] Initializing plugin: nats
micontenedor-1 | [2024-08-19 23:52:29.411] [nats] [info] configured NATS subscriber for subject: openfmb.metermodule.MeterRead
ingProfile.56a23a75-d331-412d-0000-690e14671fa5
micontenedor-1 | [2024-08-19 23:52:29.411] [nats] [info] configured NATS publisher for subject: openfmb.metermodule.MeterRead
ingProfile.56a23a75-d331-412d-a2da-690e14671fa5
micontenedor-1 | [2024-08-19 23:52:29.411] [application] [info] No configuration specified for adapter: replay
micontenedor-1 | [2024-08-19 23:52:29.411] [application] [info] No configuration specified for adapter: timescaledb
micontenedor-1 | [2024-08-19 23:52:29.429] [nats] [info] Established connection to NATS broker
micontenedor-1 | [2024-08-19 23:52:29.431] [nats] [info] Created NATS subscription: MeterReadingProfile
micontenedor-1 | [2024-08-19 23:52:30.421] [mqtt] [info] connected to MQTT server

```

Figura 118: Ejecución del adaptador OpenFMB conectado a redes NATS y MQTT.

8.5. Ejecución del entorno de pruebas completo *broker* NATS + OpenFMB + *broker* MQTT.

Retomando la instancia de pruebas propuesta al inicio de esta sección, se requiere un anfitrión Windows (para ejecutar el adaptador OpenFMB) y dos máquinas virtuales de Linux distintas (para las redes NATS y MQTT).

En la Figura 119 se muestra la estructura de ficheros que contiene cada instancia de esta prueba (el contenido de cada fichero se detalla en las secciones 8.1 a 8.4). La prueba consiste en interconectar dos redes publicador/suscriptor diferentes (NATS y MQTT) por medio de un adaptador OpenFMB, de manera que sea posible la suscripción a mensajes de NATS desde una red MQTT, y en sentido contrario recibir mensajes MQTT en una red NATS.

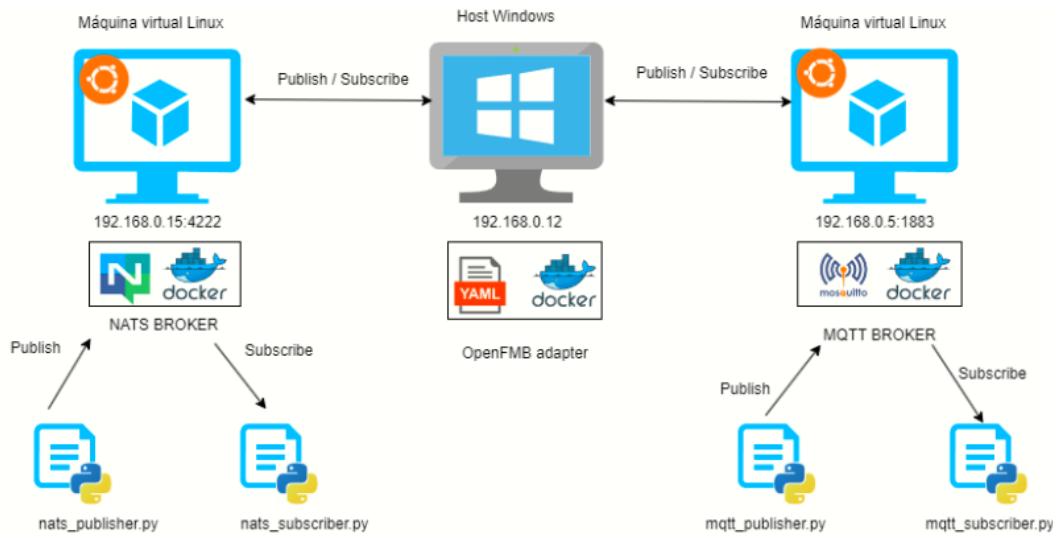


Figura 119: Entorno de prueba de OpenFMB como interconexión de redes NATS y MQTT.

■ Publicación NATS - Suscripción MQTT.

El proceso de publicación en NATS inicia con el fichero 'nats publisher.py', que especifica el tópico del mensaje y el perfil OpenFMB con sus campos. El tópico debe ser coherente con el perfil que se va a publicar, por ejemplo, para publicar un perfil 'MeterReadingProfile' el tópico en NATS es 'openfmb.metermodule.MeterReadingProfile.mRID;', donde 'mRID' corresponde al campo 'MeterReadingProfile.meter.conductingEquipment.mRID' del perfil OpenFMB que se publica. Los demás campos a incluirse en el perfil deben ser asignados antes de serializar y publicar la información al *broker* NATS. Esta estructura está detallada en la Figura 111.

El adaptador OpenFMB se suscribe al tópico de NATS 'MeterReadingProfile' con 'mRID' '56a23a75-d331-412d-0000-690e14671fa5', de manera que repubblica los mensajes recibidos al *broker* MQTT con el tópico 'openfmb/metermodule/MeterReadingProfile/56a23a75-d331-412d-0000-690e14671fa5' y el perfil serializado.

Luego el script 'mRID' define el tópico MQTT que se publicó desde el adaptador OpenFMB y procede a deserializar la información del perfil 'MeterReadingProfile'. Este escenario se puede visualizar en la Figura 120, el recuadro rojo es el publicador NATS, el azul es el suscriptor MQTT y el recuadro verde es el adaptador OpenFMB que funciona como traductor entre los tópicos de ambos protocolos.

■ Proceso de publicación MQTT y Suscripción NATS.

Similar al proceso de publicación en NATS, se inicia con el script 'mqtt publisher.py' donde se establece el tópico de publicación 'openfmb/metermodule/MeterReadingProfile/56a23a75-d331-412d-a2da-690e14671fa5' y se instancia el perfil 'MeterReadingProfile'. Este perfil debe incluir el campo 'MeterReadingProfile.meter.conductingEquipment' (que debe coincidir con el 'mRID' del respectivo tópico) y otras variables pertinentes como se ilustra en la figura 113. Luego se procede a serializar y publicar la información al *broker* MQTT.

El adaptador OpenFMB se suscribe al tópico definido en el script publicador de MQTT. Los mensajes recibidos desde el *broker* MQTT son republicados en NATS con el tópico 'openfmb.metermodule.MeterReadingProfile.56a23a75-d331-412d-a2da-690e14671fa5'. El suscriptor de NATS tiene configurado el tópico con 'mRID' '56a23a75-d331-412d-a2da-690e14671fa5'. Cuando se reciben mensajes desde el *broker* NATS se deserializan utilizando el perfil 'MeterReadingProfile' con lo que concluye el proceso.

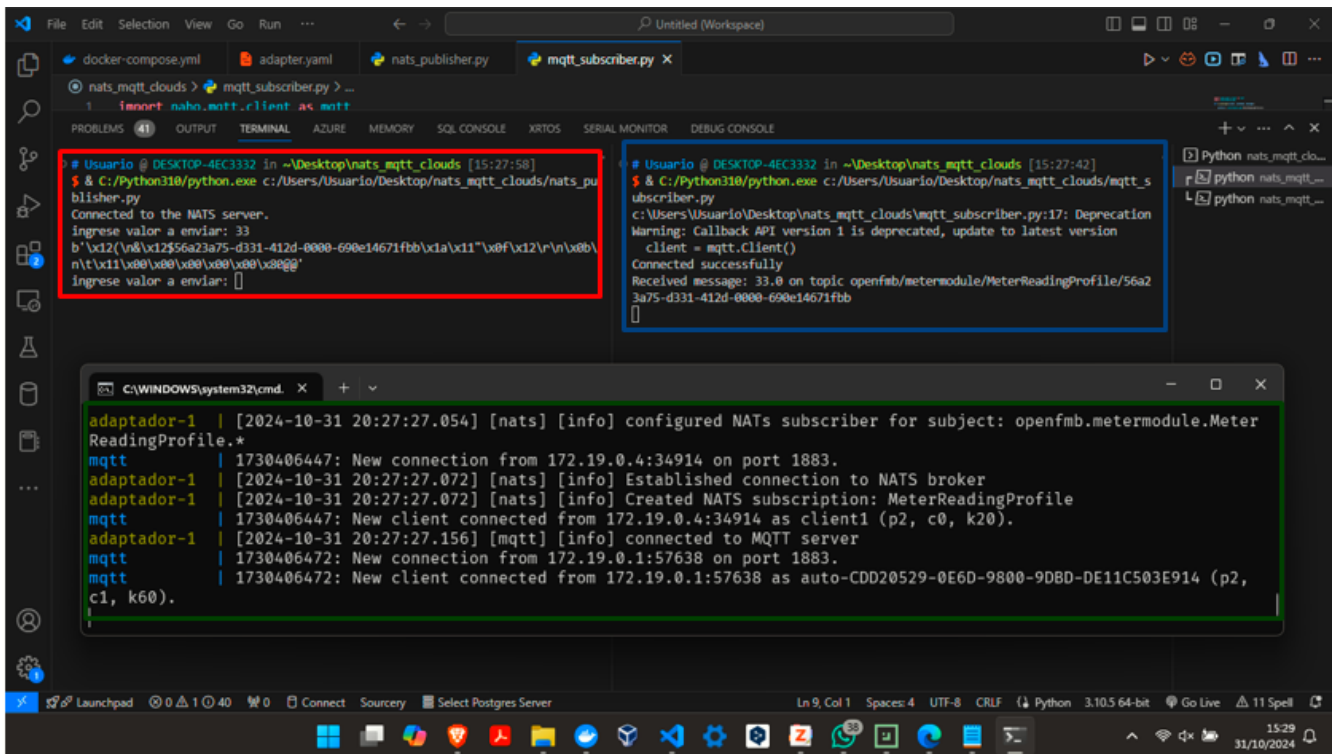


Figura 120: Visualización de la publicación NATS y suscripción MQTT con OpenFMB.

9. Conexión de OpenFMB con base de datos.

El fichero `adapter.yaml` dispone del plugin **timescaledb** con el que puede seleccionar una base de datos para guardar los datos generados por los dispositivos externos durante su funcionamiento. Este plugin debe ser configurado con información como la dirección IP de la base de datos, el usuario, la respectiva contraseña, el nombre de la base de datos, entre otros. En los siguientes pasos se explican estas configuraciones además de crear una base de datos con una tabla incluyendo el almacenamiento de la información recopilada.

9.1. Ejecutar de base de datos timescaledb con Docker Compose.

Se debe poner en ejecución el contenedor de la base de datos que almacena la información recopilada por el adaptador OpenFMB. Para esto se debe describir las propiedades del contenedor `timescaledb` en un fichero `Docker-compose.yml`. En este fichero se establece la ruta a la carpeta donde se almacenan los datos, la contraseña y el puerto para conectarse a la base de datos. El detalle de este fichero se muestra en la Figura 121.

```
timescaledb:
  image: timescale/timescaledb:latest-pg12-oss
  container_name: timescaledb
  ports:
    - "5432:5432"
  environment:
    POSTGRES_PASSWORD: password
  volumes:
    - ./timescaledir:/var/lib/postgresql/data
```

Figura 121: Descripción de fichero `docker-compose.yml` para poner en ejecución `timescaledb`.

En la Figura 121 se muestra que el servicio de la base de datos `timescaledb` requiere de la imagen a utilizar, un

nombre de contenedor, el puerto, una contraseña de acceso y una carpeta de persistencia para almacenar los datos. Esta descripción de la base de datos como servicio de Docker Compose permite su ejecución junto a los demás elementos de OpenFMB, posteriormente se explican estos detalles de integración. Se puede poner en ejecución esta base de datos con el comando ‘docker compose up’.

9.2. Estructura de la tabla SQL empleada para almacenar información de OpenFMB.

El siguiente paso es copiar un script de SQL dentro del contenedor timescaledb para la creación de una nueva tabla. Este script lo proporciona la documentación de OpenFMB en el [siguiente enlace](#). Se recomienda que este script tenga el nombre ‘timescaledb.sql’ ya que se usa un comando de Docker para copiarlo al interior de contenedor timescaledb, en la carpeta ‘/var/lib/postgresql/’. Este comando se ejecuta desde una terminal abierta en el mismo directorio donde se encuentra dicho script de SQL y, se debe tener en cuenta que el contenedor de timescaledb debe estar en ejecución. El comando es el siguiente.

- `docker cp timescaledb.sql timescaledb:/var/lib/postgresql/`

9.3. Configuraciones internas en el contenedor timescaledb.

Las siguientes configuraciones requieren de la ejecución de comandos al interior del contenedor timescaledb. Lo primero es ejecutar este comando para acceder al entorno de directorios de este contenedor.

- `docker exec -it -u postgres timescaledb ash`

Una vez dentro del almacenamiento de timescaledb (el prompt cambia a \$), debe autenticarse con el usuario por defecto para, posteriormente, crear la base de datos y la tabla con el script ‘timescaledb.sql’ copiado anteriormente. Para esto se usa el siguiente comando.

- `psql -U postgres`

Esto cambiará el prompt a ‘postgres=#’. Dentro de este usuario se ejecuta la siguiente secuencia de comandos para crear la base de datos.

- `CREATE DATABASE openfmb;`
- `exit`

Luego de este comando exit, se regresa al almacenamiento del contenedor (prompt en \$), allí, se debe cambiar de directorio a donde se copió el script ‘timescaledb.sql’ (‘/var/lib/postgresql/’).

- `cd /var/lib/postgresql/`

Luego de cambiar de carpeta, se ejecuta el script ‘timescaledb.sql’ con el comando:

- `psql -U postgres -d openfmb -a -f timescaledb.sql`

Luego de este comando, se ha creado una tabla con las características del script .sql dentro de la base de datos creada llamada ‘openfmb’. Luego de este paso se concluye con las configuraciones al interior del contenedor timescaledb.

9.4. Configuraciones del adaptador OpenFMB para conexión con base de datos timescaledb.

Para continuar, se debe configurar un plugin del adaptador de OpenFMB que permite establecer conexión con la base de datos timescaledb que se configuró en la subsección anterior. Como se enunció al inicio de esta sección, se debe especificar la URL, dirección IP, puerto, usuario, contraseña y nombre de base de datos a conectarse. Estos detalles se pueden ver en la figura 114.

```
timescaledb:
  enabled: true
  database-url: postgresql://postgres:password@timesca
  store-measurement: true
  table-name: data
  store-raw-message: false
  raw-table-name: raw_data
  raw-data-format: 0
  max-queued-messages: 100
  connect-retry-seconds: 5
  data-store-interval-seconds: 0
```

Figura 122: Configuración de plugin **timescaledb** en fichero `adapter.yaml`.

Las configuraciones mostradas en la Figura 122 corresponden al plugin **timescaledb** del adaptador OpenFMB. Luego de realizara estas configuraciones la ejecución de la plataforma almacenará la información que entre a la plataforma en la base de datos SQL de manera similar a como se muestra en la Figura 123.

```
adapter-1 | [2025-04-19 20:49:56.173] [timescaledb] [info] Successfully inserted 1 row(s) into data.
adapter-1 | [2025-04-19 20:49:56.174] [modbus-master] [info] Finished transaction: poll sequence
adapter-1 | [2025-04-19 20:49:56.347] [modbus-master] [info] Starting transaction: poll sequence
```

Figura 123: OpenFMB almacenando información en base de datos timescaledb.

10. Visualización de información de base de datos con Grafana.

En esta sección se propone una serie de pasos para conectar el aplicativo de visualiación de información Grafana con la base de datos timescaledb, cuya configuración se explicó en la sección anterior. Este visualizador de datos se puede poner en marcha con Docker Compose, al igual que los elementos de OpenFMB y timescaledb. Los detalles de cada paso se proporcionan a continuación.

10.1. Configuración de Grafana con Docker Compose.

Grafana provee una interfaz web, accesible desde un navegador, que permite visualizar la información de una base de datos. Esta interfaz se despliega en un contenedor que se describe utilizando un fichero `docker-compose.yml`, para hacer compatible la ejecución de esta herramienta con los demás elementos de OpenFMB y timescaledb.

```
grafana:
  image: grafana/grafana
  container_name: grafana
  ports:
    - "3000:3000"
```

Figura 124: Descripción de fichero `docker-compose.yml` para ejecutar Grafana.

En la configuración de la Figura 124 sólo se define la imagen del contenedor a usar, el nombre de contenedor y el puerto de Grafana.

10.2. Acceso a la interfaz web de Grafana.

Con el contenedor de Grafana en ejecución, se puede ingresar a cualquier navegador e ingresar a la dirección 'localhost:3000', se debe llegar a una vista como la Figura 125.

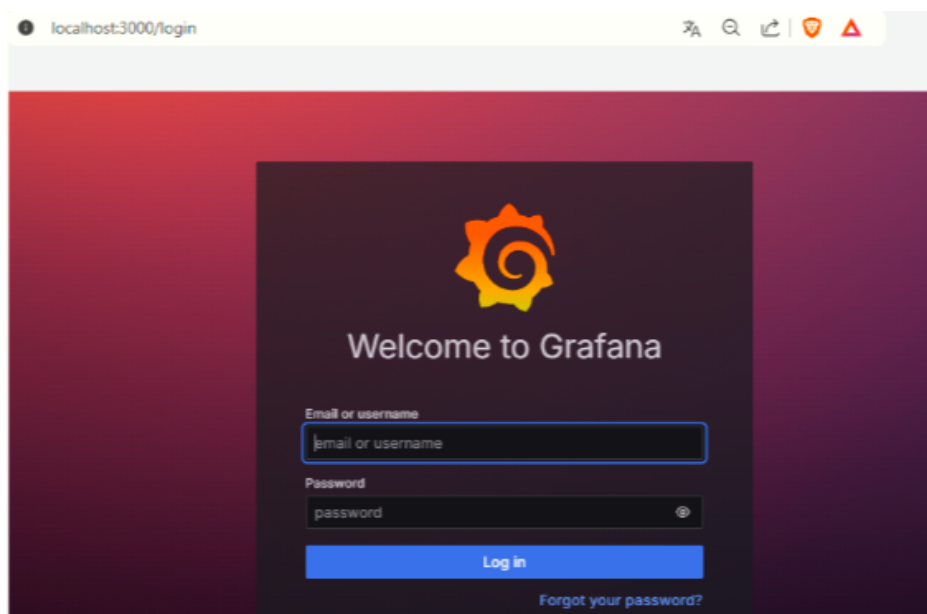


Figura 125: Inicio de sesión en entorno web de Grafana.

Para autenticarse en la interfaz de la Figura 125, se debe escribir 'admin' en los campos de 'Email' y 'Password'. Luego de esto, se pide cambiar la contraseña por seguridad.

10.3. Agregar la conexión de Grafana con timescaledb.

Dentro de la interfaz web de Grafana, se debe agregar una conexión con un nuevo origen de datos, para esto se seleccionan las opciones: 'Connections/Data source/ Add data source', disponibles en el menú lateral izquierdo. En la vista de Grafana - Add data source se busca el aplicativo PostgreSQL y se agrega la nueva conexión con sus respectivos parámetros como se muestra en la Figura 126.

Figura 126: Configuración de conexión de Grafana con base de datos timescaledb parte 1.

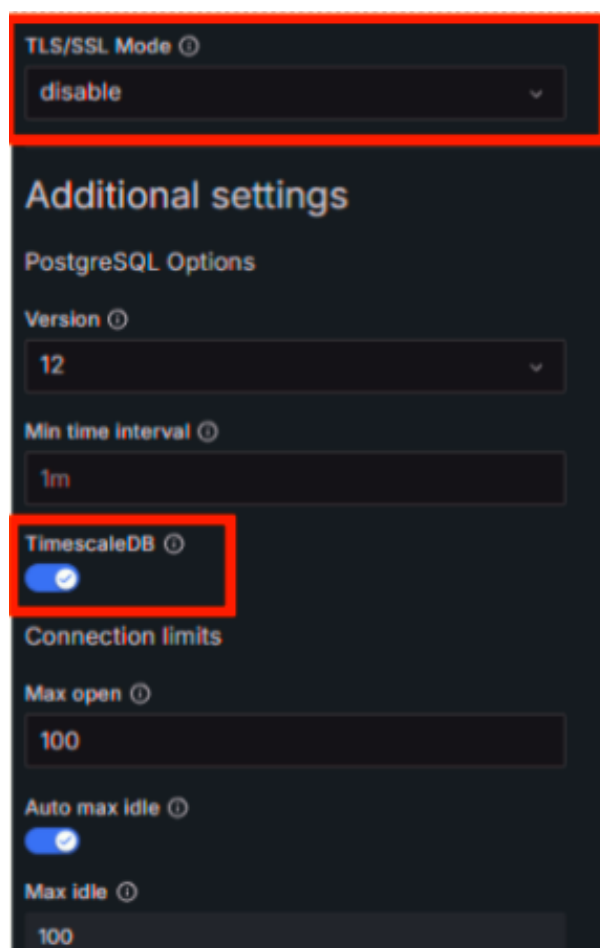


Figura 127: Configuración de conexión de Grafana con base de datos timescaledb parte 2.

10.4. Creación de nuevo dashboard en Grafana.

Primero se accede nuevamente al menú lateral y se seleccionan las siguientes opciones: 'Create dashboard/Add visualization'; luego, en el buscador que aparece, se selecciona la conexión creada en la subsección anterior. La vista debe ser similar a la Figura 128.

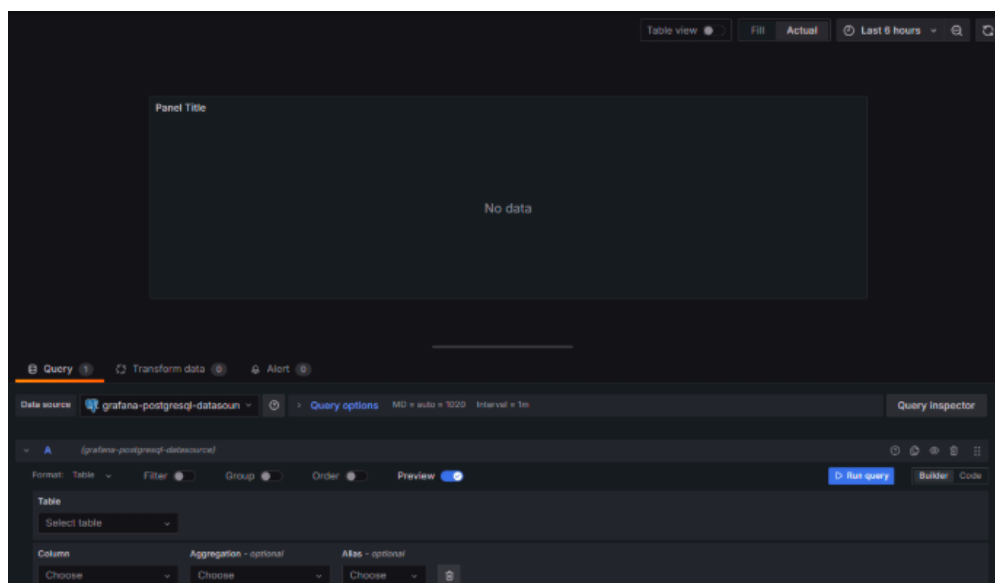


Figura 128: Interfaz para la creación de un nuevo dashboard.

10.5. Selección de columnas de la base de datos a mostrar en Dashboard.

Una vez en el panel de la figura 128, se pueden seleccionar las columnas que se quieren incluir en el gráfico por medio de una consulta SQL. Para esto se da click en la opción **Code** a la derecha de la opción **Run Query**, se escribe la consulta y luego se ejecuta dando click en el botón 'Run query'. Para más detalles ver la figura 129.

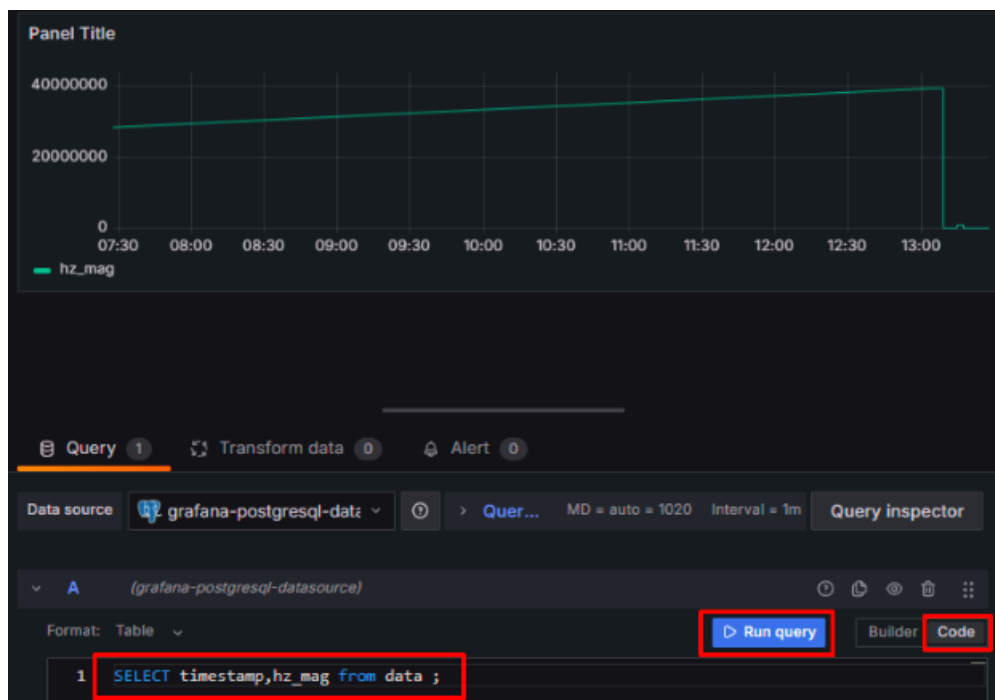


Figura 129: Consulta SQL para graficar variables en dashboard.

Luego de seguir estos pasos, aceptar y guardar cambios, se tendrá un dashboard con una única gráfica (según la consulta SQL utilizada), se pueden repetir los pasos de este apartado para agregar tantos gráficos como sea necesario. Se deja como ejemplo un dashboard construido siguiendo estos pasos.

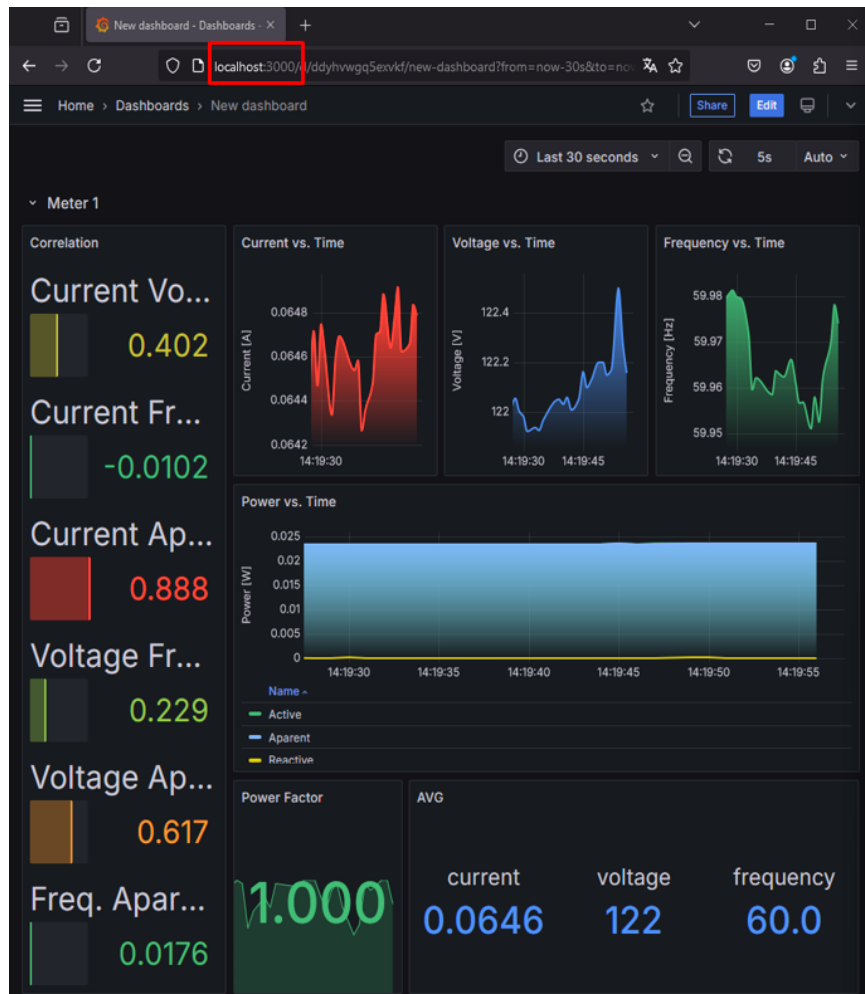


Figura 130: Dashboard de ejemplo con múltiples gráficos en Grafana.

La Figura 130 muestra que Grafana permite crear dashboards con diferentes variables y diferentes tipos de gráficos, donde se puede ajustar la ubicación y tamaño de estos elementos de manera sencilla para lograr la organización deseada según la aplicación.

11. Escritura de dispositivos vinculados a OpenFMB con interfaz gráfica HMI.

En las **Secciones 6 y 7** se explicó la escritura de registros y coils Modbus respectivamente. Allí se muestra el uso de Python y de los plugins **modbus-master** y **modbus-outstation**, sin embargo, es posible emplear la interfaz gráfica HMI y sus diagramas interactivos para lograr este mismo objetivo, es decir, modificar el estado de registros o coils Modbus, pero esta vez por medio del entorno gráfico de OpenFMB.

Esta nueva forma de modificar el estado de los dispositivos es más sencilla de implementar, dado que sólo requiere del plugin **modbus-master** y el HMI, eliminando la necesidad de configuraciones adicionales para el plugin **modbus-outstation** y el uso de Python para generar comandos de escritura Modbus en registros o coils. A continuación se recomienda una serie de pasos para lograr la escritura de registros y coils Modbus, configurando apropiadamente el plugin **modbus-master** y el HMI.

11.1. Escritura de registros Modbus con HMI de OpenFMB.

Inicialmente se requiere la estructura de ficheros y directorios que permite configurar los componentes de la plataforma OpenFMB para lograr la escritura de registros Modbus.

Siguiendo los pasos de las **Sección 6.3** se generan ÚNICAMENTE los ficheros para el adaptador OpenFMB y el plugin **modbus-master** utilizando la herramienta OACT. Además, Para este nuevo método de escritura se va a emplear un perfil de escritura llamado ‘ResourceDiscreteControlProfile’ en el plugin **modbus-master**.

- I. Estructura de ficheros para la escritura de registros con HMI: se requiere un directorio ‘config’ para el fichero de configuración del adaptador OpenFMB; luego se requiere el directorio ‘hmi’, que puede ser la carpeta por defecto del demo de OpenFMB; sigue el directorio ‘modbus-master’ que va a contener el fichero de configuración del plugin **modbus-master**. Finalmente se crea un fichero docker-compose.yml para la ejecución de la plataforma OpenFMB con todos los componentes necesarios. La Figura 131 muestra la estructura de directorios descrita.

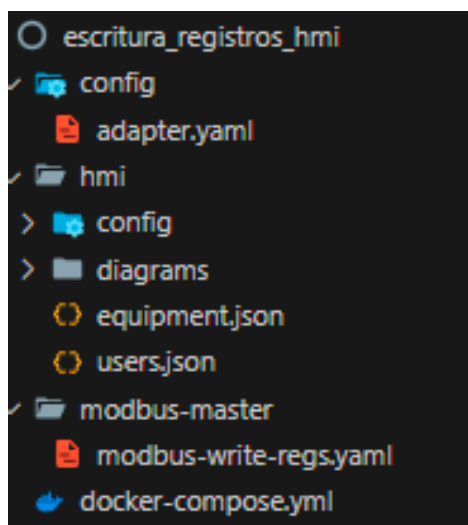


Figura 131: Estructura de directorios para escritura de registros Modbus con HMI.

- II. Uso de Modbus Pal para simular dispositivos conectados a OpenFMB: como se explicó en la **Sección 6.2**, Modbus Pal es un emulador que permite crear esclavos Modbus de manera simple para diferentes pruebas con OpenFMB, por esta razón se implementa la escritura de registros Modbus con este software.

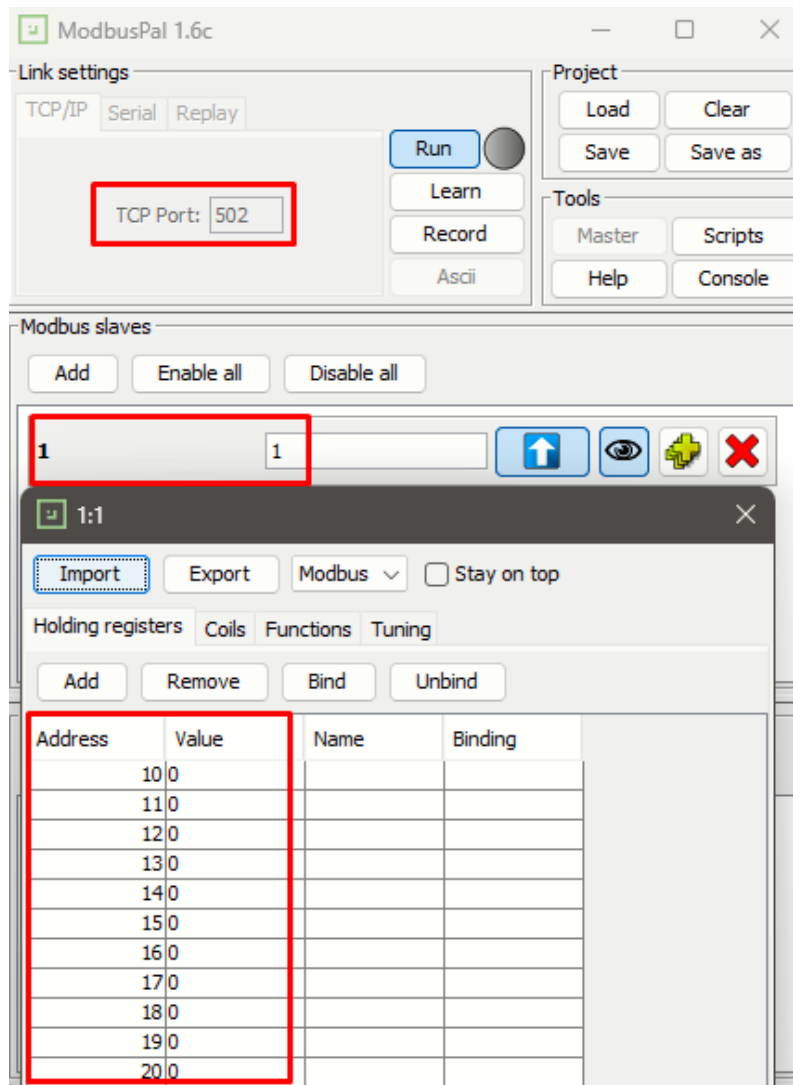


Figura 132: conjunto de registros emulados en esclavo 1 de Modbus Pal.

- III. Fichero del plugin **modbus-master**: recordando la configuración básica que tiene este fichero, tiene 2 secciones: el direccionamiento del equipo físico y el perfil OpenFMB (de lectura o escritura); la primera se configura con IP 'local host', 'port' en 502 (el puerto de Modbus Pal) y 'unit-identifier' en 1.

Para la segunda sección, como se mencionó al inicio de esta sección, se seleccionó el perfil 'ResourceDiscreteControlProfile'. Este perfil de escritura, como se explicó en la **Sección 6.5**, requiere de un identificador 'mRID' e información del registro a modificar. Estos detalles se muestran en la Figura 133.

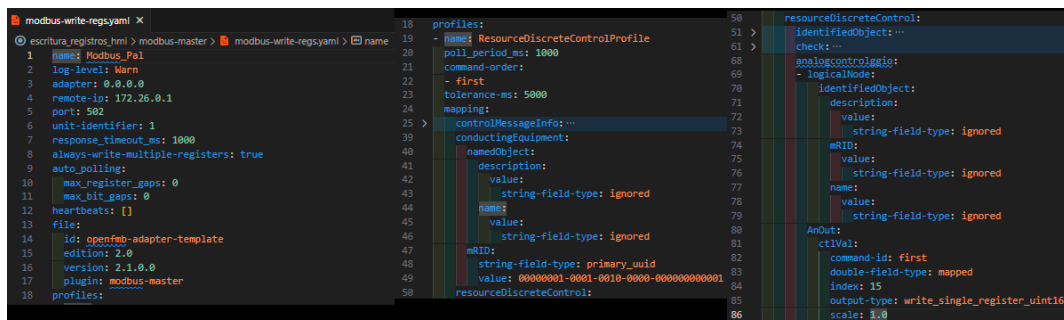
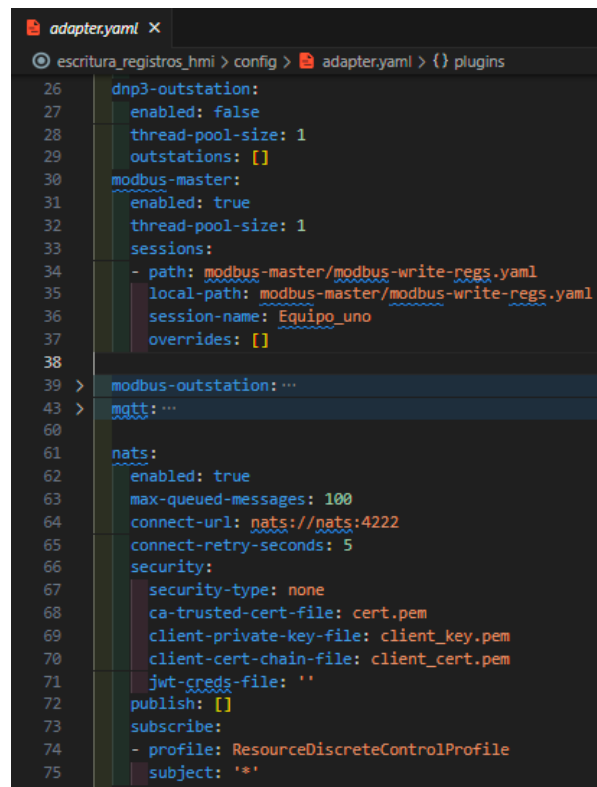


Figura 133: Configuración del plugin **modbus-master** para escritura de registros Modbus.

- IV. Fichero del adaptador OpenFMB: este adaptador debe contener la instancia del fichero del plugin **modbus-master** configurado previamente, además el plugin **nats**, debe estar configurado para suscribirse a tópicos del tema 'ResourceDiscreteControlProfile', lo que le permitirá recibir los comandos de escritura del HMI. Los detalles de la configuración de este adaptador se muestran en la Figura 134.



```
26 dnp3-outstation:
27   enabled: false
28   thread-pool-size: 1
29   outstations: []
30 modbus-master:
31   enabled: true
32   thread-pool-size: 1
33   sessions:
34   - path: modbus-master/modbus-write-regs.yaml
35     local-path: modbus-master/modbus-write-regs.yaml
36     session-name: Equipo_uno
37     overrides: []
38
39 > modbus-outstation:...
43 > mqtt:...
60
61 nats:
62   enabled: true
63   max-queued-messages: 100
64   connect-url: nats://nats:4222
65   connect-retry-seconds: 5
66   security:
67     security-type: none
68     ca-trusted-cert-file: cert.pem
69     client-private-key-file: client_key.pem
70     client-cert-chain-file: client_cert.pem
71   jwt-creds-file: ''
72   publish: []
73   subscribe:
74   - profile: ResourceDiscreteControlProfile
75     subject: '*'
```

Figura 134: Configuración del adaptador OpenFMB para escritura de registros con HMI.

- V. Fichero docker-compose.yml: este fichero define únicamente los servicios de NATS, el adaptador OpenFMB y la interfaz HMI. Cada servicio cuenta con propiedades específicas ya explicadas en otras secciones, que permiten compartir directorios relevantes para su funcionamiento apropiado. El detalle de estas configuraciones se muestra en la Figura 135.

```

docker-compose.yml x
escritura_registros_hmi > docker-compose.yml > {} services > {} hmi
docker-compose.yml - The Compose specification establishes a standard for
1 version: "3.9"
2 services:
3   nats:
4     image: nats
5     ports:
6     - "4222:4222"
7     expose:
8     - 4222
9     adapter:
10    image: oesinc/openfmb.adapters:a30d1d0
11    volumes:
12    - ./config:/cfg
13    - ./modbus-master:/modbus-master
14    command: -c /cfg/adapter.yaml
15  hmi:
16    image: oesinc/openfmb.hmi:v2.1.0
17    volumes:
18    - ./hmi:/cfg
19    environment:
20    APP_CONF: /cfg/config/app.toml
21    APP_DIR_NAME: /cfg
22    ports:
23    - "32771:32771"
24    expose:
25    - 32771

```

Figura 135: Estructura de directorios para escritura de registros Modbus con HMI.

VI. Creación de diagrama en HMI para escritura de registros: luego de editar los ficheros de configuración de los componentes de la plataforma, se pasa a ponerla en ejecución, lo que es necesario para acceder al HMI y continuar con la implementación del diagrama de escritura. En las siguientes Figuras se muestra el ingreso al HMI, el registro del dispositivo con su 'mRID', la creación de un diagrama nuevo con un botón para modificar el registro.

The screenshot shows the HMI web application interface. On the left is a dark sidebar with navigation options: HMI, DIAGRAMS, DATA CONNECTION, SETTINGS, Users, and Devices (highlighted with a red box). The main area shows a list of devices with columns for Name, Meter, Solar, Battery, and Switch. An 'ADD DEVICE' button is at the top. A modal titled 'Add new device' is open, showing fields for Device Name (Modbus Pal - Esclavo), Device Type (resource), and mRID (00000001-0001-0010-0000-000000000001, highlighted with a red box). There are 'Generate New MRID', 'SAVE', and 'Cancel' buttons.

Figura 136: Registro del esclavo de Modbus Pal por medio del 'mRID' asignado desde OpenFMB.

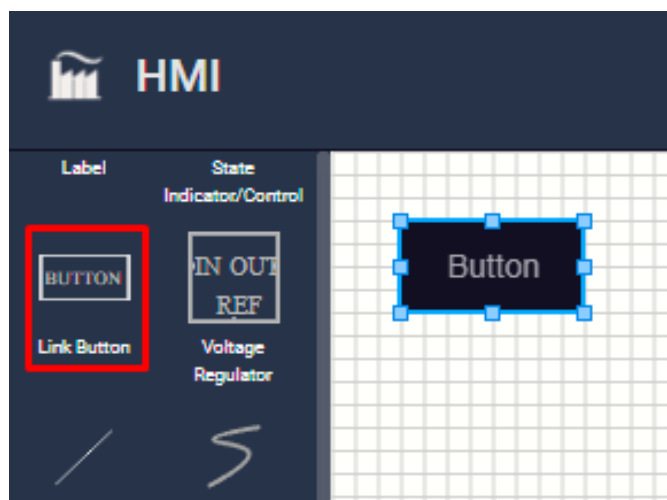


Figura 137: Creación de diagrama con un botón para modificar registros.

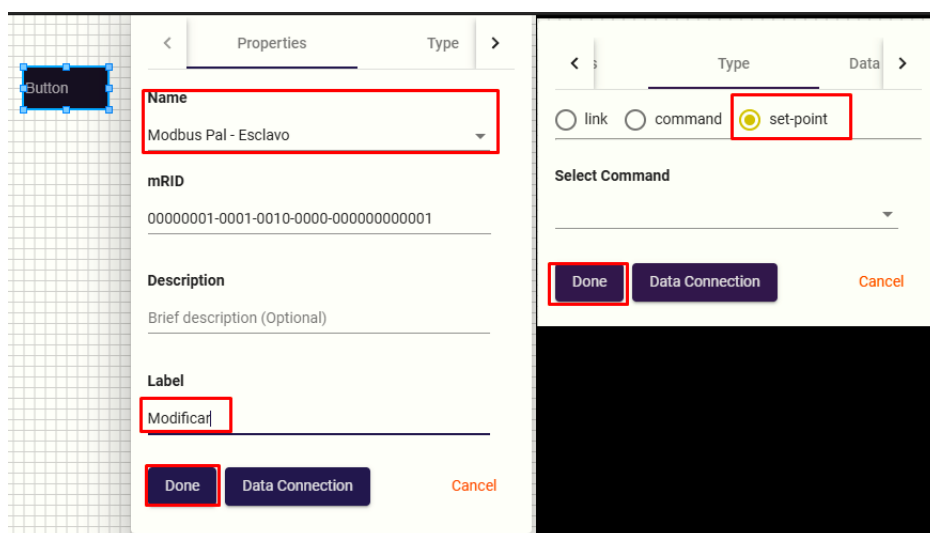


Figura 138: configuración del botón del diagrama para vincularlo al esclavo de Modbus Pal.

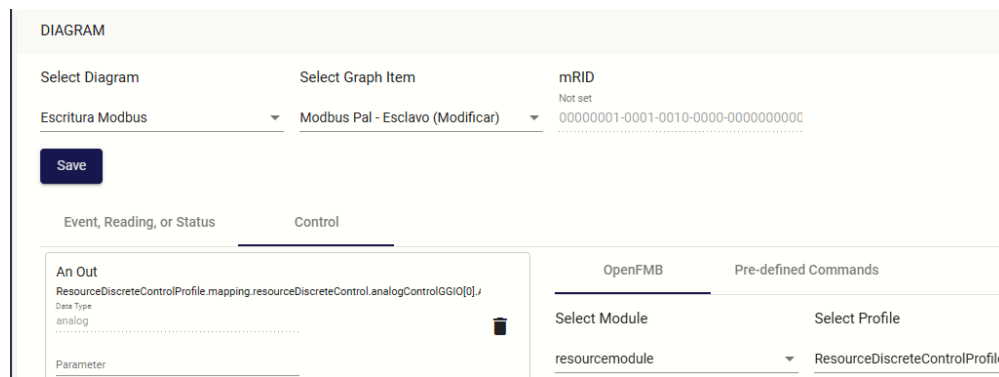


Figura 139: Configuración en 'Data Connections' del botón que vincula el esclavo de Modbus Pal.

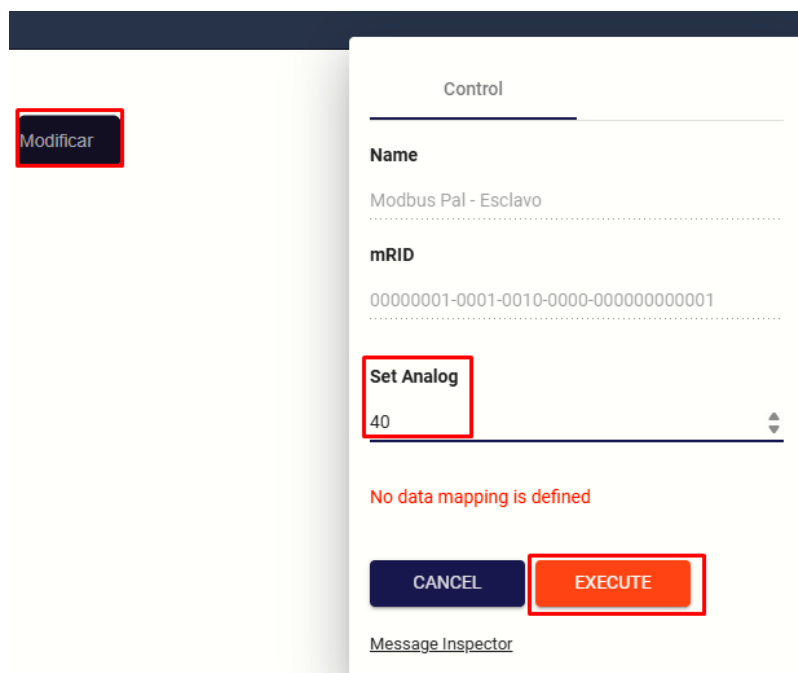


Figura 140: Ejecución del diagrama y uso del botón.

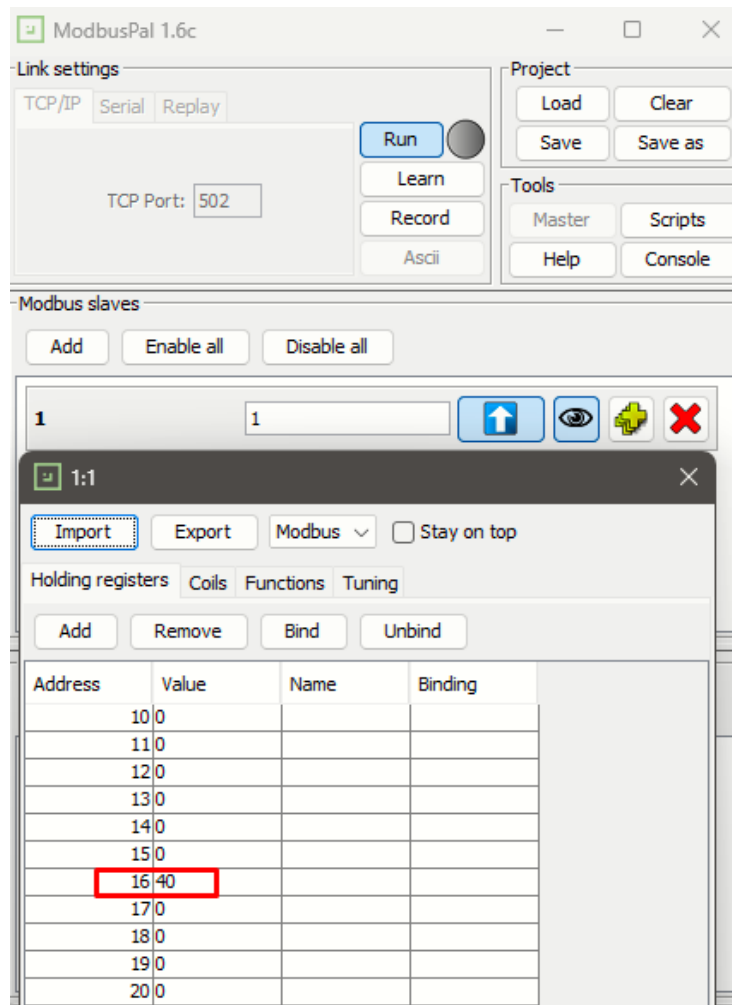


Figura 141: Evidencia de la escritura en el esclavo Modbus Pal.

La Figura 136 muestra la forma de registrar dispositivos nuevos en el HMI para gestionarlos (leer o escribir su información) con diagramas interactivos. Sigue la Figura 137 que muestra un diagrama nuevo con un único elemento, que es el 'Link Button', con el que se va a modificar el registro del esclavo de Modbus Pal. Lo que sigue es configurar el botón, primero enlazando su comportamiento al identificador 'mRID' del esclavo de Modbus Pal y luego, definiendo que el botón será un **set-point** para enviar valores numéricos a este esclavo Modbus, esto último se evidencia en la Figura 138. Continuando con las configuraciones, se pasa a la sección 'Data Connections' del botón (Figura 139), donde se selecciona el perfil que se configuró en el plugin **modbus-master** y luego se agrega al comportamiento de control del botón, la variable que se mapeó en este perfil, dentro del respectivo fichero de configuración (Figura 133). Luego de esto, resta ejecutar el respectivo diagrama y dar doble click sobre el botón configurado y escribir un valor a escribir en el registro. Esto último se muestra en las Figuras 140 y 141.

11.2. Control de coils Modbus con HMI de OpenFMB.

Al igual que con la escritura de registros, se va a generar un conjunto de directorios nuevos que permitan trabajar cómodamente con la plataforma OpenFMB pero esta vez, configurada para lograr la modificación del estado de apertura o cierre de un coil Modbus.

En esta sección se trabaja nuevamente con el adaptador OpenFMB y el plugin **modbus-master**, sin embargo, esta vez se emplean 2 perfiles: 'SwitchStatusProfile' y 'SwitchDiscreteControlProfile'. Esto es porque el HMI cuenta con elementos gráficos que permiten visualizar el estado del coil (perfil de lectura) y al mismo tiempo modificarlo (perfil de escritura). A continuación se detallan las configuraciones para esta nueva prueba de escritura.

- I. Estructura de ficheros para escritura de coils Modbus: al igual que el caso anterior (escritura de registros con HMI), se requieren los directorios ‘config’, ‘modbus-master’, ‘hmi’ (que puede ser del demo) y un fichero docker-compose.yml para la ejecución de todos los servicios de la plataforma.

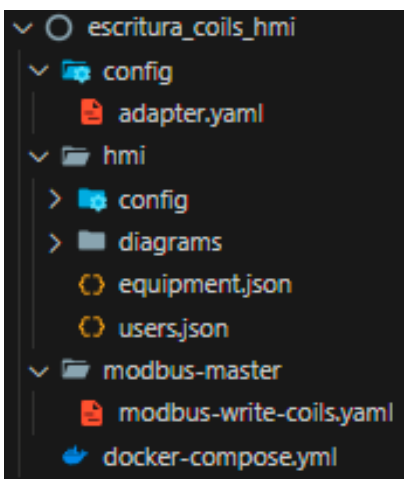


Figura 142: Estructura de directorios para escritura de coils Modbus con HMI.

- II. Uso de Modbus Pal para simular coils Modbus: al igual que en el paso anterior, se emplea Modbus Pal como emulador, pero esta vez, se agregan coils al esclavo 1. Esto se puede observar en la Figura 143.

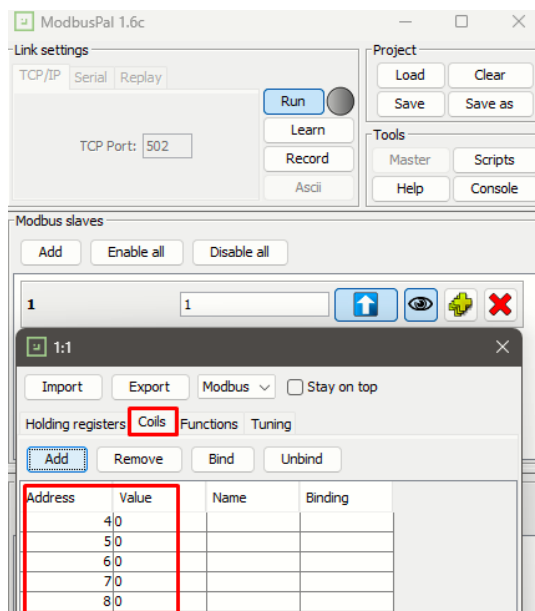


Figura 143: Coils del esclavo de Modbus Pal.

- III. Fichero del plugin **modbus-master**: como en el caso anterior, se definen las propiedades de direccionamiento del esclavo de Modbus Pal. El campo de IP en ‘local host’, ‘port’ en 502 y ‘unit-identifier’ en 1. Luego se configuran los perfiles que permitirán la lectura y escritura de coils en el esclavo Modbus. En este caso, ambos perfiles tendrán el mismo identificador ‘mRID’ ya que se va a emplear un mismo elemento de diagrama para visualizar el estado del coil y cambiarlo. A continuación se muestra la configuración del plugin **modbus-master** para manejar lectura y escritura de un coil Modbus.


```

modbus-write-coils.yaml x
escritura_coils_hmi > modbus-master > modbus-write

1  name: Modbus_Pal_coils
2  log-level: Warn
3  adapter: 0.0.0.0
4  remote-ip: 172.26.0.1
5  port: 502
6  unit-identifier: 1
7  response_timeout_ms: 1000
8  always-write-multiple-registers: true
9  auto_polling:
10   max_register_gaps: 0
11   max_bit_gaps: 0
12  heartbeats: []
13  file:
14   id: openfmb-adapter-template
15   edition: 2.0
16   version: 2.1.0.0
17   plugin: modbus-master
18  profiles:
19   - name: SwitchStatusProfile

```

Figura 144: Configuración del plugin **modbus-master** para escritura de coils (parte 1).

```

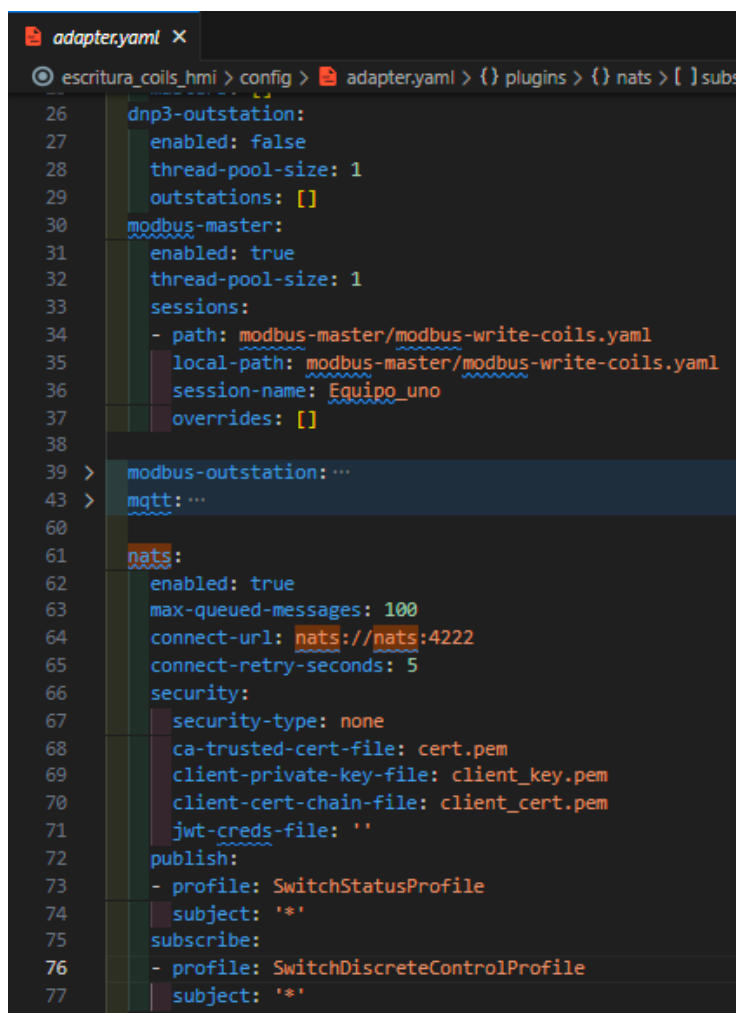
18  profiles:
19   - name: SwitchStatusProfile
20     poll_period_ms: 1000
21     mapping:
22 >    statusMessageInfo: ...
36     protectedSwitch:
37       conductingEquipment:
38         namedObject:
39           description:
40             value:
41               string-field-type: ignored
42             name:
43               value:
44                 string-field-type: ignored
45             mRID:
46               string-field-type: primary_uuid
47               value: 00000006-0006-0060-0000-000000000006
48             switchStatus:
49 >             statusValue: ...
63             switchStatusXSWI:
64 >             logicalNodeForEventAndStatus: ...
103 >             DynamicTest: ...
110             Pos:
111               phs3:
112                 q:
113                   quality-field-type: ignored
114                   stVal:
115                     enum-field-type: mapped
116                     source-type: coil
117                     enum-mapping-type: single_bit
118                     index: 5
119                     when-true: DbPosKind_closed
120                     when-false: DbPosKind_open
121                     t:
122                       timestamp-field-type: ignored
123                   phsA:
124                     q:
174   - name: SwitchDiscreteControlProfile
175     poll_period_ms: 1000
176     command-order:
177     - first
178     tolerance-ms: 5000
179     mapping:
180 >    controlMessageInfo: ...
194     protectedSwitch:
195       conductingEquipment:
196         namedObject:
197           description:
198             value:
199               string-field-type: ignored
200             name:
201               value:
202                 string-field-type: ignored
203             mRID:
204               string-field-type: primary_uuid
205               value: 00000006-0006-0060-0000-000000000006
206             switchDiscreteControl:
207 >             controlValue: ...
224 >             check: ...
231             switchDiscreteControlXSWI:
232 >             logicalNodeForControl: ...
244             Pos:
245               phs3:
246                 ctlVal:
247                   bool-field-type: mapped
248                   when-true:
249                     - output-type: write_single_coil
250                       command-id: first
251                       index: 5
252                       value: true
253                   when-false:
254                     - output-type: write_single_coil
255                       command-id: first
256                       index: 5
257                       value: false
258               phsA:

```

Figura 145: Configuración del plugin **modbus-master** para escritura de coils (parte 2).

IV. Fichero del adaptador OpenFMB: este adaptador a diferencia del anterior, debe definir en su plugin **nats** la publicación de perfiles ‘SwitchStatusProfile’ y la suscripción de perfiles ‘SwitchDiscreteControlProfile’, de esta

manera el adaptador podrá enviar información del estado del coil al HMI, al tiempo que recibe los comandos de modificación desde diagramas de esta interfaz gráfica.



```
26   dnp3-outstation:
27     enabled: false
28     thread-pool-size: 1
29     outstations: []
30   modbus-master:
31     enabled: true
32     thread-pool-size: 1
33     sessions:
34     - path: modbus-master/modbus-write-coils.yaml
35       local-path: modbus-master/modbus-write-coils.yaml
36       session-name: Equipo_uno
37     overrides: []
38
39 >   modbus-outstation: ...
43 >   mqtt: ...
60
61   nats:
62     enabled: true
63     max-queued-messages: 100
64     connect-url: nats://nats:4222
65     connect-retry-seconds: 5
66     security:
67       security-type: none
68       ca-trusted-cert-file: cert.pem
69       client-private-key-file: client_key.pem
70       client-cert-chain-file: client_cert.pem
71       jwt-creds-file: ''
72     publish:
73     - profile: SwitchStatusProfile
74       subject: '*'
75     subscribe:
76     - profile: SwitchDiscreteControlProfile
77       subject: '*'
```

Figura 146: Configuración del adaptador OpenFMB para escritura de coils con HMI.

- V. Fichero docker-compose.yml: este fichero permanece igual que el anterior, es decir, puede contener la misma configuración para los servicios de NATS, el adaptador de OpenFMB y el HMI. Esta información se muestra en la Figura 135.
- VI. Creación de diagrama en HMI para escritura de coil: con la plataforma en ejecución, se ingresa al HMI y primero se debe registrar el nuevo dispositivo, en este caso el identificador 'mRID' es '00000006-0006-0060-0000-000000000006' y el tipo de dispositivo es 'switch'.

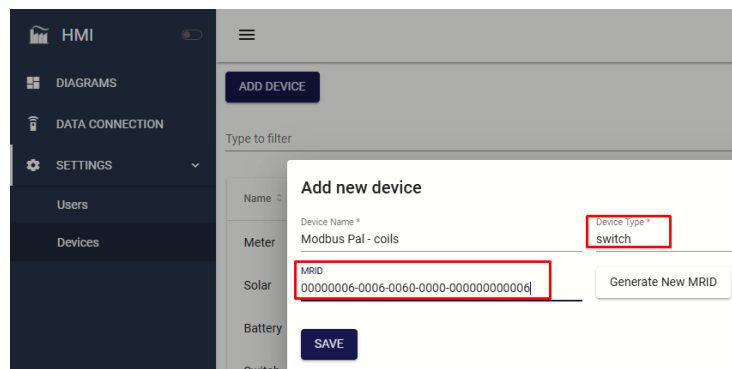


Figura 147: Registro del esclavo de Modbus Pal tipo 'switch'.

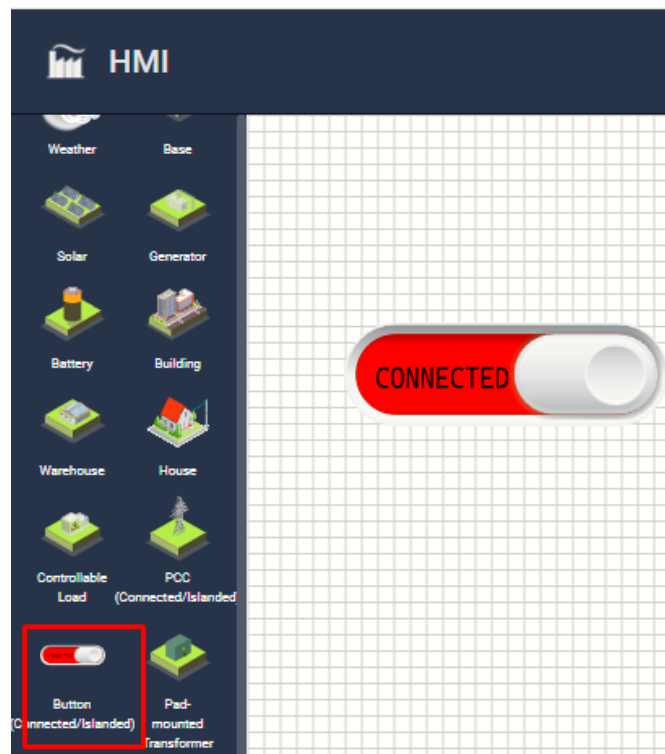


Figura 148: creación de diagrama con botón tipo 'switch'.

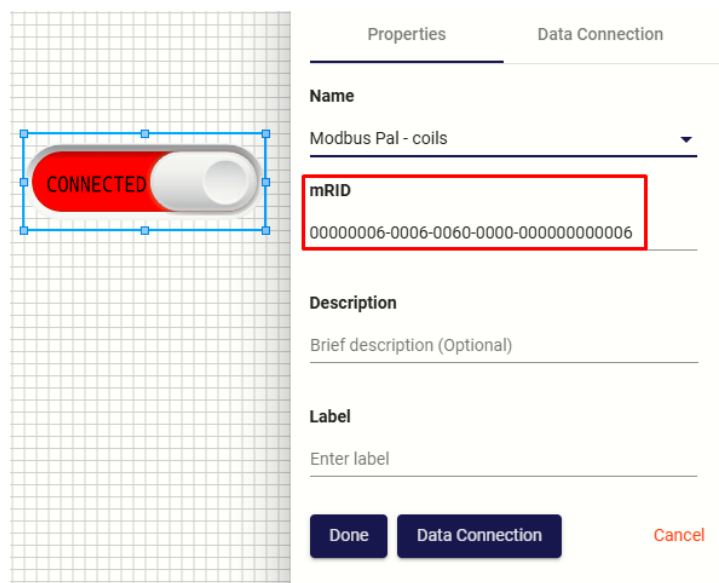


Figura 149: asignación de 'mRID' al botón tipo 'switch'.

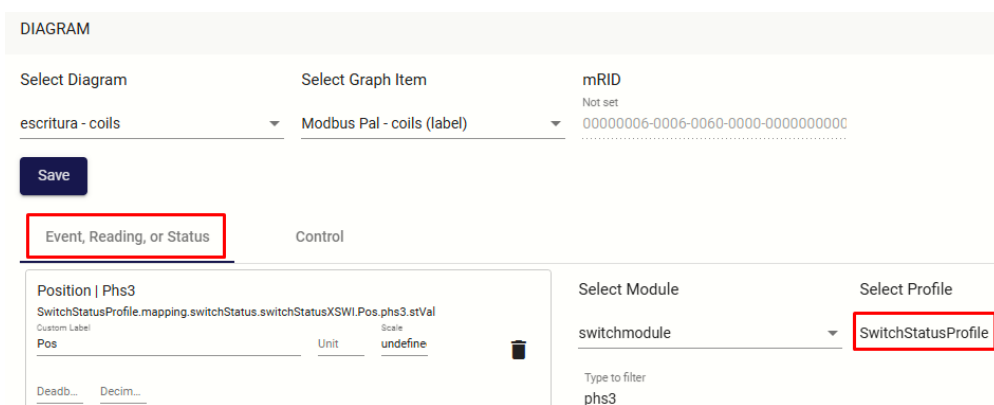


Figura 150: Configuración 'Data Connections' - 'Status' del botón tipo 'switch'.

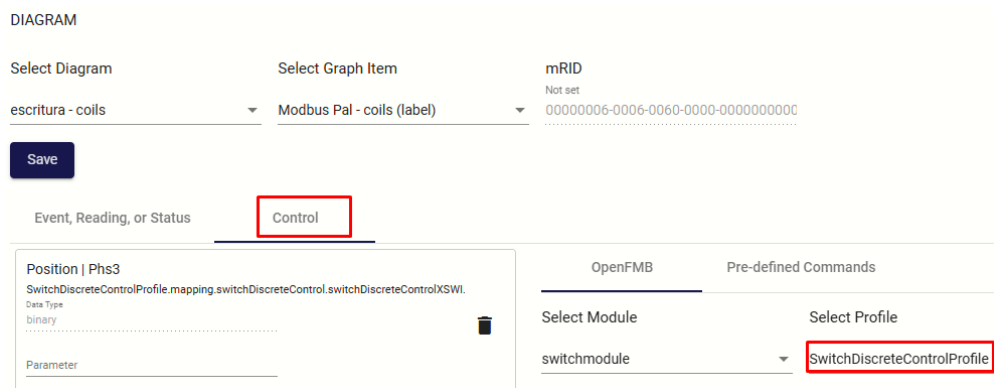


Figura 151: Configuración 'Data Connections' - 'Control' del botón tipo 'switch'.

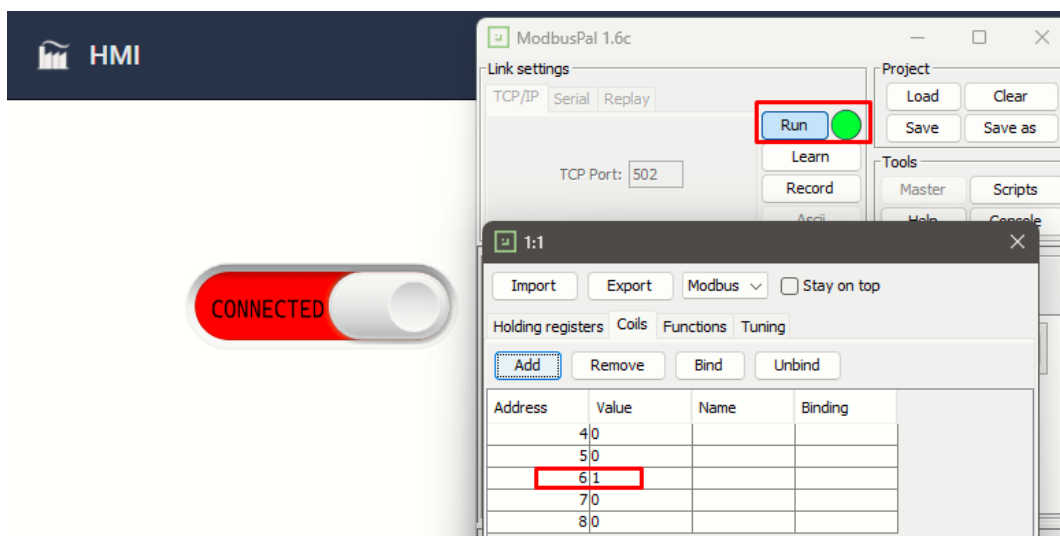


Figura 152: Ejecución de diagrama con ‘Status’ y ‘Control’ sobre el coil 6 de Modbus Pal.

En la Figura 147 se observa el registro del esclavo de Modbus Pal que ahora es tipo ‘switch’ porque sus variables son coils. Posteriormente se crea un diagrama con un nuevo botón que permite capturar el estado del coil leído (rojo es 1 y verde es cero) y permite modificar su estado. Ambas cosas son posibles simultáneamente por las configuraciones de ‘Status’ y ‘Control’ guardadas en ‘Data Connections’ (mostrado en las Figuras 150 y 151). Finalmente la Figura 152 muestra el resultado de ejecutar el diagrama y utilizar el botón.

12. Identificadores ‘mRID’ dentro OpenFMB.

Los ‘mRID’ son secuencias de 32 dígitos hexadecimales, que permiten identificar los perfiles de datos dentro de la plataforma. A su vez, estos perfiles mapean los equipos físicos que se conectan a la microrred (medidores, reles, sensores, inversores de potencia, etc), creando una representación virtual del dispositivo.

De esta manera, un ‘mRID’ proporciona información acerca de su respectivo perfil. Por esto el uso de esquemas de codificación de identificadores pueden estandarizar la forma de asignar identificadores a para mapear la información de los equipos que actualmente se han vinculado a al red y, facilite la integración de nuevos equipos al sistema, siguiendo una regla simple basada en esta codificación de identificadores.